

Heap Thrust Reference

ICPC Reference (Date 0 version)

Team Members:

Enrique Job Calderón (ksobrenat32)
Luis Daniel Salazar (luisdakan)
Gustavo Valenzuela (GusTimeTraveler)

Institution:

Facultad de Ingeniería, UNAM - CPCFI

April 11, 2026

Contents

1 Python stuff	4		
1.1 Regular input	4		
1.2 Fast I/O (If we use this, we're down bad)	4		
1.3 List comprehensions	4		
1.4 Strings	4		
1.5 Regex	5		
1.5.1 Regex notes	5		
1.6 Itertools	6		
1.7 Scripts	7		
1.7.1 Combinatorics calculator	7		
1.7.2 Random array generator	7		
1.7.3 Generate all subsets	7		
2 C++ utils	7		
2.1 Common STL Algorithms	7		
2.2 Random number generator	9		
2.2.1 Graph generator	9		
3 Binary Search in the Answer	9		
4 Bitwise Techniques	10		
4.1 All Subsets	10		
4.2 FWHT	10		
4.3 Gray Code	11		
4.4 XOR Basis	11		
5 Graph Theory	12		
5.1 Bellman-Ford	12		
5.2 Dijkstra	12		
5.3 Dinic	13		
5.4 Directed Cycles	14		
5.5 Edmond-Karp	15		
5.6 Floyd-Warshall	15		
5.7 Kahn Topological Sort (Python)	15		
5.8 Kruskal	16		
5.9 Lava Flow	16		
5.10 Negative Cycles	17		
5.11 Prim Algorithm	18		
5.12 Strongly Connected Components	18		
5.13 Topological Sort	19		
5.14 2-SAT	19		
5.15 Undirected Cycles	20		
5.16 Articulation Points	20		
5.17 Bridges	21		
5.18 Bridges Online	21		
5.19 Bridge Tree	22		
5.20 Dynamic Connectivity	23		
5.21 Eulerian Path	24		
5.22 Gale-Shapley	25		
5.23 Hungarian Algorithm	25		
5.24 SPFA	26		
5.25 Tarjan	26		
6 Data Structures	27		
6.1 2D Segment Tree	27		
6.2 Bit Trie	28		
6.3 Disjoint Set Union	28		
6.4 Fenwick Tree	29		
6.5 GP Hash Table	29		
6.6 Implicit Treap	30		
6.7 Iterative Segment Tree	30		
6.8 Lazy Segment Tree (Max)	31		
6.9 Lazy Segment Tree (Sum)	32		
6.10 Lazy Treap	33		
6.11 Merge Sort Tree	34		
6.12 Ordered Set	35		
6.13 Recursive Segment Tree	35		
6.14 Treap	35		
6.15 Cycle Detection Online	37		
6.16 DSU with Rollbacks	37		
6.17 Fenwick Lazy	37		
6.18 Implicit Treap 2	38		
6.19 Online Bipartiteness	39		
6.20 Path Copy Persistent Array	39		
6.21 Persistent Array	40		
6.22 Persistent Segment Tree	40		
6.23 Prefix Suffix Segment Tree	41		
6.24 Wicho Lazy Segment Tree	41		
6.25 Li Chao Tree	42		
7 Range Queries	43		

7.1	Sparse Table	43	8.33	Sqrt and Inverse Sqrt	56
8	Mathematics	44	8.34	Sum of Divisors	56
8.1	Formula Sheet (Contest)	44	8.35	Ternary Search	56
8.1.1	Combinatorics	44	8.36	Trivial Factorization	56
8.2	Cayley's Formula	44	9	String Algorithms	57
8.3	Geometric series	44	9.1	Aho-Corasick	57
8.4	Divisors	44	9.2	DP on Trie	58
8.5	Number of primes between 1 and n	44	9.3	KMP	58
8.6	Pythagorean triplets	45	9.4	Manacher Algorithm	59
8.7	Derangments	45	9.5	Palindromic Tree	59
8.7.1	Sums	45	9.6	Polynomial Hash	60
8.7.2	Geometry	45	9.7	Rabin-Karp	60
8.7.3	Probability	45	9.8	Suffix Array + LCP	61
8.7.4	Number Theory and Counting Essentials	45	9.9	Suffix Automaton	62
8.8	Binary Exponentiation	46	9.10	Suffix Automata 2	63
8.9	Binomial Coefficients	46	9.11	String Stream	63
8.10	Extended Euclidean	46	9.12	XOR Hash	63
8.11	FFT	46	9.13	Z Algorithm	64
8.12	Big Integer	47	10	Computational Geometry	64
8.13	NTT	48	10.1	Convex Hull	64
8.14	Graph Paths	49	10.2	Complex Point	65
8.15	Graph Paths Min-Plus	49	10.3	Line Segment Intersection	66
8.16	Log Fibonacci	50	10.4	Minimum Euclidean Distance	67
8.17	Matrix Determinant	50	10.5	Minimum Manhattan Distance	67
8.18	Matrix Rank	50	10.6	Point In Polygon	67
8.19	Gauss-Jordan	51	10.7	Point Location Test	68
8.20	Miller-Rabin (Primality Test)	51	10.8	Polar Sort	69
8.21	Linear Sieve	52	10.9	Polygon Area	69
8.22	Euler Totient Function	52	11	Dynamic Programming	70
8.23	Fast Totient Function	52	11.1	Coin Problem	70
8.24	Diophantine Equations	53	11.2	Coin Problem (Min Coins)	70
8.25	Factorials	54	11.3	Coin Problem (Sorted)	70
8.26	Number of Divisors	54	11.4	Knapsack	70
8.27	Modular Inverse	54	11.5	LCS	71
8.28	Discrete Logarithm	54	11.6	LIS	71
8.29	Segmented Sieve	54	11.7	Two Sets	71
8.30	Sieve of Eratosthenes	55	11.8	Bitmask DP	71
8.31	Simpson Method	55	11.9	Digit DP	72
8.32	Precompute Pascal Triangle	55			

11.10	Edit Distance	72	12.7	Heavy-Light Decomposition	77
11.11	Retrieve LIS	73	12.8	Lowest Common Ancestor	78
11.12	SOS DP	73	12.9	Successor DP	78
12	Trees	73	13	Miscellaneous	79
12.1	Diameter	73	13.1	2D Prefix Sum	79
12.2	Euler Tour	74	13.2	Kadane	79
12.3	Tree Isomorphism	75	13.3	Index Compression	79
12.4	Max Distance	75	13.4	Berlekamp-Massey	79
12.5	Binary Lifting	76	13.5	Moore's Voting	80
12.6	Centroid Decomposition	76	13.6	Compiler Optimizations	81

1 Python stuff

1.1 Regular input

```

1 s = input() # This is a string
2 n = int(input()) # This is an integer
3
4 arr = input().split() # This is an array of strings
5
6 # Array of integers (list comprehension)
7 a = [int(x) for x in input().split()]
8 x,y,z = [int(i) for i in input().split()] # Multiple integers

```

1.2 Fast I/O (If we use this, we're down bad)

```

1 import sys
2 input = sys.stdin.readline
3 n = 220
4 sys.stdout.write(str(n) + '\n')

```

1.3 List comprehensions

```

1 # 1 to N
2 nums = [i for i in range(1, N+1)]
3
4 # Filter values
5 evens = [x for x in arr if x%2 == 0]
6
7 # Filter primes
8 primes = [x for x in arr if all(x%d for d in range(2,int(x**0.5)+1))
9           ]
10
11 # Square every element
12 sqr = [x*x for x in arr]
13
14 # Convert string digits to int
15 digits = [int(c) for c in s]
16
17 # Nested loops
18 pairs = [(i,j) for i in range(N) for j in range(M)]
19
20 # Generate NxM matrix
21 matrix = [[i*j for j in range(M)] for i in range(N)]
22
23 # Generate all combinations of two lists
24 combs = [(a,b) for a in list1 for b in list2]

```

```

24
25 # Prefix Sum
26 from itertools import accumulate
27 pref = list(accumulate(arr))
28
29 # Count occurrences of a value (without count)
30 cnt = sum(1 for x in arr if x == target)

```

1.4 Strings

```

1 s = "  Hola reference  "
2 # Remove trailing/whitespaces
3 print(s.strip()) # also .lstrip and .rstrip
4 print(s.split()) # Separates by spaces on list
5
6 s = "hola,reference,icpc,2026"
7 print(s.split(',')) # Separate by character
8
9 strs = ['vamos', 'a', 'ganar', 'icpc']
10
11 # Join a list of strings
12 joined = ' '.join(strs)
13 print(joined)
14
15 print(s.replace('hola','adios')) # Replace substrings
16 s.find("reference") # Finds the first index of the substring
17 s.rfind("reference") # Finds the last index of substring
18
19 s.startswith('hola') # Checks for a prefix
20 s.endswith('2026') # Checks for a suffix
21
22 s.count('icpc') # Counts the number of non overlapping occurrences
23 s.lower() # to lower
24 s.upper() # to upper
25
26 s.capitalize() # Capitalizes the word
27
28 c = 'a'
29 c.isalpha() # Is from alphabet
30
31 c='2'
32 c.isdigit() # Checks if it's a digit
33
34 c = '69'
35 c.isnumeric() # Checks if it is a number
36
37 # Slice a string (substring)
38
39 subs = s[2:5] # Substring from indices 2 to 5
40 rev = s[::-1] # Reverse a substring

```

```

41 step = s[::2] # Transform into every second character
42
43 c = 'z'
44 ord(c) # Unicode code point of the character
45 chr(65) # Reverse of ord

```

1.5 Regex

We must be careful with these functions since they could blow into $O(2^n)$ complexity.

```

1 import re
2
3 # 1. Basic matching
4 re.match(r'foo', 'foobar')           # <re.Match object>
5 re.search(r'bar', 'foobar')          # <re.Match object>
6 re.fullmatch(r'foo', 'foo')          # <re.Match object>
7
8 # 2. Find all / finditer
9 re.findall(r'\d+', 'a12b34')          # ['12', '34']
10 re.finditer(r'\d+', 'a12b34')         # iterator over <Match> objects
11
12 # 3. Replace
13 re.sub(r'\d+', '#', 'a12b34')         # 'a#b#'
14 re.subn(r'\d+', '#', 'a12b34')        # ('a#b#', 2)
15
16 # 4. Split
17 re.split(r'\W+', 'a,b;c')             # ['a', 'b', 'c']
18
19 # 5. Escape a literal
20 re.escape(r'foo.bar*')                # 'foo\.\bar\*'

```

1.5.1 Regex notes

```

1 The "must-know" Python 're' functions (and their runtimes)
2
3 | # / Function / Typical use / Time complexity (average-case) /
4 | ---|-----|-----|-----|-----|-----|-----|-----|
5
6 | 1 | 're.match(pattern, string, flags=0)' | Test if the *start* of
7 |   | 'string' matches 'pattern'. | **O(n)** (linear in the length of
8 |   | the part of the string examined) | **Exponential** if the
9 |   | pattern contains nested back-tracking groups ('.*' etc.) |
10 |   | Most patterns are linear; the exponential blow-up only happens
11 |   | with "catastrophic backtracking" |

```

```

6 | 2 | 're.search(pattern, string, flags=0)' | Find the *first* match
7 |   | anywhere in 'string'. | **O(n)** (linear in the length of the
8 |   | string) | **Exponential** for the same reasons as 'match' | Use
9 |   | when you don't care where the match starts |
10 | 3 | 're.fullmatch(pattern, string, flags=0)' | Require the *entire*
11 |   | string to match. | **O(n)** | **Exponential** for pathological
12 |   | patterns | Equivalent to '^pattern$' but usually faster |
13 | 4 | 're.compile(pattern, flags=0)' | Pre-compile a pattern for
14 |   | reuse. | **O(m)** where *m* is the pattern length | **
15 |   | Exponential** for the same reason as above | Compile once, call
16 |   | many times - huge speed win |
17 | 5 | 're.findall(pattern, string, flags=0)' | Return a list of *all*
18 |   | non-overlapping matches. | **O(n)** | **Exponential** if
19 |   | pattern backtracks | Use when you only need the substrings, not
20 |   | the match objects |
21 | 6 | 're.finditer(pattern, string, flags=0)' | Return an iterator
22 |   | of 'Match' objects (lazy). | **O(n)** | **Exponential** | Prefer
23 |   | for streaming / huge strings |
24 | 7 | 're.split(pattern, string, maxsplit=0, flags=0)' | Split '
25 |   | string' at every match of 'pattern'. | **O(n)** | **Exponential**
26 |   | 'maxsplit' limits the number of splits |
27 | 8 | 're.sub(pattern, repl, string, count=0, flags=0)' | Replace
28 |   | all (or first 'count') matches with 'repl'. | **O(n)** (plus the
29 |   | cost of building the replacement) | **Exponential** if the
30 |   | pattern backtracks | 'repl' may be a string or a callable |
31 | 9 | 're.subn(pattern, repl, string, count=0, flags=0)' | Same as '
32 |   | sub', but also returns the number of substitutions. | **O(n)** |
33 |   | **Exponential** | Useful if you want the replacement count |
34 | 10 | 're.escape(string)' | Escape all regex metacharacters in a
35 |   | literal string. | **O(m)** where *m* is the input length | **
36 |   | Linear** | Handy when you want to use a user-supplied string
37 |   | literally |
38 | 11 | 're.finditer' + 'match.group()' | Extract captured groups
39 |   | lazily. | **O(n)** | **Exponential** | Each group extraction is
40 |   | *O(1)*; the search itself dominates |
41 | 12 | 're.Scanner' (rare) | Perform multiple patterns in a single
42 |   | pass. | **O(n)** | **Exponential** for patterns that backtrack |
43 |   | Good for tokenizers (lexers) |

```

```

17 > **General rule of thumb**
18
19 > *If your pattern does not contain nested quantifiers or back-
20 |   | references, the engine runs in **linear time** in the length of
21 |   | the input string.*
22
23 > *If you do have nested '*', '+', '{m,n}' or back-references, the
24 |   | worst-case becomes **exponential** - this is the "catastrophic
25 |   | backtracking" phenomenon.*

```

Why the exponential blow-up happens

Python's regex engine is a **back-tracking NFA**.

When the engine tries a branch (e.g., a '.' that can match **any** character) it keeps a stack of alternatives. If the branch fails later, it **pops** to the previous branch and tries the **next**

```

alternative.
26 If a pattern has *k* nested alternations, the number of states
    explored can be as high as **2k**.
27
28 Typical examples:
29
30 '''python
31 pattern = r'^(a+)$'          # matches only "a", but the engine
    explores 2^(len(s)) possibilities
32 '''
33
34 > **Tip** - If you ever hit a "too slow" regex, try to rewrite it so
    that the number of back-tracking branches is bounded or replace
    it with a deterministic automaton (e.g., use 're.fullmatch(r'[a
    -z]+)') instead of 're.match(r'.*)').
35
36 ### Pre-compilation matters
37
38 '''python
39 pat = re.compile(r'\bfoo\b')
40 for line in sys.stdin:
41     if pat.search(line):
42         ...
43 '''
44
45 *Compiling once* keeps the **O(m)** cost to a single place, then
    every call to 'search', 'match', or 'sub' runs in *O(n)*.
46
47 ### Quick-reference cheat-sheet
48
49 '''python
50 import re
51
52 # 1. Basic matching
53 re.match(r'foo', 'foobar')      # <re.Match object>
54 re.search(r'bar', 'foobar')     # <re.Match object>
55 re.fullmatch(r'foo', 'foo')     # <re.Match object>
56
57 # 2. Find all / finditer
58 re.findall(r'\d+', 'a12b34')    # ['12', '34']
59 re.finditer(r'\d+', 'a12b34')   # iterator over <Match> objects
60
61 # 3. Replace
62 re.sub(r'\d+', '#', 'a12b34')   # 'a#b#'
63 re.subn(r'\d+', '#', 'a12b34')  # ('a#b#', 2)
64
65 # 4. Split
66 re.split(r'\W+', 'a,b;c')       # ['a', 'b', 'c']
67
68 # 5. Escape a literal
69 re.escape(r'foo.bar*')          # 'foo\.\bar\|*'
70
71 # 6. Pre-compile

```

```

72 pat = re.compile(r'\bfoo\b')
73 for line in sys.stdin:
74     if pat.search(line):
75         ...
76
77 ### Summary of complexities
78
79 | Function | Avg. time | Worst-case | Typical pattern | When you
    should be careful |
80 |-----|-----|-----|-----|-----|
81
82 | 'match', 'search', 'fullmatch' | O(n) | 2^k (k = nested
    quantifiers) | Simple anchors, fixed literals | Patterns with
    '.*', '.*+', or back-references |
83 | 'findall', 'finditer' | O(n) | 2^k | Non-overlapping matches |
    Same as above |
84 | 'split' | O(n) | 2^k | Separator patterns | Avoid '.*?' inside the
    separator |
85 | 'sub', 'subn' | O(n) | 2^k | Replacement string or callable |
    Replacement that triggers a new search inside the replacement
    can amplify the cost |
86 | 'escape' | O(m) | O(m) | Any literal | None |
87 | 'compile' | O(m) | 2^k (in compile) | Pattern parsing | Pre-
    compile once per program run |
88
89 > **Bottom line** - For the vast majority of CP problems you will
    never hit the exponential worst-case. Just avoid nested
    quantifiers or back-references, and you're safely in linear-time
    territory. When you do need them, test the regex on a few
    worst-case inputs to be sure it runs in time.

```

1.6 Itertools

```

1 import itertools
2
3 a = [1,2,3,4]
4 b = [5,6,7,8]
5
6 # All 2 digit numbers composed of digits 1,2,3
7 # Cartesian product
8 for a, b in itertools.product('123', repeat=2):
9     print(a+b)
10
11 # All k-subsets (combinations)
12 for team in itertools.combinations(['A','B','C','D','E'],3):
13     print(team)
14
15 # Subsets with repetitions (combinations_with_replacement)

```

```

16 # All ways to choose 3 balls from 2 colours where each colour can be
    used any number of times
17 for bag in itertools.combinations_with_replacement(['red','blue'],3)
    :
18     print(bag)
19
20 # All ways to order 3 letters O(n!)
21 for order in itertools.permutations('abc'):
22     print(''.join(order))
23
24 # Prefix XOR
25 pref_xor = list(itertools.accumulate(a, func=lambda x,y : x^y))
26
27 # Group consecutive equal keys
28
29 s = "AAABBBCCDA"
30 for key, group in itertools.groupby(s):
31     print(key, list(group))
32 # A ['A','A','A']
33 # B ['B','B','B']
34 # C ['C','C']
35 # D ['D']
36 # A ['A']

```

```

23 factorial()
24 print(comb(a,b))

```

1.7.2 Random array generator

```

1 import random
2
3 with open("in.txt", 'w') as f:
4     n = random.randint(1,100000)
5     f.write(str(n)+'\n')
6     for i in range(n):
7         f.write(str(random.randint(1,100))+ ' ')

```

1.7.3 Generate all subsets

```

1 k = int(input())
2 S = [int(x) for x in input().split()]
3 subsets = [[S[j] for j in range(k) if (i >> j) & 1] for i in range(1
    << k)]
4 print(subsets)

```

1.7 Scripts

1.7.1 Combinatorics calculator

```

1 a,b = [int(x) for x in input().split()]
2
3 MAX = 1000001
4 MOD = 1000000007
5
6 fact = [0] * MAX
7 i_fact = [0] * MAX
8
9 def inv(x : int):
10     return pow(x, MOD-2,MOD)
11
12 def factorial():
13     fact[0] = fact[1] = 1
14     for i in range(2,MAX):
15         fact[i] = (fact[i-1] * i) % MOD
16     i_fact[MAX-1] = inv(fact[MAX-1])
17     for i in range(MAX-2, 0, -1):
18         i_fact[i] = (i_fact[i+1]*(i+1))%MOD
19
20 def comb(n : int, k: int):
21     return fact[n] * i_fact[n-k]%MOD * i_fact[k] % MOD
22

```

2 C++ utils

2.1 Common STL Algorithms

Sorting Algorithms

Function	Parameters	Description
sort	begin, end, [comp]	Standard unstable sort (O(n log n))
stable_sort	begin, end, [comp]	Stable sort preserves element order
is_sorted	begin, end, [comp]	Checks if range is sorted (returns bool)
nth_element	begin, nth, end, [comp]	Partitions around nth element

Searching Functions

Function	Parameters	Description
lower_bound	begin, end, val, [comp]	First element $\leq$ value
upper_bound	begin, end, val, [comp]	First element $>$ value
binary_search	begin, end, val, [comp]	Existence check in sorted range
find	begin, end, val	Linear search for value
find_if	begin, end, pred	Find first matching predicate

Sequence Operations

Function	Parameters	Description
reverse	begin, end	Reverse elements in-place
rotate	begin, mid, end	Rotate elements left
next_permutation	begin, end	Generate next permutation
unique	begin, end, [pred]	Remove consecutive duplicates
remove	begin, end, val	Remove elements equal to value

Numerical Functions

Function	Parameters	Description
accumulate	begin, end, init, [op]	Sum/accumulate elements
partial_sum	begin, end, dest, [op]	Compute prefix sums
__gcd	a, b	Greatest common divisor (C++17)
lcm	a, b	Least common multiple (C++17)
iota	begin, end, val	Fill with consecutive values

Memory/Array Operations

Function	Parameters	Description
memset	ptr, value, count	Fill memory with byte value
fill	begin, end, value	Fill range with value
fill_n	begin, count, value	Fill N elements with value
copy	src_b, src_e, dest	Copy range to destination
copy_if	src_b, src_e, dest, pred	Copy elements matching predicate

Utility Functions

Function	Parameters	Description
swap	a, b	Swap two values
max_element	begin, end, [comp]	Find maximum element
min_element	begin, end, [comp]	Find minimum element
count	begin, end, val	Count element occurrences
all_of	begin, end, pred	Check all elements satisfy condition

Mathematical / Bitwise Builtins

Function	Parameters	Description
__builtin_popcount	x (int)	Count number of 1-bits
__builtin_popcountll	x (long long)	Count number of 1-bits (64-bit)
__builtin_clz	x (unsigned int)	Count leading zeros
__builtin_clzll	x (unsigned long long)	Leading zeros (64-bit)
__builtin_ctz	x (unsigned int)	Count trailing zeros
__builtin_ctzll	x (unsigned long long)	Trailing zeros (64-bit)
__builtin_parity	x	Return 1 if #bits is odd
__builtin_ffs	x	Position of least significant 1-bit (1-indexed)
__lg	x	Floor of $\log_2(x)$ (index of highest bit)

Priority Queues and Heaps

Function	Parameters	Description
priority_queue	[type], [container], [comp]	Max-heap by default
make_heap	begin, end, [comp]	Turn range into heap
push_heap	begin, end, [comp]	Push element into heap
pop_heap	begin, end, [comp]	Pop max element into end
sort_heap	begin, end, [comp]	Heap sort

Set / Map Utilities

Operation	Usage	Description
s.lower_bound(x)	set/map	First element $\geq x$
s.upper_bound(x)	set/map	First element $> x$
s.equal_range(x)	multiset/map	Pair of lower/upper bound
s.erase(it)	iterator	Erase element at iterator
s.find(x)	key	Iterator to key or end

String Functions

Function	Parameters	Description
stoi, stol, stoll	string, [pos], [base]	Convert string $\rightarrow$ int/long/ll
stoul, stoull	string, [pos], [base]	Convert string $\rightarrow$ unsigned
stod, stof, stold	string	Convert string $\rightarrow$ double/float/long double
to_string	value	Convert number $\rightarrow$ string
substr	pos, len	Substring
find	str, pos	Find first occurrence
rfind	str, pos	Find last occurrence

Random Number Utilities

Type / Function	Usage	Description
mt19937 rng	chrono::steady_clock::now()	Fast random generator
uniform_int_distribution	dist(a,b)(rng)	Random int in [a,b]
shuffle	begin, end, rng	Random shuffle

Other Useful Utilities

Function	Parameters	Description
chrono::high_resolution_clock	now()	Get precise current time
_int128	value	128-bit integer (manual I/O needed)
bitset $< N >$	ops: $\&$, $\mid$, $\&$, $\ll$, $\gg$	Fixed-size bitset manipulation
tuple	get $< i >$ (t)	Store and access heterogeneous data
pair	first, second	Store pair of values

2.2 Random number generator

```
1 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
```

2.2.1 Graph generator

```
1 import random
2 n,m = [int(x) for x in input().split()]
3
4 class Edge:
5     def __init__(self, u:int, v:int):
6         self.u = u
7         self.v = v
8
9     def __eq__(self, o):
10         return o.u == self.u and o.v == self.v
11
12     def __hash__(self):
13         return hash((self.u, self.v))
14
15
16 with open("in.txt",'w') as f:
17     if m != n-1:
18         f.write(str(n)+" "+str(m)+'\n')
19     else:
20         f.write(str(n)+'\n')
21     edges = set()
22     for i in range(m):
23         u,v = [0,0]
24         while u == v or Edge(u,v) in edges:
25             u = random.randint(1,n)
26             v = random.randint(1,n)
27         edges.add(Edge(u,v))
28         f.write(str(u)+" "+str(v)+'\n')
29 print(edges)
```

3 Binary Search in the Answer

```
1 // Standard binary search (iterative)
2 int binary_search(vector<int>& arr, int target) {
3     int left = 0, right = arr.size() - 1;
4     while (left <= right) {
5         int mid = left + (right - left) / 2;
6         if (arr[mid] == target) return mid;
7         if (arr[mid] < target) left = mid + 1;
8         else right = mid - 1;
9     }
10 }
```

```

9     }
10    return -1;
11 }
12
13 // Lower bound (first element >= target)
14 int lower_bound(vector<int>& arr, int target) {
15     int left = 0, right = arr.size();
16     while (left < right) {
17         int mid = left + (right - left) / 2;
18         arr[mid] < target ? left = mid + 1
19                          : right = mid;
20     }
21     return left;
22 }
23
24 // Upper bound (first element > target)
25 int upper_bound(vector<int>& arr, int target) {
26     int left = 0, right = arr.size();
27     while (left < right) {
28         int mid = left + (right - left) / 2;
29         arr[mid] <= target ? left = mid + 1
30                          : right = mid;
31     }
32     return left;
33 }
34
35 // Binary search on real numbers (e.g. sqrt)
36 double sqrt_precision(double n, double eps=1e-6) {
37     double left = 0, right = n;
38     for (int i = 0; i < 100; ++i) { // or while (right-left > eps)
39         double mid = (left + right) / 2;
40         if (mid*mid < n) left = mid;
41         else right = mid;
42     }
43     return left;
44 }
45
46 // Binary search on answer space (monotonic condition)
47 int find_min_valid(vector<int>& nums, int k) {
48     auto is_valid = [&](int x) {
49         /* condition check */
50     };
51
52     int left = 0, right = 1e9; // adjust bounds
53     while (left < right) {
54         int mid = left + (right - left) / 2;
55         is_valid(mid) ? right = mid
56                     : left = mid + 1;
57     }
58     return left;
59 }
60
61 // Binary search using for loop

```

```

62
63 int binary_search_for_loop(vector<int>& arr, int target) {
64     int left = 0, right = arr.size() - 1;
65     for (; left <= right; ) {
66         int mid = left + (right - left) / 2;
67         if (arr[mid] == target) return mid;
68         if (arr[mid] < target) left = mid + 1;
69         else right = mid - 1;
70     }
71     return -1;
72 }

```

4 Bitwise Techniques

4.1 All Subsets

```

1 void minSumSubset(vector<int> &a, ll total_zum){
2     int n = (int)a.size();
3
4     for(int i=1; i<=(1<<n); i++){
5         ll zum = 0;
6         for(int j=0; j<n; j++){
7             if((i & (1<<j)) != 0){
8                 zum+=a[j];
9             }
10        }
11        ll diff = abs(total_zum - 2LL * zum);
12        min_diff = min(diff,min_diff);
13    }
14
15 }

```

4.2 FWHT

```

1 const ll MOD = 1e9+7;
2
3 // MAX should be a power of 2 and >= 2*10^5
4 // FFT works with sizes that are powers of 2
5 // 2^18 = 262144 > 200000
6 const int MAX = 1 << 20;
7
8 typedef complex<double> cd;
9 double PI = acos(-1);
10
11 // Transformada de Wicho Cachondo
12 vector<ll> fwht(vector<ll> a, int d = 1) {

```

```

13     int n = a.size();
14     vector<ll> r = a;
15
16
17     for (int m = 2; m <= n; m *= 2) {
18         for (int k = 0; k < n; k += m) {
19             for (int j = 0; j < m / 2; j++) {
20                 ll u = r[k + j];
21                 ll t = r[k + j + m / 2];
22
23                 r[k + j] = u + t;
24                 r[k + j + m / 2] = u - t;
25             }
26         }
27     }
28
29     if (d == -1) {
30         for (int i = 0; i < n; i++)
31             r[i] /= n;
32     }
33
34     return r;
35 }

```

4.3 Gray Code

```

1 vector<string> grayCode(int n){
2     vector<string> arns;
3     for(int i=0; i<(1<<n); i++){
4         int g = i^(i>>1);
5         bitset<32> r(g);
6         string arnsito = r.to_string().substr(32-n, 32);
7         arns.push_back(arnsito);
8     }
9     return arns;
10 }

```

4.4 XOR Basis

```

1 const int D = 32;
2 ll basis[D];
3 ll bs_len = 0;
4
5 void add(int x){
6     for(int i=0; i<D; i++){
7         if((x & 1 << i) == 0) continue;
8

```

```

9         if(!basis[i]){
10             basis[i] = x;
11             bs_len++;
12             return;
13         }
14
15         x ^= basis[i];
16     }
17 }
18
19 void add2(ll x){
20     for(int i=D-1; i>=0; i--){
21         if((x>>i) & 1){
22             if(!basis[i]){
23                 basis[i] = x;
24                 return;
25             }
26             x ^= basis[i];
27         }
28     }
29 }
30
31 inline void solve(){
32     int n; cin>>n;
33     vector<int> st(n);
34     for(int &x : st) cin>>x;
35
36     for(int vec : st){
37         add2(vec);
38     }
39
40     ll max_xor = 0;
41
42     for(int i=D-1; i>=0; i--){
43         max_xor = max(max_xor, max_xor^basis[i]);
44     }
45
46     cout<<max_xor<<endl;
47 }
48 const int D = 32;
49 ll basis[D];
50 ll bs_len = 0;
51
52 void add(int x){
53     for(int i=0; i<D; i++){
54         if((x & 1 << i) == 0) continue;
55
56         if(!basis[i]){
57             basis[i] = x;
58             bs_len++;
59             return;
60         }
61

```

```

62     x ^= basis[i];
63     }
64 }
65
66 void add2(ll x){
67     for(int i=D-1; i>=0; i--){
68         if((x>>i) & 1){
69             if(!basis[i]){
70                 basis[i] = x;
71                 return;
72             }
73             x^=basis[i];
74         }
75     }
76 }
77
78 inline void solve(){
79     int n; cin>>n;
80     vector<int> st(n);
81     for(int &x : st) cin>>x;
82
83     for(int vec : st){
84         add2(vec);
85     }
86
87     ll max_xor = 0;
88
89     for(int i=D-1; i>=0; i--){
90         max_xor = max(max_xor, max_xor^basis[i]);
91     }
92
93     cout<<max_xor<<endl;
94 }

```

```

12     dist[start] = 0;
13     vector<int> prev(V, -1);
14
15     for (int i = 0; i < V-1; i++){
16         for (int j = 0; j < E; j++){
17             int u = edges[j].u;
18             int v = edges[j].v;
19             ll w = edges[j].w;
20             if (dist[u] == INF) continue;
21             dist[v] = min(dist[v], w+dist[u]);
22             if (dist[v] > dist[u] + w){
23                 dist[v] = max(MIN_INF, dist[u] + w);
24             }
25         }
26     }
27
28     for (int i = 0; i < V-1; i++){
29         for (int j = 0; j < E; j++){
30             int u = edges[j].u;
31             int v = edges[j].v;
32             ll w = edges[j].w;
33             if (dist[u] == INF) continue;
34             dist[v] = max(MIN_INF, dist[v]);
35             if (dist[u] == MIN_INF or dist[v] > dist[u] + w){
36                 dist[v] = MIN_INF;
37             }
38         }
39     }
40 }
41 }

```

5 Graph Theory

5.1 Bellman-Ford

```

1  const ll INF = 1e17;
2  const ll MIN_INF = (-1)*INF;
3
4  vector<ll> dist(MAX, INF);
5
6  struct Edge {
7      int u, v;
8      ll w;
9  };
10
11 void bellmanFord(int V, int E, vector<Edge>& edges, int start) {

```

5.2 Dijkstra

```

1  // Old implementation, use global variables.
2
3  const int MAX = 2e5+3;
4
5  #define INF LLONG_MAX
6  typedef pair<ll, ll> pll;
7
8  vector<ll> dijkstra(int n, int source, vector<vector<pll>> &adj) {
9      vector<ll> dist(n, INF);
10     priority_queue<pll, vector<pll>, greater<pll>> pq;
11     dist[source] = 0;
12     pq.push({0, source});
13
14     while (!pq.empty()) {
15         ll d = pq.top().first;
16         ll u = pq.top().second;
17         pq.pop();

```

```

18         if (d > dist[u]) continue;
19
20     for (auto &edge : adj[u]) {
21         ll v = edge.first;
22         ll weight = edge.second;
23
24         if (dist[u] + weight < dist[v]) {
25             dist[v] = dist[u] + weight;
26             pq.push({dist[v], v});
27         }
28     }
29 }
30
31 return dist;
32 }

```

5.3 Dinic

```

1 const int MAX = 503;
2
3 #define INF LLONG_MAX
4 #define pii pair<int,int>
5 #define fi first
6 #define sc second
7 #define all(x) x.begin(),x.end()
8
9 struct FlowEdge{
10     int u,v;
11     ll flow=0, cap;
12     FlowEdge(int u, int v, ll cap) : u(u), v(v), cap(cap){};
13 };
14
15 struct Dinic{
16     vector<FlowEdge> edges;
17     vector<vector<int>>> adj;
18     int n, s, t; // Vertices, Source, Sink
19     int id = 0;
20     vector<int> level, next;
21     queue<int> q;
22
23     Dinic(int n, int s, int t) : n(n), s(s), t(t){
24         adj.resize(n);
25         level.resize(n);
26         next.resize(n);
27         fill(all(level), -1);
28         level[s] = 0;
29         q.push(s);
30     }
31

```

```

32
33 void addEdge(int u, int v, ll cap){
34     edges.emplace_back(u,v,cap);
35     edges.emplace_back(v,u,0);
36     adj[u].push_back(id++);
37     adj[v].push_back(id++);
38 }
39
40 bool bfs(){
41     while(!q.empty()){
42         int u = q.front();
43         q.pop();
44         for(auto e : adj[u]){
45             // Saturated
46             if(edges[e].cap - edges[e].flow < 1) continue;
47             // Visited
48             if(level[edges[e].v] != -1) continue;
49
50             level[edges[e].v] = level[edges[e].u] + 1;
51             q.push(edges[e].v);
52         }
53     }
54
55     return level[t] != -1;
56 }
57
58 ll dfs(int u, ll flow){
59     if(flow == 0) return 0;
60     if(u == t) return flow;
61     // cid is current id
62     for(int &cid = next[u]; cid < adj[u].size(); cid++){
63         int e = adj[u][cid];
64         int v = edges[e].v;
65
66         if(level[edges[e].u] + 1 != level[v] or edges[e].cap -
67             edges[e].flow < 1)
68             continue;
69         ll f = dfs(v, min(flow, edges[e].cap - edges[e].flow));
70         if(f == 0) continue;
71
72         edges[e].flow += f;
73         edges[e^1].flow -= f;
74         return f;
75     }
76     return 0;
77 }
78
79 ll maxFlow(){
80     ll flow = 0;
81     while(bfs()){ // While there are aug paths
82         fill(all(next), 0);
83         for(ll f = dfs(s, INF); f != 0; f = dfs(s, INF)) flow += f;
84         fill(all(level), -1);
85     }
86 }

```

```

84         level[s] = 0;
85         q.push(s);
86     }
87
88     return flow;
89 }
90
91 vector<pii> minCut(){
92     vector<pii> arns;
93     fill(all(level), -1);
94     level[s] = 0;
95     q.push(s);
96     while(!q.empty()){
97         int u = q.front();
98         q.pop();
99         for(int id = 0; id < adj[u].size(); id++){
100             int e = adj[u][id];
101             if(level[edges[e].v] == -1 and edges[e].cap - edges[
102                 e].flow > 0){
103                 q.push(edges[e].v);
104                 level[edges[e].v] = level[edges[e].u] + 1;
105             }
106         }
107
108         for(int i=0; i<level.size(); i++){
109             if(level[i] != -1){
110                 for(int id=0; id<adj[i].size(); id++){
111                     int e = adj[i][id];
112                     if(level[edges[e].v] == -1 and edges[e].cap -
113                         edges[e].flow == 0)
114                         arns.emplace_back(edges[e].u, edges[e].v);
115                 }
116             }
117         }
118         return arns;
119     }
120
121     vector<pii> maxMatching(){
122         vector<pii> arns;
123         fill(all(level), -1); // Reset levels.
124         level[s] = 0;
125         q.push(s);
126         while(!q.empty()){
127             int u = q.front();
128             q.pop();
129             for(int id = 0; id < adj[u].size(); id++){
130                 int e = adj[u][id];
131                 if(level[edges[e].v] == -1 and edges[e].cap - edges[
132                     e].flow == 0 and edges[e].flow != 0ll){
133                     q.push(edges[e].v);
134                     level[edges[e].v] = level[edges[e].u] + 1;

```

```

134         }
135     }
136 }
137
138 for(int i=0; i<level.size(); i++){
139     if(level[i] != -1){
140         for(int id=0; id<adj[i].size(); id++){
141             int e = adj[i][id];
142             if(edges[e].u != s and edges[e].v != t and edges
143                 [e].cap - edges[e].flow == 0 and edges[e].
144                 flow != 0ll)
145                 arns.emplace_back(edges[e].u, edges[e].v
146                     );
147         }
148     }
149 }
150 };

```

5.4 Directed Cycles

```

1 int n; // number of vertices
2 vector<vector<int>> adj(MAX);
3 stack<int> st;
4 vector<bool> visited(MAX);
5 vector<bool> stFlg(MAX);
6 vector<int> arns;
7 int depth = 0;
8
9 bool dfs(int v) {
10     visited[v] = true;
11     st.push(v);
12     stFlg[v] = true;
13     for(auto &z : adj[v]){
14         if(!visited[z])
15             if(dfs(z)) return true;
16
17         if(stFlg[z]){
18             st.push(z);
19             return true;
20         }
21     }
22
23     st.pop();
24     stFlg[v] = false;
25     return false;
26 }
27

```

```

28 void printCycle(){
29     int tmp = st.top();
30     while(!st.empty()){
31         arns.push_back(st.top());
32         st.pop();
33         if(arns.back() == tmp and arns.size() != 1) break;
34     }
35     reverse(arns.begin(), arns.end());
36
37     cout<<arns.size()<<endl;
38     for(auto v: arns) cout<<v<<" ";
39     cout<<endl;
40 }

```

5.5 Edmond-Karp

```

1 vector<vector<int>> adj(MAX);
2 vector<vector<int>> cap(MAX, vector<int>(MAX));
3
4 int bfs(int s, int t, vector<int> &parent){
5     parent[s] = -2;
6     queue<pii> q;
7     q.push({s, INF});
8
9     while(!q.empty()){
10         int u = q.front().fi;
11         int flow = q.front().sc;
12         q.pop();
13
14         for(int v : adj[u]){
15             if(parent[v] == -1 and cap[u][v]){
16                 parent[v] = u;
17                 int new_flow = min(flow, cap[u][v]);
18                 if(v == t) return new_flow;
19                 q.push({v, new_flow});
20             }
21         }
22     }
23     return 0;
24 }
25
26 int edmond_karp(int s, int t, int n){
27     vector<int> parent(n+1, -1);
28     int flow = 0;
29     int new_flow;
30     while(new_flow = bfs(s, t, parent)){
31         flow += new_flow;
32         int cur = t;
33         while(cur != s){
34             int prev = parent[cur];

```

```

35         cap[prev][cur] -= new_flow;
36         cap[cur][prev] += new_flow;
37         cur = prev;
38     }
39     fill(parent.begin(), parent.end(), -1);
40 }
41
42 return flow;
43 }

```

5.6 Floyd-Warshall

```

1 void floydWarshall(vector<vector<ll>> &d, int n){
2     for (int k = 0; k < n; ++k) {
3         for (int i = 0; i < n; ++i) {
4             for (int j = 0; j < n; ++j) {
5                 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
6             }
7         }
8     }
9 }

```

5.7 Kahn Topological Sort (Python)

```

1 def kahnTopoSort(self, adj: List[List[int]]) -> List[int]:
2     #print(adj)
3     in_deg = [0] * len(adj)
4     for i in range(len(adj)):
5         for u in adj[i]:
6             in_deg[u] += 1
7
8     q = []
9     for i in range(len(in_deg)):
10         if in_deg[i] == 0:
11             q.append(i)
12
13     arns = []
14     while len(q) > 0:
15         u = q[0]
16         q.pop(0)
17         arns.append(u)
18
19         for v in adj[u]:
20             in_deg[v] -= 1
21             if in_deg[v] == 0:
22                 q.append(v)
23

```



```

24     print(str(len(arns))+" "+str(len(adj)))
25     if(len(arns) != len(adj)):
26         return []
27
28     return arns

```

5.8 Kruskal

```

1 struct Edge {
2     int u, v, w;
3     bool operator<(Edge const& other) {
4         return w < other.w;
5     }
6 };
7
8 class DisjointSets {
9 private:
10     vector<int> parents;
11     vector<int> rank; // Use rank instead of size
12
13 public:
14     DisjointSets(int size) : parents(size), rank(size, 0) {
15         for (int i = 0; i < size; i++) parents[i] = i;
16     }
17
18     int find(int x) {
19         if (parents[x] != x) parents[x] = find(parents[x]);
20         return parents[x];
21     }
22
23     bool unite(int x, int y) {
24         int x_root = find(x);
25         int y_root = find(y);
26         if (x_root == y_root) return false;
27
28         if (rank[x_root] < rank[y_root]) swap(x_root, y_root);
29         parents[y_root] = x_root;
30         if (rank[x_root] == rank[y_root]) rank[x_root]++;
31         return true;
32     }
33 };
34
35 int kruskal(int n, vector<Edge> &edges, vector<Edge> &ans){
36     ll cost = 0;
37     int ed_cnt = 0;
38     vector<bool> sup(n, false);
39     sort(edges.begin(), edges.end());
40
41     for(Edge e : edges){

```

```

43         if(!sup[e.v]){
44             sup[e.v] = true;
45             cost += e.w;
46             ed_cnt++;
47         }
48     }
49
50     if (ed_cnt != n - 1) return -1;
51     return cost;
52 }

```

5.9 Lava Flow

```

1 struct Cell{
2     int x,y,t;
3 };
4
5 const int MAX = 1005;
6 int n,m;
7
8 char maze[MAX][MAX];
9 int vis[MAX][MAX];
10 int player[MAX][MAX];
11 char path[MAX][MAX];
12 set<pii> isExit;
13 queue<Cell> q;
14 string res;
15
16 bool isValid(int x, int y){
17     if(x < 0 || x >= n || y < 0 || y >= m) return false;
18     if(maze[x][y] == '#') return false;
19     return true;
20 }
21
22 bool isSafe(int x, int y, int u, int v){
23     return player[x][y] == -1 and maze[x][y] != 'M' and (vis[x][y]
24         == -1 or player[u][v] + 1 < vis[x][y]);
25 }
26
27 void restorePath(int u, int v, int x, int y){
28     while (x != u || y != v) {
29         res.push_back(path[u][v]);
30
31         if (res.back() == 'U') u++;
32         if (res.back() == 'D') u--;
33         if (res.back() == 'L') v++;
34         if (res.back() == 'R') v--;
35     }
36 }

```

```

37
38 bool lavaFlow(int x,int y){
39     q.push({x,y,1});
40     player[x][y] = 0;
41
42     while(!q.empty()){
43         int u = q.front().x;
44         int v = q.front().y;
45         int t = q.front().t;
46
47         q.pop();
48
49         vector<pii> dir = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
50
51         for(auto it: dir){
52             int i = u+it.fs;
53             int j = v+it.sc;
54
55             if(isValid(i,j)){
56                 if(t == 0){
57                     if(vis[i][j] == -1){
58                         vis[i][j] = vis[u][v]+1;
59                         q.push(Cell{i,j,0});
60                     }
61                 }else{
62                     if(isSafe(i,j,u,v)){
63                         path[i][j] = (it.fs == 1) ? 'D' : (it.fs ==
64                             -1) ? 'U' : (it.sc == 1) ? 'R' : 'L';
65                         player[i][j] = player[u][v]+1;
66                         q.push(Cell{i,j,1});
67                         if (isExit.find({i,j}) != isExit.end()) {
68                             if (player[i][j] < vis[i][j] || vis[i][j]
69                                 ] == -1) {
70                                 restorePath(i, j, x, y);
71                                 return true;
72                             }
73                         }
74                     }
75                 }
76             }
77         }
78     }
79     return false;
80 }

```

5.10 Negative Cycles

```

1 struct Edge {

```

```

2     int u, v;
3     ll w;
4 };
5
6 void bellmanFord(int V, int E, vector<Edge>& edges, int start) {
7     vector<ll> dist(V, INF);
8     dist[start] = 0;
9     vector<int> prev(V, -1);
10
11     int x;
12     for (int i = 0; i < V; i++){
13         x = -1;
14         for (int j = 0; j < E; j++){
15             int u = edges[j].u;
16             int v = edges[j].v;
17             ll weight = edges[j].w;
18             if (dist[u] != INF && dist[u] + weight < dist[v]) {
19                 dist[v] = max(-INF, (ll) dist[u] + weight);
20                 prev[v] = u;
21                 x = v;
22             }
23         }
24     }
25
26     if(x == -1){
27         cout<<"NO"<<endl; // No negative Cycle
28     }else{
29         cout<<"YES"<<endl;
30         int curr = x;
31         for (int i = 0; i < V; ++i)
32             curr = prev[curr];
33
34         vector<int> cycle;
35         for (int x = curr;; x = prev[x]) {
36             cycle.push_back(x);
37             if (x == curr and cycle.size() > 1) break;
38         }
39
40         reverse(cycle.begin(), cycle.end());
41
42         for(int v : cycle)
43             cout<<v+1<<" ";
44         //cout<<cycle[0]+1<<endl;
45     }
46 }
47
48
49 }
50
51 void solve(){
52     int n,m; cin>>n>>m;
53     vector<Edge> edges;
54

```

```

55     for(int i=0; i<m; i++){
56         int u,v; ll w; cin>>u>>v>>w;
57         edges.push_back({u-1,v-1,w});
58     }
59
60     int dummyNode = n;
61     for (int i = 0; i < n; ++i) {
62         edges.push_back({dummyNode, i, 0LL});
63     }
64
65     bellmanFord(n + 1, m + n, edges, dummyNode);
66
67 }
68

```

```

34     }
35 }
36
37     for(int i=0; i<V; i++){
38         if(!visited[i])
39             return -1;
40     }
41
42     return res;
43 }

```

5.11 Prim Algorithm

```

1  const int MAX = 2e5+3;
2
3  #define INF LLONG_MAX
4  typedef pair<ll, ll> pll;
5
6  ll prim(int V, int E, vector<vector<pll>> &adj) {
7
8      priority_queue<pll, vector<pll>, greater<pll>> pq;
9
10     vector<bool> visited(V, false);
11
12     ll res = 0;
13
14     pq.push({0, 0});
15
16     while(!pq.empty()){
17         auto p = pq.top();
18         pq.pop();
19
20         int wt = p.first;
21         int u = p.second;
22
23         if(visited[u] == true){
24             continue;
25         }
26
27         res += wt;
28         visited[u] = true;
29
30         for(auto v : adj[u]){
31             if(visited[v.first] == false){
32                 pq.push({v.second, v.first});
33             }
34         }
35     }
36 }

```

5.12 Strongly Connected Components

```

1  const int MAX = 2e5+3;
2
3
4  #define INF LLONG_MAX
5  typedef pair<ll, ll> pll;
6
7  vector<vector<int>> adj(MAX);
8  vector<vector<int>> t_adj(MAX);
9  vector<int> v_id(MAX);
10 bool visited[MAX];
11 vector<int> order; // Append the respective nodes with dfsA order
12
13 // Calculate vertex order
14 void dfsA(int s){
15     visited[s] = true;
16     for(int u : adj[s]){
17         if(!visited[u])
18             dfsA(u);
19     }
20     order.push_back(s);
21 }
22
23 // Set its component id to every vertex
24 void dfsB(int s, int c){
25     visited[s] = true;
26     for(int u : t_adj[s]){
27         if(!visited[u])
28             dfsB(u, c);
29     }
30     v_id[s] = c;
31 }
32
33
34 void solve(){
35     int n,m; cin>>n>>m;
36
37     for(int i=0; i<m; i++){

```

```

38     int u,v; cin>>u>>v;
39     adj[u].push_back(v);
40     t_adj[v].push_back(u);
41 }
42
43 for(int i=1; i<=n; i++){
44     if(!visited[i])
45         dfsA(i);
46 }
47
48 for(int i=1; i<=n; i++)
49     visited[i] = false;
50
51 reverse(order.begin(),order.end());
52
53 int c = 0;
54 for(int i=0; i<order.size(); i++){
55     if(!visited[order[i]]){
56         c++;
57         dfsB(order[i],c);
58     }
59 }
60
61 if(c == 1){
62     cout<<"YES"<<endl;
63 }else{
64     bool flag1 = false, flag2 = false;
65     cout<<"NO"<<endl;
66     int arns[2];
67     for(int i=1; i<=n; i++){
68         if(v_id[i] == 1 and flag1 == false){
69             flag1 = true;
70             arns[0] = i;
71         }
72
73         if(v_id[i] == 2 and flag2 == false){
74             flag2 = true;
75             arns[1] = i;
76         }
77
78         if(flag1 == true and flag2 ==true) break;
79     }
80     cout<<arns[1]<<" "<<arns[0]<<endl;
81 }
82 }

```

5.13 Topological Sort

```

1 void dfs(int v) {
2     visited[v] = true;

```

```

3     for (int u : adj[v]) {
4         if (!visited[u])
5             dfs(u);
6     }
7     ans.push_back(v);
8 }
9
10 void topological_sort() {
11     visited.assign(n+1, false);
12     ans.clear();
13     for (int i = 1; i <= n; ++i) {
14         if (!visited[i]) {
15             dfs(i);
16         }
17     }
18     reverse(ans.begin(), ans.end());
19 }

```

5.14 2-SAT

```

1 // Kosaraju Algorithm
2 struct TwoSAT{
3     int vars;
4     int n;
5     vector<vector<int>> adj, adj_t;
6     vector<bool> used;
7     vector<int> order, comp;
8     vector<bool> values;
9
10     TwoSAT(int _vars) :
11         vars(_vars),
12         n(2 * vars),
13         adj(n),
14         adj_t(n),
15         used(n),
16         order(),
17         comp(n,-1),
18         values(n){
19         order.reserve(n);
20     }
21
22     void dfsA(int s){
23         used[s] = true;
24         for(int u : adj[s]){
25             if(!used[u]) dfsA(u);
26         }
27         order.push_back(s);
28     }
29
30     void dfsB(int s, int sc){

```

```

31     comp[s] = sc;
32     for(int u : adj_t[s]){
33         if(comp[u] == -1) dfsB(u,sc);
34     }
35 }
36
37 bool twoSAT(){
38     order.clear();
39     used.assign(n, false);
40
41     for(int i=0; i<n; i++){
42         if(!used[i]) dfsA(i);
43     }
44
45     comp.assign(n, -1);
46
47     for(int i=0, j=0; i<n; i++){
48         int v = order[n-i-1];
49         if(comp[v] == -1) dfsB(v, j++);
50     }
51
52     values.assign(vars, false);
53
54     for(int i=0; i<n; i+=2){
55         if(comp[i] == comp[i+1]) return false;
56         values[i/2] = comp[i] > comp[i+1];
57     }
58     return true;
59 }
60
61 void add_disjunction(int a, bool na, int b, bool nb){
62     a = 2* a ^ na;
63     b = 2* b ^ nb;
64
65     int not_a = a^1;
66     int not_b = b^1;
67
68     adj[not_a].push_back(b);
69     adj[not_b].push_back(a);
70     adj_t[b].push_back(not_a);
71     adj_t[a].push_back(not_b);
72 }
73 };

```

5.15 Undirected Cycles

```

1 bool dfs(int v, int par) { // passing vertex and its parent vertex
2     visited[v] = true;
3     for (int u : adj[v]) {
4         if(u == par) continue; // skipping edge to parent vertex

```

```

5         if (visited[u]) {
6             cycle_end = v;
7             cycle_start = u;
8             return true;
9         }
10        parent[u] = v;
11        if (dfs(u, parent[u]))
12            return true;
13    }
14    return false;
15 }
16
17 void find_cycle() {
18     visited.assign(n+1, false);
19     parent.assign(n+1, -1);
20     cycle_start = -1;
21
22     for (int v = 0; v < n; v++) {
23         if (!visited[v] && dfs(v, parent[v]))
24             break;
25     }
26
27     if (cycle_start == -1) {
28         cout << "IMPOSSIBLE" << endl;
29     } else {
30         vector<int> cycle;
31         cycle.push_back(cycle_start);
32         for (int v = cycle_end; v != cycle_start; v = parent[v])
33             cycle.push_back(v);
34         cycle.push_back(cycle_start);
35
36         cout << cycle.size() << endl;
37         for (int v : cycle)
38             cout << v << " ";
39         cout << endl;
40     }
41 }

```

5.16 Articulation Points

```

1 // adj[u] = adjacent nodes of u
2 // ap = AP = articulation points
3 // p = parent
4 // disc[u] = discovery time of u
5 // low[u] = 'low' node of u
6
7 int dfsAP(int u, int p) {
8     int children = 0;
9     low[u] = disc[u] = ++Time;
10    for (int& v : adj[u]) {

```

```

11     if (v == p) continue; // we don't want to go back through the
12         // same path.
13         // if we go back is because we found
14         // another way back
15     if (!disc[v]) { // if V has not been discovered before
16         children++;
17         dfsAP(v, u); // recursive DFS call
18         if (disc[u] <= low[v]) // condition #1
19             ap[u] = 1;
20         low[u] = min(low[u], low[v]); // low[v] might be an ancestor
21         // of u
22     } else // if v was already discovered means that we found an
23         // ancestor
24         low[u] = min(low[u], disc[v]); // finds the ancestor with the
25         // least discovery time
26 }
27 return children;
28 }
29
30 void AP() {
31     ap = low = disc = vector<int>(adj.size());
32     Time = 0;
33     for (int u = 0; u < adj.size(); u++)
34         if (!disc[u])
35             ap[u] = dfsAP(u, u) > 1; // condition #2
36 }

```

5.17 Bridges

```

1 // br = bridges, p = parent
2
3 vector<pair<int, int>> br;
4
5 int dfsBR(int u, int p) {
6     low[u] = disc[u] = ++Time;
7     for (int& v : adj[u]) {
8         if (v == p) continue; // we don't want to go back through the
9             // same path.
10            // if we go back is because we found
11            // another way back
12     if (!disc[v]) { // if V has not been discovered before
13         dfsBR(v, u); // recursive DFS call
14         if (disc[u] < low[v]) // condition to find a bridge
15             br.push_back({u, v});
16         low[u] = min(low[u], low[v]); // low[v] might be an ancestor
17         // of u
18     } else // if v was already discovered means that we found an
19         // ancestor
20         low[u] = min(low[u], disc[v]); // finds the ancestor with the
21         // least discovery time
22 }

```

```

17     }
18 }
19
20 void BR() {
21     low = disc = vector<int>(adj.size());
22     Time = 0;
23     for (int u = 0; u < adj.size(); u++)
24         if (!disc[u])
25             dfsBR(u, u)
26 }

```

5.18 Bridges Online

```

1 vector<int> par, dsu_2ecc, dsu_cc, dsu_cc_size;
2 int bridges;
3 int lca_iteration;
4 vector<int> last_visit;
5
6 void init(int n) {
7     par.resize(n);
8     dsu_2ecc.resize(n);
9     dsu_cc.resize(n);
10    dsu_cc_size.resize(n);
11    lca_iteration = 0;
12    last_visit.assign(n, 0);
13    for (int i=0; i<n; ++i) {
14        dsu_2ecc[i] = i;
15        dsu_cc[i] = i;
16        dsu_cc_size[i] = 1;
17        par[i] = -1;
18    }
19    bridges = 0;
20 }
21
22 int find_2ecc(int v) {
23     if (v == -1)
24         return -1;
25     return dsu_2ecc[v] == v ? v : dsu_2ecc[v] = find_2ecc(dsu_2ecc[v]);
26 }
27
28 int find_cc(int v) {
29     v = find_2ecc(v);
30     return dsu_cc[v] == v ? v : dsu_cc[v] = find_cc(dsu_cc[v]);
31 }
32
33 void make_root(int v) {
34     int root = v;
35     int child = -1;
36     while (v != -1) {

```

```

37     int p = find_2ecc(par[v]);
38     par[v] = child;
39     dsu_cc[v] = root;
40     child = v;
41     v = p;
42 }
43 dsu_cc_size[root] = dsu_cc_size[child];
44 }
45
46 void merge_path (int a, int b) {
47     ++lca_iteration;
48     vector<int> path_a, path_b;
49     int lca = -1;
50     while (lca == -1) {
51         if (a != -1) {
52             a = find_2ecc(a);
53             path_a.push_back(a);
54             if (last_visit[a] == lca_iteration){
55                 lca = a;
56                 break;
57             }
58             last_visit[a] = lca_iteration;
59             a = par[a];
60         }
61         if (b != -1) {
62             b = find_2ecc(b);
63             path_b.push_back(b);
64             if (last_visit[b] == lca_iteration){
65                 lca = b;
66                 break;
67             }
68             last_visit[b] = lca_iteration;
69             b = par[b];
70         }
71     }
72 }
73
74 for (int v : path_a) {
75     dsu_2ecc[v] = lca;
76     if (v == lca)
77         break;
78     --bridges;
79 }
80 for (int v : path_b) {
81     dsu_2ecc[v] = lca;
82     if (v == lca)
83         break;
84     --bridges;
85 }
86 }
87
88 void add_edge(int a, int b) {
89     a = find_2ecc(a);

```

```

90     b = find_2ecc(b);
91     if (a == b)
92         return;
93
94     int ca = find_cc(a);
95     int cb = find_cc(b);
96
97     if (ca != cb) {
98         ++bridges;
99         if (dsu_cc_size[ca] > dsu_cc_size[cb]) {
100             swap(a, b);
101             swap(ca, cb);
102         }
103         make_root(a);
104         par[a] = dsu_cc[a] = b;
105         dsu_cc_size[cb] += dsu_cc_size[a];
106     } else {
107         merge_path(a, b);
108     }
109 }

```

5.19 Bridge Tree

```

1 vector<pair<int, int>> g[MAXN];
2 bool used[MAXN], isBridge[MAXN];
3 int comp[MAXN], tin[MAXN], minAncestor[MAXN];
4
5 vector<int> tree[MAXN]; // Store 2-edge-connected component tree.(
6     Bridge tree).
7
8 void dfs(int v, int p) {
9     tin[v] = minAncestor[v] = ++timer;
10    used[v] = 1;
11    for(auto &e: g[v]) {
12        int to, id;
13        tie(to, id) = e;
14        if(to == p) continue;
15        if(used[to]) {
16            minAncestor[v] = min(minAncestor[v], tin[to]);
17        } else {
18            dfs(to, v);
19            minAncestor[v] = min(minAncestor[v], minAncestor[to]);
20            if(minAncestor[to] > tin[v]) {
21                isBridge[id] = true;
22            }
23        }
24    }
25 }
26 void dfs1(int v, int p) {

```

```

27     used[v] = 1;
28     comp[v] = compid;
29     for(auto &e: g[v]) {
30         int to, id;
31         tie(to, id) = e;
32
33         if(isBridge[id]) { // avoid traversing from this edge. so we
34             get full component.
35             continue;
36         }
37         if(used[to]) {
38             continue;
39         }
40         dfs1(to, v);
41     }
42 }
43 vector<pair<int, int>> edges;
44
45 void addEdge(int from, int to, int id) {
46     g[from].push_back({to, id});
47     g[to].push_back({from, id});
48     edges[id] = {from, to};
49 }
50
51 void initB() {
52
53     for(int i = 0; i <= compid; ++i)
54         tree[i].clear();
55     for(int i = 1; i <= N; ++i)
56         used[i] = false;
57     for(int i = 1; i <= M; ++i)
58         isBridge[i] = false;
59
60     timer = 0;
61     compid = 0;
62 }
63
64 void bridge_tree() {
65
66     initB();
67
68     dfs(1, -1); //Assuming graph is connected.
69
70     for(int i = 1; i <= N; ++i)
71         used[i] = 0;
72
73     for(int i = 1; i <= N; ++i) {
74         if(!used[i]) {
75             dfs1(i, -1);
76             ++compid;
77         }
78     }

```

```

79
80     for(int i = 1; i <= M; ++i) {
81         if(isBridge[i]) {
82             int u, v;
83             tie(u, v) = edges[i];
84             // connect two componets using edge.
85             tree[comp[u]].push_back(comp[v]);
86             tree[comp[v]].push_back(comp[u]);
87         }
88     }
89 }
90
91 void init() {
92     edges.clear(); edges.resize(M + 1);
93     for(int i = 1; i <= N; ++i)
94         g[i].clear();
95 }

```

5.20 Dynamic Connectivity

```

1  const int BS=18;
2  //ascii https://elcodigoascii.com.ar/
3
4
5  struct query{
6      int citya,cityb,l,r,id;
7      query(int citya, int cityb, int l, int r,int id): citya(citya),
8          cityb(cityb), l(l), r(r),
9          id(id)
10     {}
11 };
12
13 class DSU{
14     private:
15         vector<int> leader;
16         vector<int> sz;
17         int n;
18
19         int getLeader(int a){
20             if(leader[a]==a) return a;
21             leader[a]=getLeader(leader[a]);
22             return leader[a];
23         }
24     public:
25         DSU(int n): n(n){
26             leader=vector<int> (n+1);
27             sz=vector<int> (n+1,1);
28             FO(i,n+1){
29                 leader[i]=i;

```



```

30     }
31 }
32
33 void clear(){
34     leader=vector<int> (n+1);
35     sz=vector<int> (n+1,1);
36     FO(i,n+1){
37         leader[i]=i;
38     }
39 }
40
41 void unite(int a,int b){
42     a=getLeader(a);
43     b=getLeader(b);
44     if(a==b) return;
45     if(sz[a]<sz[b]){
46         swap(a,b);
47     }
48     leader[b]=a;
49     sz[a]+=sz[b];
50 }
51
52 bool isConnected(int a,int b){
53     return getLeader(a)==getLeader(b);
54 }
55
56 };
57
58
59 inline void solve()
60 {
61     int n,m,q; cin>>n>>m>>q;
62     vector<vector<query>> timeline(m);
63     vector<vector<query>> nxt(m);
64     vector<pair<int,int>> edges(m);
65     vector<int> res(q,-2);
66     DSU dj(n);
67     FO(i,m){
68         int a,b; cin>>a>>b;
69         edges[i]={a,b};
70     }
71
72     FO(i,q){
73         int a,b; cin>>a>>b;
74         if(a==b) res[i]=-1;
75         else timeline[(m-1)/2].push_back({a,b,0,m-1,i});
76     }
77     FO(i,BS){
78         dj.clear();
79         FO(i,m) nxt[i].clear();
80         for(int j=0;j<m;j++)
81         {
82             dj.unite(edges[j].X,edges[j].Y);

```

```

83
84     for(query &q:timeline[j])
85     {
86         if(q.l>q.r) continue;
87         if(dj.isConnected(q.citya,q.cityb)){
88
89             res[q.id]=j;
90             int mid=(j-1+q.l)/2;
91             nxt[mid].push_back(query(q.citya,q.cityb,q.l,j
92                                     -1,q.id));
93         }
94         else{
95             int mid=(j+1+q.r)/2;
96             nxt[mid].push_back(query(q.citya,q.cityb,j+1,q.r
97                                     ,q.id));
98         }
99     }
100     timeline.swap(nxt);
101 }
102 for(int r:res){
103     cout<<r+1<<endl;
104 }

```

5.21 Eulerian Path

```

1 int n, m;
2 vector<vector<int>> g;
3 vector<int> in, out, path;
4
5 // Undirected
6
7 int n, m;
8 vector<vector<pair<int, int>>> g;
9 vector<int> path;
10 vector<bool> seen;
11
12 void dfs(int node) {
13     while (!g[node].empty()) {
14         auto [son, idx] = g[node].back();
15         g[node].pop_back();
16         if (seen[idx]) { continue; }
17         seen[idx] = true;
18         dfs(son);
19     }
20     path.push_back(node);
21 }
22
23 // Directed

```

```

24 void dfs(int node) {
25     while (!g[node].empty()) {
26         int son = g[node].back();
27         g[node].pop_back();
28         dfs(son);
29     }
30     path.push_back(node);
31 }

```

5.22 Gale-Shapley

```

1  // Checks if woman 'w' prefers 'm1' over 'm'
2  bool wPrefersM1OverM(vector<vector<int>> &prefer, int w, int m, int
   m1)
3  {
4      int N = prefer[0].size();
5      for (int i = 0; i < N; i++)
6      {
7          // If m1 comes before m, w prefers
8          // her current engagement
9          if (prefer[w][i] == m1)
10             return true;
11
12         // If m comes before m1, w prefers m
13         if (prefer[w][i] == m)
14             return false;
15     }
16 }
17
18 // Implements the stable marriage algorithm
19 vector<int> stableMarriage(vector<vector<int>> &prefer)
20 {
21     int N = prefer[0].size();
22
23     // Stores women's partners
24     vector<int> wPartner(N, -1);
25
26     // Tracks free men
27     vector<bool> mFree(N, false);
28     int freeCount = N;
29
30     while (freeCount > 0)
31     {
32         int m;
33         for (m = 0; m < N; m++)
34             if (!mFree[m])
35                 break;
36
37         // Process each woman in m's preference list
38         for (int i = 0; i < N && !mFree[m]; i++)

```

```

39     {
40         int w = prefer[m][i];
41         if (wPartner[w - N] == -1)
42         {
43             // Engage m and w if w is free
44             wPartner[w - N] = m;
45             mFree[m] = true;
46             freeCount--;
47         }
48         else
49         {
50             int m1 = wPartner[w - N];
51             // If w prefers m over her current partner, reassign
52             if (!wPrefersM1OverM(prefer, w, m, m1))
53             {
54                 wPartner[w - N] = m;
55                 mFree[m] = true;
56                 mFree[m1] = false;
57             }
58         }
59     }
60 }
61 return wPartner;
62 }

```

5.23 Hungarian Algorithm

```

1  vector<int> u (n+1), v (m+1), p (m+1), way (m+1);
2  for (int i=1; i<=n; ++i) {
3      p[0] = i;
4      int j0 = 0;
5      vector<int> minv (m+1, INF);
6      vector<bool> used (m+1, false);
7      do {
8          used[j0] = true;
9          int i0 = p[j0], delta = INF, j1;
10         for (int j=1; j<=m; ++j)
11             if (!used[j]) {
12                 int cur = A[i0][j]-u[i0]-v[j];
13                 if (cur < minv[j])
14                     minv[j] = cur, way[j] = j0;
15                 if (minv[j] < delta)
16                     delta = minv[j], j1 = j;
17             }
18         for (int j=0; j<=m; ++j)
19             if (used[j])
20                 u[p[j]] += delta, v[j] -= delta;
21         else
22             minv[j] -= delta;
23         j0 = j1;

```

```

24     } while (p[j0] != 0);
25     do {
26         int j1 = way[j0];
27         p[j0] = p[j1];
28         j0 = j1;
29     } while (j0);
30 }
31
32 vector<int> ans (n+1);
33 for (int j=1; j<=m; ++j)
34     ans[p[j]] = j;
35
36 int cost = -v[0];

```

5.24 SPFA

```

1  const int INF = 1000000000;
2  vector<vector<pair<int, int>>> adj;
3
4  bool spfa(int s, vector<int>& d) {
5      int n = adj.size();
6      d.assign(n, INF);
7      vector<int> cnt(n, 0);
8      vector<bool> inqueue(n, false);
9      queue<int> q;
10
11      d[s] = 0;
12      q.push(s);
13      inqueue[s] = true;
14      while (!q.empty()) {
15          int v = q.front();
16          q.pop();
17          inqueue[v] = false;
18
19          for (auto edge : adj[v]) {
20              int to = edge.first;
21              int len = edge.second;
22
23              if (d[v] + len < d[to]) {
24                  d[to] = d[v] + len;
25                  if (!inqueue[to]) {
26                      q.push(to);
27                      inqueue[to] = true;
28                      cnt[to]++;
29                      if (cnt[to] > n)
30                          return false; // negative cycle
31                  }
32              }
33          }
34      }

```

```

35     return true;
36 }

```

5.25 Tarjan

```

1  /** Takes in an adjacency list and calculates the SCCs of the graph.
2     */
3  class TarjanSolver {
4  private:
5      vector<vector<int>>> rev_adj;
6      vector<int> post;
7      vector<int> comp;
8
9      vector<bool> visited;
10     int timer = 0;
11     int id = 0;
12
13     void fill_post(int at) {
14         visited[at] = true;
15         for (int n : rev_adj[at]) {
16             if (!visited[n]) { fill_post(n); }
17         }
18         post[at] = timer++;
19     }
20
21     void find_comp(int at) {
22         visited[at] = true;
23         comp[at] = id;
24         for (int n : adj[at]) {
25             if (!visited[n]) { find_comp(n); }
26         }
27     }
28
29 public:
30     const vector<vector<int>>> &adj;
31
32     TarjanSolver(const vector<vector<int>>> &adj)
33         : adj(adj), rev_adj(adj.size()), post(adj.size()), comp(adj.size()),
34         visited(adj.size()) {
35         vector<int> nodes(adj.size());
36         for (int n = 0; n < adj.size(); n++) {
37             nodes[n] = n;
38             for (int next : adj[n]) { rev_adj[next].push_back(n); }
39         }
40
41         for (int n = 0; n < adj.size(); n++) {
42             if (!visited[n]) { fill_post(n); }
43         }
44
45         std::sort(nodes.begin(), nodes.end(),

```

```

44         [&](int n1, int n2) { return post[n1] > post[n2]; });
45
46     visited.assign(adj.size(), false);
47     for (int n : nodes) {
48         if (!visited[n]) {
49             find_comp(n);
50             id++;
51         }
52     }
53 }
54
55 int comp_num() const { return id; }
56
57 int get_comp(int n) const { return comp[n]; }
58 };

```

6 Data Structures

6.1 2D Segment Tree

```

1  class SegmentTree2D{
2  private:
3      vector<vector<int>> segtree;
4      int n;
5      int m;
6      int d1;
7      int d2;
8
9  public:
10
11     void build(vector<vector<int>> &a){
12         n = a.size();
13         m = a[0].size();
14         d1=1;
15         while(d1<n) d1<<=1;
16
17         d2=1;
18         while(d2<m) d2<<=1;
19
20         segtree.assign(2*d1, vector<int>(2*d2,0));
21
22         // Set leaves
23         for(int i=0; i<n; i++){
24             for(int j=0; j<m; j++){
25                 segtree[d1+i][d2+j] = a[i][j];
26
27             // Propagate on columns
28             for(int i=d1; i<2*d1; i++){
29                 for(int j=d2-1; j>0; j--){

```

```

30                 segtree[i][j] = segtree[i][2*j] + segtree[i][2*j+1];
31             }
32
33             // Propagate on rows
34             for(int i=d1-1; i>0; i--){
35                 for(int j=1; j<2*d2; j++){
36                     segtree[i][j] = segtree[2*i][j] + segtree[2*i+1][j];
37                 }
38             }
39
40     void update(int x, int y, int k){
41         int r = x + d1;
42         int c = y + d2;
43
44         segtree[r][c] = k;
45
46         // Update columns for the row corresponding to x
47         for(int j=c>>1; j>0; j>>=1)
48             segtree[r][j] = segtree[r][2*j] + segtree[r][2*j+1];
49
50
51         for(int i=r>>1; i>0; i>>=1){
52             segtree[i][c] = segtree[2*i][c] + segtree[2*i+1][c];
53             for(int j=c>>1; j>0; j>>=1)
54                 segtree[i][j] = segtree[i][2*j] + segtree[i][2*j+1];
55         }
56     }
57
58
59
60     int query(int x1, int y1, int x2, int y2){
61         int lx = x1+d1, rx = x2+d1+1;
62
63         ll arns = 0;
64         while(lx<rx) {
65             if(lx & 1){
66                 arns += query_y(lx, y1, y2);
67                 ++lx;
68             }
69             if (rx & 1){
70                 --rx;
71                 arns += query_y(rx, y1, y2);
72             }
73             lx >>= 1; rx >>= 1;
74         }
75         return arns;
76     }
77
78     int query_y(int row, int y1, int y2) {
79         int ly = y1 + d2, ry = y2 + 1 + d2;
80         int s = 0;
81         while (ly < ry) {
82             if(ly & 1) s+=segtree[row][ly++];

```

```

83         if(ry & 1) s+=segtree[row][--ry];
84         ly >>= 1; ry >>= 1;
85     }
86     return s;
87 }
88
89 };

```

6.2 Bit Trie

```

1  class Trie{
2  private:
3      Trie *children[2];
4      bool term_st; // is_end
5
6      Trie* searchPrefix(int x){
7          Trie* node = this;
8
9          for(int i=29; i>=0; i--){
10             int bit = (x>>i)&1;
11             if(!node->children[bit]) return nullptr;
12             node = node->children[bit];
13         }
14
15         return node;
16     }
17
18 public:
19     Trie() : term_st(false) {
20         for(int i=0; i<2; i++){
21             children[i] = nullptr;
22         }
23
24     void insert(int x) {
25         Trie* node = this;
26         for(int i=29; i>=0; i--){
27             int bit = (x>>i)&1;
28             if(!node->children[bit]) node->children[bit] = new Trie
29                 ();
30             node = node->children[bit];
31         }
32
33         node->term_st = true;
34     }
35
36     bool search(int x) {
37         Trie* node = searchPrefix(x);
38         return node != nullptr && node->term_st;
39     }

```

```

40     bool startsWith(int x) {
41         Trie* node = searchPrefix(x);
42         return node != nullptr;
43     }
44
45     int maxXor(int x){
46         Trie* node = this;
47         int arns = 0;
48         for(int i=29; i>=0; i--){
49             int bit = (x>>i)&1;
50             int not_bit = 1-bit;
51             if(node->children[not_bit]){
52                 arns|=(1<<i);
53                 node = node->children[not_bit];
54             }else{
55
56                 node = node->children[bit];
57             }
58         }
59
60         return arns;
61     }
62 };

```

6.3 Disjoint Set Union

```

1  class DisjointSets{
2  private:
3      vector<int> parents;
4      vector<int> sizes;
5      int components;
6
7  public:
8      DisjointSets(int size) : parents(size), sizes(size,1),
9          components(size){
10         for(int i=0; i<size; i++){parents[i] = i;}
11     }
12
13     int find(int x) {return parents[x] == x ? x : (parents[x] =
14         find(parents[x]));}
15
16     bool unite(int x, int y){
17         int x_root = find(x);
18         int y_root = find(y);
19
20         if(x_root == y_root) {return false;}
21
22         if(sizes[x_root] < sizes[y_root]) {swap(x_root,y_root);}
23         sizes[x_root] += sizes[y_root];
24         parents[y_root] = x_root;
25         components--;

```

```

23     return true;
24 }
25
26 vector<int> getAllComponentSizes(){
27     map<int, int> component_sizes;
28     for (int i = 0; i < parents.size(); ++i){
29         int root = find(i);
30         if (component_sizes.find(root) == component_sizes.
31             end()){
32             component_sizes[root] = sizes[root];
33         }
34     }
35
36     vector<int> result;
37     for (auto& [root, size] : component_sizes) {
38         result.push_back(size);
39     }
40
41     return result;
42 }
43
44 bool connected(int x, int y) { return find(x) == find(y);}
45 int getSize(int x) {return sizes[find(x)];}
46 int getComponents() const {return components;}
47 };

```

6.4 Fenwick Tree

```

1 // Fenwick Tree (Binary Indexed Tree)
2 class FenwickTree {
3     vector<long long> tree;
4     int n;
5
6 public:
7     FenwickTree(int n) : n(n), tree(n + 1, 0) {}
8
9     void update(int idx, int delta) {
10         for (++idx; idx <= n; idx += idx & -idx) {
11             tree[idx] += delta;
12         }
13     }
14
15     long long query(int idx) {
16         long long sum = 0;
17         for (++idx; idx > 0; idx -= idx & -idx) {
18             sum += tree[idx];
19         }
20         return sum;
21     }
22 }

```

```

22
23     long long query(int l, int r) {
24         return query(r) - (l > 0 ? query(l - 1) : 0);
25     }
26 };

```

6.5 GP Hash Table

```

1 using namespace __gnu_pbds;
2
3 #define ll long long
4 #define pll pair<ll,ll>
5 #define fi first
6 #define sc second
7 #define u_map gp_hash_table
8
9 struct custom_hash {
10     static uint64_t splitmix64(uint64_t x) {
11         // http://xorshift.di.unimi.it/splitmix64.c
12         x += 0x9e3779b97f4a7c15;
13         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
14         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
15         return x ^ (x >> 31);
16     }
17
18     size_t operator()(uint64_t x) const {
19         static const uint64_t FIXED_RANDOM = chrono::steady_clock::
20             now().time_since_epoch().count();
21         return splitmix64(x + FIXED_RANDOM);
22     }
23 };
24
25 inline void solve(){
26     int n; ll t;
27     cin>>n>>t;
28     vector<ll> arr(n);
29     for(ll &x: arr) cin>>x;
30     u_map<ll,pll, custom_hash> h;
31
32     for(int i=0; i<n; i++){
33         for(int j=i+1; j<n; j++){
34             ll key = t - arr[i] - arr[j];
35             if(h.find(key) != h.end()){
36                 cout<<i+1<<" "<<j+1<<" "<<h[key].fi+1<<" "<<h[key].sc+1<<endl;
37                 return;
38             }
39         }
40     }
41
42     for(int j=0; j<i; j++) h[arr[i]+arr[j]] = {j,i};
43 }

```

```

42     cout<<"IMPOSSIBLE"<<endl;
43 }
44

```

6.6 Implicit Treap

```

1  // Implicit Treap (sequence by position)
2  struct ImplicitTreap {
3      static int nextPriority() {
4          static mt19937 rng(chrono::steady_clock::now().
5              time_since_epoch().count());
6          static uniform_int_distribution<int> dist(INT_MIN, INT_MAX);
7          return dist(rng);
8      }
9
10     struct Node {
11         int val;
12         int prior;
13         int sz;
14         long long sum;
15         Node *l, *r;
16
17         explicit Node(int v)
18             : val(v), prior(nextPriority()), sz(1), sum(v), l(
19                 nullptr), r(nullptr) {}
20
21     };
22
23     using pNode = Node*;
24     pNode root = nullptr;
25
26     int sz(pNode t) { return t ? t->sz : 0; }
27     long long sum(pNode t) { return t ? t->sum : 0; }
28
29     void pull(pNode t) {
30         if (!t) return;
31         t->sz = 1 + sz(t->l) + sz(t->r);
32         t->sum = t->val + sum(t->l) + sum(t->r);
33     }
34
35     void split(pNode t, int leftSize, pNode &l, pNode &r) {
36         if (!t) {
37             l = r = nullptr;
38             return;
39         }
40         int curLeft = sz(t->l);
41         if (leftSize <= curLeft) {
42             split(t->l, leftSize, l, t->l);
43             r = t;
44         } else {
45             split(t->r, leftSize - curLeft - 1, t->r, r);
46

```

```

43         l = t;
44     }
45     pull(t);
46 }
47
48 pNode merge(pNode l, pNode r) {
49     if (!l || !r) return l ? l : r;
50     if (l->prior > r->prior) {
51         l->r = merge(l->r, r);
52         pull(l);
53         return l;
54     }
55     r->l = merge(l, r->l);
56     pull(r);
57     return r;
58 }
59
60 void insert(int pos, int val) { // insert before pos
61     pNode L, R;
62     split(root, pos, L, R);
63     root = merge(merge(L, new Node(val)), R);
64 }
65
66 void erase(int pos) {
67     pNode A, B, C;
68     split(root, pos, A, B);
69     split(B, 1, B, C); // B is erased node
70     root = merge(A, C);
71 }
72
73 long long rangeSum(int l, int r) {
74     pNode A, B, C;
75     split(root, r + 1, B, C);
76     split(B, l, A, B);
77     long long ans = sum(B);
78     root = merge(merge(A, B), C);
79     return ans;
80 }
81 };
82
83 // Example of use:
84 // ImplicitTreap it;
85 // it.insert(0, 5); it.insert(1, 7); it.insert(1, 3); // [5,3,7]
86 // long long s = it.rangeSum(0, 2); // 15

```

6.7 Iterative Segment Tree

```

1  class SegmentTree{
2  private:
3      vector<int> segtree;

```

```

4   int n;
5   int d;
6
7 public:
8
9   void build(vector<int> &a){
10      n = a.size();
11      d=1;
12      while(d<n) d<<=1;
13      segtree.assign(2*d, 0);
14      for(int i=0; i<n; i++)
15          segtree[d+i] = 1;
16
17      for(int i=d-1; i>=1; i--)
18          segtree[i] = segtree[2*i] + segtree[2*i+1];
19  }
20
21  int find_first(int x){
22      if(segtree[1]<x) return -1;
23      int i=1;
24
25      while(i<d){
26          if(segtree[2*i] >= x) i = 2*i;
27          else i = 2*i+1;
28      }
29
30      return i-d;
31  }
32
33
34  int find_kth(int x){
35      //if(segtree[1]<x) return -1;
36      int i=1;
37
38      while(i<d){
39          if(segtree[2*i] >= x) i = 2*i;
40          else{
41              x -= segtree[2*i];
42              i = 2*i+1;
43          }
44      }
45      return i-d;
46  }
47
48
49  void update(int pos){
50      pos+=d;
51      segtree[pos] = 0;
52
53      while(pos>1){
54          pos>>=1;
55          segtree[pos] = segtree[2*pos] + segtree[2*pos+1];
56      }

```

```

57  }
58
59
60  int query(int l, int r){
61      l+=d;
62      r+=d;
63
64      int mx = INT_MIN;
65      while(l<=r){
66          if(l%2 == 1) mx = max(mx, segtree[l++]);
67          if(r%2 == 0) mx = max(mx, segtree[r--]);
68          l>>=1;
69          r>>=1;
70      }
71      return mx;
72  }
73
74  };

```

6.8 Lazy Segment Tree (Max)

```

1  enum QueryType{ MAX, NONE };
2
3  struct Query{
4      QueryType type = NONE;
5      ll val = 0;
6  };
7
8  template <typename T> class LazySegTree {
9  private:
10     const int n;
11     vector<T> tree;
12     vector<Query> lazy;
13
14     void build(int v, int l, int r, const vector<T> &a){
15         if(l==r) tree[v] = a[l];
16         else{
17             int m = (l+r)/2;
18             build(2*v, l, m, a);
19             build(2*v+1, m+1, r, a);
20             tree[v] = max(tree[2*v], tree[2*v+1]);
21         }
22     }
23
24     void apply(int v, int len, const Query &x){
25         tree[v] += x.val;
26         lazy[v].val += x.val;
27         lazy[v].type = MAX;
28     }
29

```



```

30 void push(int v, int l, int r){
31     int m = (l+r)/2;
32     apply(2*v, m-1+1, lazy[v]);
33     apply(2*v+1, r-m, lazy[v]);
34     lazy[v] = Query();
35 }
36
37 void update(int v, int l, int r, int ql, int qr, const Query
38     &x){
39     if(qr < l or ql > r) return;
40
41     if(ql<=l and r <= qr) apply(v,r-1+1,x);
42     else{
43         push(v,l,r);
44         int m = (l+r)/2;
45         update(2*v,l,m,ql,qr,x);
46         update(2*v+1,m+1,r,ql,qr,x);
47         tree[v] = max(tree[2*v], tree[2*v+1]);
48     }
49
50 T query(int v, int l, int r, int ql, int qr){
51     if(qr < l || ql > r) return LLONG_MIN;
52     if(l>=ql and r<=qr) return tree[v];
53     push(v,l,r);
54     int m = (l+r)/2;
55
56     return max(query(2*v, l, m, ql, qr), query(2*v+1,m+1,r,
57         ql,qr));
58 }
59
60 public:
61 LazySegTree(const vector<T> &a) : n(a.size()-1), tree(8 * (a
62     .size()-1)), lazy(8 * (a.size()-1)){
63     build(1, 0, n, a);
64 }
65
66 void update(int ql, int qr, const Query &x){
67     update(1,0, n,ql,qr,x);
68 }
69
70 T query(int ql, int qr){ return query(1,0,n,ql,qr);};
};

```

```

4     QueryType type = NONE;
5     ll val = 0;
6 };
7
8 template <typename T> class LazySegTree {
9     private:
10         const int n;
11         vector<T> tree;
12         vector<Query> lazy;
13
14     void build(int v, int l, int r, const vector<T> &a){
15         if(l==r) tree[v] = a[l];
16         else{
17             int m = (l+r)/2;
18             build(2*v, l, m, a);
19             build(2*v+1, m+1, r, a);
20             tree[v] = tree[2*v] + tree[2*v+1];
21         }
22     }
23
24     void apply(int v, int len, const Query &x){
25         if(x.type == ADD){
26             if(lazy[v].type != SET)
27                 lazy[v] = Query{ADD, lazy[v].val + x.val};
28             else
29                 lazy[v] = Query{SET, lazy[v].val + x.val};
30
31             tree[v] += x.val*len;
32         }else if(x.type == SET){
33             tree[v] = x.val*len;
34             lazy[v] = x;
35         }
36     }
37
38     void push(int v, int l, int r){
39         int m = (l+r)/2;
40         apply(2*v, m-1+1, lazy[v]);
41         apply(2*v+1, r-m, lazy[v]);
42         lazy[v] = Query();
43     }
44
45     void update(int v, int l, int r, int ql, int qr, const Query
46         &x){
47         if(qr < l or ql > r) return;
48
49         if(ql<=l and r <= qr) apply(v,r-1+1,x);
50         else{
51             push(v,l,r);
52             int m = (l+r)/2;
53             update(2*v,l,m,ql,qr,x);
54             update(2*v+1,m+1,r,ql,qr,x);
55             tree[v] = tree[2*v] + tree[2*v+1];
56         }
57     }
58 }

```

6.9 Lazy Segment Tree (Sum)

```

1 enum QueryType{ ADD, SET, NONE };
2
3 struct Query{

```

```

56     }
57
58     T query(int v, int l, int r, int ql, int qr){
59         if(qr < l || ql > r) return 0;
60         if(l>=ql and r<=qr) return tree[v];
61         push(v,l,r);
62         int m = (l+r)/2;
63
64         return query(2*v, l, m, ql, qr) + query(2*v+1,m+1,r,ql,
65             qr);
66     }
67
68     public:
69     LazySegTree(const vector<T> &a) : n(a.size()), tree(4 * n),
70         lazy(4 * n){
71         build(1, 0, n - 1, a);
72     }
73
74     void update(int ql, int qr, const Query &x){
75         update(1,0, n-1,ql,qr,x);
76     }
77
78     T query(int ql, int qr){ return query(1,0,n-1,ql,qr);};
79 };

```

6.10 Lazy Treap

```

1 // Lazy Implicit Treap (range add, range sum, range reverse)
2 struct LazyTreap {
3     static int nextPriority() {
4         static mt19937 rng(chrono::steady_clock::now().
5             time_since_epoch().count());
6         static uniform_int_distribution<int> dist(INT_MIN, INT_MAX);
7         return dist(rng);
8     }
9
10    struct Node {
11        int val;
12        int prior;
13        int sz;
14        long long sum;
15        long long lazyAdd;
16        bool rev;
17        Node *l, *r;
18
19        explicit Node(int v)
20            : val(v), prior(nextPriority()), sz(1), sum(v), lazyAdd
21              (0), rev(false), l(nullptr), r(nullptr) {}
22    };
23 };

```

```

21
22     using pNode = Node*;
23     pNode root = nullptr;
24
25     int sz(pNode t) { return t ? t->sz : 0; }
26     long long sum(pNode t) { return t ? t->sum : 0; }
27
28     void applyAdd(pNode t, long long add) {
29         if (!t) return;
30         t->val += add;
31         t->sum += add * sz(t);
32         t->lazyAdd += add;
33     }
34
35     void applyRev(pNode t) {
36         if (!t) return;
37         t->rev ^= 1;
38         swap(t->l, t->r);
39     }
40
41     void push(pNode t) {
42         if (!t) return;
43         if (t->rev) {
44             applyRev(t->l);
45             applyRev(t->r);
46             t->rev = false;
47         }
48         if (t->lazyAdd != 0) {
49             applyAdd(t->l, t->lazyAdd);
50             applyAdd(t->r, t->lazyAdd);
51             t->lazyAdd = 0;
52         }
53     }
54
55     void pull(pNode t) {
56         if (!t) return;
57         t->sz = 1 + sz(t->l) + sz(t->r);
58         t->sum = t->val + sum(t->l) + sum(t->r);
59     }
60
61     void split(pNode t, int leftSize, pNode &l, pNode &r) {
62         if (!t) {
63             l = r = nullptr;
64             return;
65         }
66         push(t);
67         int curLeft = sz(t->l);
68         if (leftSize <= curLeft) {
69             split(t->l, leftSize, l, t->l);
70             r = t;
71         } else {
72             split(t->r, leftSize - curLeft - 1, t->r, r);
73             l = t;

```

```

74     }
75     pull(t);
76 }
77
78 pNode merge(pNode l, pNode r) {
79     if (!l || !r) return l ? l : r;
80     if (l->prior > r->prior) {
81         push(l);
82         l->r = merge(l->r, r);
83         pull(l);
84         return l;
85     }
86     push(r);
87     r->l = merge(l, r->l);
88     pull(r);
89     return r;
90 }
91
92 void insert(int pos, int val) {
93     pNode L, R;
94     split(root, pos, L, R);
95     root = merge(merge(L, new Node(val)), R);
96 }
97
98 void rangeAdd(int l, int r, long long add) {
99     pNode A, B, C;
100    split(root, r + 1, B, C);
101    split(B, l, A, B);
102    applyAdd(B, add);
103    root = merge(merge(A, B), C);
104 }
105
106 void rangeReverse(int l, int r) {
107     pNode A, B, C;
108     split(root, r + 1, B, C);
109     split(B, l, A, B);
110     applyRev(B);
111     root = merge(merge(A, B), C);
112 }
113
114 long long rangeSum(int l, int r) {
115     pNode A, B, C;
116     split(root, r + 1, B, C);
117     split(B, l, A, B);
118     long long ans = sum(B);
119     root = merge(merge(A, B), C);
120     return ans;
121 }
122 };
123
124 // Example of use:
125 // LazyTreap lt;
126 // lt.insert(0, 1); lt.insert(1, 2); lt.insert(2, 3); // [1,2,3]

```

```

127 // lt.rangeAdd(0, 1, 5); // [6,7,3]
128 // lt.rangeReverse(0, 2); // [3,7,6]
129 // long long s = lt.rangeSum(0, 2); // 16

```

6.11 Merge Sort Tree

```

1  class MergeSortTree{
2  private:
3      vector<vector<int>> segtree;
4      int n;
5      int d;
6
7  public:
8      void build(vector<int> &a){
9          n = a.size();
10         d=1;
11         while(d<n) d<=<=1;
12         segtree.assign(2*d, {0});
13         for(int i=0; i<n; i++)
14             segtree[d+i] = {a[i]};
15
16         for(int i=d-1; i>=1; i--){
17             segtree[i].resize(segtree[2*i].size() + segtree[2*i].
18                             size());
19             merge(segtree[2*i].begin(), segtree[2*i].end(),
20                 segtree[2*i+1].begin(), segtree[2*i+1].end(),
21                 segtree[i].begin());
22         }
23     }
24
25     int query(int l, int r, int x){
26         l+=d;
27         r+=d;
28
29         int arns = 0;
30         while(l<=r){
31             if(l%2 == 1){
32                 arns += lower_bound(segtree[l].begin(), segtree[l].
33                                     end(), x) - segtree[l].begin();
34                 l++;
35             }
36             if(r%2 == 0){
37                 arns += lower_bound(segtree[r].begin(), segtree[r].
38                                     end(), x) - segtree[r].begin();
39                 r--;
40             }
41             l>>=1;
42             r>>=1;
43         }
44     }
45 }

```

```

42     return arns;
43 }
44
45 };

```

6.12 Ordered Set

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3
4 #define pii pair<int,int>
5
6 using namespace std;
7 using namespace __gnu_pbds;
8
9 #define int long long
10
11 const int MAX = 2e5+1;
12 const int INF = LLONG_MAX;
13
14 template<typename T> using Tree = tree<T, null_type, less<T>,
15     rb_tree_tag, tree_order_statistics_node_update>;
16
17 void solve(){
18     int n,q; cin>>n>>q;
19     vector<int> v(n);
20     Tree<pii> oset;
21
22     for(int &x : v) cin>>x;
23
24     for(int i=0; i<n; i++){
25         oset.insert({v[i],i});
26     }
27
28     for(int i=0; i<q; i++){
29         char ty; cin>>ty;
30         if(ty == '!'){
31             int k, s; cin>>k>>s;
32             k--;
33             oset.erase({v[k],k});
34             v[k] = s;
35             oset.insert({v[k],k});
36         }else{
37             int a, b; cin>>a>>b;
38             cout<<oset.order_of_key({b,INF}) - oset.order_of_key({a-1,INF})<<endl;
39         }
40     }
41 }

```

6.13 Recursive Segment Tree

```

1 ll t[4*MAX];
2
3 // Shout-out to CP algo for the SegTree implementation: https://cp-
4 // algorithms.com/data_structures/segment_tree.html#memory-
5 // efficient-implementation
6
7 void buildSegTree(vector<ll> &a, int v, int tl, int tr) {
8     if (tl == tr) {
9         t[v] = a[tl];
10    } else {
11        int tm = (tl + tr) / 2;
12        buildSegTree(a, v*2, tl, tm);
13        buildSegTree(a, v*2+1, tm+1, tr);
14        t[v] = t[v*2] + t[v*2+1];
15    }
16 }
17
18 ll sum(int v, int tl, int tr, int l, int r) {
19     if (l > r)
20         return 0;
21     if (l == tl && r == tr) {
22         return t[v];
23     }
24     int tm = (tl + tr) / 2;
25     return sum(v*2, tl, tm, l, min(r, tm))
26         + sum(v*2+1, tm+1, tr, max(l, tm+1), r);
27 }
28
29 void update(int v, int tl, int tr, int pos, ll new_val) {
30     if (tl == tr) {
31         t[v] = new_val;
32     } else {
33         int tm = (tl + tr) / 2;
34         if (pos <= tm)
35             update(v*2, tl, tm, pos, new_val);
36         else
37             update(v*2+1, tm+1, tr, pos, new_val);
38         t[v] = t[v*2] + t[v*2+1];
39     }
40 }

```

6.14 Treap

```

1 // Treap (BST by key + heap by priority)
2 struct Treap {
3     static int nextPriority() {

```

```

4      static mt19937 rng(chrono::steady_clock::now().
5          time_since_epoch().count());
6      static uniform_int_distribution<int> dist(INT_MIN, INT_MAX);
7      return dist(rng);
8  }
9
10     struct Node {
11         int key;
12         int prior;
13         int cnt;
14         Node *l, *r;
15
16         explicit Node(int k) : key(k), prior(nextPriority()), cnt(1)
17             , l(nullptr), r(nullptr) {}
18     };
19
20     using pNode = Node*;
21     pNode root = nullptr;
22
23     int cnt(pNode t) { return t ? t->cnt : 0; }
24
25     void pull(pNode t) {
26         if (!t) return;
27         t->cnt = 1 + cnt(t->l) + cnt(t->r);
28     }
29
30     void split(pNode t, int key, pNode &l, pNode &r) {
31         if (!t) {
32             l = r = nullptr;
33             return;
34         }
35         if (t->key <= key) {
36             split(t->r, key, t->r, r);
37             l = t;
38         } else {
39             split(t->l, key, l, t->l);
40             r = t;
41         }
42         pull(t);
43     }
44
45     pNode merge(pNode l, pNode r) {
46         if (!l || !r) return l ? l : r;
47         if (l->prior > r->prior) {
48             l->r = merge(l->r, r);
49             pull(l);
50             return l;
51         }
52         r->l = merge(l, r->l);
53         pull(r);
54         return r;
55     }

```

```

55     bool contains(pNode t, int key) {
56         while (t) {
57             if (t->key == key) return true;
58             if (key < t->key) t = t->l;
59             else t = t->r;
60         }
61         return false;
62     }
63
64     void insert(int key) {
65         if (contains(root, key)) return; // set behavior (no
66             duplicates)
67         pNode L, R;
68         split(root, key, L, R);
69         root = merge(merge(L, new Node(key)), R);
70     }
71
72     void erase(int key) {
73         pNode A, B, C;
74         split(root, key, A, C);
75         split(A, key - 1, A, B); // B contains only key (if exists)
76         // delete subtree B if needed for memory cleanup
77         root = merge(A, C);
78     }
79
80     int order_of_key(int key) { // count of elements < key
81         pNode t = root;
82         int ans = 0;
83         while (t) {
84             if (key <= t->key) {
85                 t = t->l;
86             } else {
87                 ans += 1 + cnt(t->l);
88                 t = t->r;
89             }
90         }
91         return ans;
92     }
93
94     int find_by_order(int k) { // 0-indexed kth smallest
95         pNode t = root;
96         while (t) {
97             int left = cnt(t->l);
98             if (k < left) t = t->l;
99             else if (k == left) return t->key;
100             else {
101                 k -= left + 1;
102                 t = t->r;
103             }
104         }
105         return -1; // invalid k
106     }

```

```

107
108 // Example of use:
109 // Treap tr;
110 // tr.insert(10); tr.insert(4); tr.insert(20);
111 // int x = tr.find_by_order(1); // 10
112 // int y = tr.order_of_key(15); // 2

```

6.15 Cycle Detection Online

```

1 class CycleDetectionDSU {
2     vector<int> parent;
3     vector<int> size;
4
5 public:
6     CycleDetectionDSU(int n) : parent(n), size(n, 1) {
7         iota(parent.begin(), parent.end(), 0);
8     }
9
10    int find(int x) {
11        return parent[x] == x ? x : parent[x] = find(parent[x]);
12    }
13
14    // Returns true if adding edge u-v creates a cycle
15    bool add_edge(int u, int v) {
16        int u_root = find(u);
17        int v_root = find(v);
18        if (u_root == v_root) return true;
19
20        if (size[u_root] < size[v_root]) swap(u_root, v_root);
21        parent[v_root] = u_root;
22        size[u_root] += size[v_root];
23        return false;
24    }
25 };

```

6.16 DSU with Rollbacks

```

1 class DSU {
2     private:
3         vector<int> p, sz;
4         // stores previous unites
5         vector<pair<int &, int>> history;
6
7     public:
8         DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
9
10        int get(int x) { return x == p[x] ? x : get(p[x]); }

```

```

11
12    void unite(int a, int b) {
13        a = get(a);
14        b = get(b);
15        if (a == b) { return; }
16        if (sz[a] < sz[b]) { swap(a, b); }
17
18        // save this unite operation
19        history.push_back({sz[a], sz[a]});
20        history.push_back({p[b], p[b]});
21
22        p[b] = a;
23        sz[a] += sz[b];
24    }
25
26    int snapshot() { return history.size(); }
27
28    void rollback(int until) {
29        while (snapshot() > until) {
30            history.back().first = history.back().second;
31            history.pop_back();
32        }
33    }
34 };

```

6.17 Fenwick Lazy

```

1 // Range Update - Range Query (1-based indexing)
2 struct FenwickRURQ {
3     int n;
4     std::vector<int> B1, B2;
5
6     FenwickRURQ(int size) : n(size + 1), B1(size + 2), B2(size + 2) {}
7
8     // Add val to range [l, r] (1-based)
9     void range_add(int l, int r, int val) {
10         add(B1, l, val);
11         add(B1, r + 1, -val);
12         add(B2, l, val * (l - 1));
13         add(B2, r + 1, -val * r);
14     }
15
16     // Get sum of range [l, r] (1-based)
17     int range_sum(int l, int r) {
18         return prefix_sum(r) - prefix_sum(l - 1);
19     }
20
21     private:
22         void add(std::vector<int>& b, int idx, int val) {

```

```

23     for(; idx < n; idx += idx & -idx)
24         b[idx] += val;
25 }
26
27 int sum(const std::vector<int>& b, int idx) {
28     int total = 0;
29     for(; idx > 0; idx -= idx & -idx)
30         total += b[idx];
31     return total;
32 }
33
34 int prefix_sum(int idx) {
35     return sum(B1, idx) * idx - sum(B2, idx);
36 }
37 };

```

6.18 Implicit Treap 2

```

1 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
2
3 #define ll long long
4
5 struct TreapNode{
6     ll key, pr, sz;
7     TreapNode *l, *r;
8 };
9
10 typedef TreapNode* Treap;
11
12 int getSize(Treap &t){
13     return t ? t->sz : 0;
14 }
15
16 void updateSize(Treap &t){
17     if (t) t->sz = 1 + getSize(t->l) + getSize(t->r);
18 }
19
20 void split(Treap& t, ll k, Treap &l, Treap &r){
21     if(not t) l = r = nullptr;
22
23     else if(k < t->key){
24         split(t->l, k, l, t->l);
25         r = t;
26         updateSize(r);
27     }else{
28         split(t->r, k, t->r, r);
29         l = t;
30         updateSize(l);
31     }
32 }

```

```

33
34 void insert(Treap& t, Treap a){
35     if(not t) t=a;
36     else if(a->pr > t->pr){
37         split(t, a->key, a->l, a->r);
38         t = a;
39     }else{
40         if(a->key < t->key) insert(t->l, a);
41         else insert(t->r, a);
42     }
43     updateSize(t);
44 }
45
46 void merge(Treap &t, Treap l, Treap r){
47     if(not l) t = r;
48     else if(not r) t = l;
49
50     else if(l->pr > r->pr){
51         merge(l->r, l->r, r);
52         t=l;
53         updateSize(t);
54     }else{
55         merge(r->l, l, r->l);
56         t=r;
57         updateSize(t);
58     }
59 }
60
61 void erase(Treap &t, ll k){
62     if(not t) return;
63     if(t->key == k) merge(t, t->l, t->r);
64
65     else{
66         if(k < t->key) erase(t->l, k);
67         else erase(t->r, k);
68     }
69     updateSize(t);
70 }
71
72 bool find(Treap& t, ll k){
73     if (not t) return false;
74     if(t->key == k) return true;
75     if(k < t->key) return find(t->l, k);
76     return find(t->r, k);
77 }
78
79 void insertValue(Treap &t, ll k){
80     if(not find(t, k)){
81         Treap new_node = new TreapNode {k, rng(), 0, nullptr, nullptr};
82         insert(t, new_node);
83     }
84 }

```

```

85
86 ll getKth(Treap &t, int k){
87     if(!t || k<=0 || k>getSize(t)) return 0;
88     int leftSize = getSize(t->l);
89     if(k == leftSize+1) return t->key;
90     if(k <= leftSize) return getKth(t->l,k);
91     return getKth(t->r, k-leftSize-1);
92 }

```

6.19 Online Bipartiteness

```

1 void make_set(int v) {
2     parent[v] = make_pair(v, 0);
3     rank[v] = 0;
4     bipartite[v] = true;
5 }
6
7 pair<int, int> find_set(int v) {
8     if (v != parent[v].first) {
9         int parity = parent[v].second;
10        parent[v] = find_set(parent[v].first);
11        parent[v].second ^= parity;
12    }
13    return parent[v];
14 }
15
16 void add_edge(int a, int b) {
17     pair<int, int> pa = find_set(a);
18     a = pa.first;
19     int x = pa.second;
20
21     pair<int, int> pb = find_set(b);
22     b = pb.first;
23     int y = pb.second;
24
25     if (a == b) {
26         if (x == y)
27             bipartite[a] = false;
28     } else {
29         if (rank[a] < rank[b])
30             swap (a, b);
31         parent[b] = make_pair(a, x^y^1);
32         bipartite[a] ^= bipartite[b];
33         if (rank[a] == rank[b])
34             ++rank[a];
35     }
36 }
37
38 bool is_bipartite(int v) {
39     return bipartite[find_set(v).first];

```

```

40 }

```

6.20 Path Copy Persistent Array

```

1 struct Node {
2     int val;
3     Node *l, *r;
4
5     Node(ll x) : val(x), l(nullptr), r(nullptr) {}
6     Node(Node *ll, Node *rr) : val(0), l(ll), r(rr) {}
7 };
8
9 int n, a[100001];    // The initial array and its size
10 Node *roots[100001]; // The persistent array's roots
11
12 Node *build(int l = 0, int r = n - 1) {
13     if (l == r) return new Node(a[l]);
14     int mid = (l + r) / 2;
15     return new Node(build(l, mid), build(mid + 1, r));
16 }
17
18 Node *update(Node *node, int val, int pos, int l = 0, int r = n - 1)
19 {
20     if (l == r) return new Node(val);
21     int mid = (l + r) / 2;
22     if (pos > mid) return new Node(node->l, update(node->r, val, pos,
23         mid + 1, r));
24     else return new Node(update(node->l, val, pos, l, mid), node->r);
25 }
26
27 int query(Node *node, int pos, int l = 0, int r = n - 1) {
28     if (l == r) return node->val;
29     int mid = (l + r) / 2;
30     if (pos > mid) return query(node->r, pos, mid + 1, r);
31     return query(node->l, pos, l, mid);
32 }
33
34 int get_item(int index, int time) {
35     // Gets the array item at a given index and time
36     return query(roots[time], index);
37 }
38
39 void update_item(int index, int value, int prev_time, int curr_time)
40 {
41     // Updates the array item at a given index and time
42     roots[curr_time] = update(roots[prev_time], index, value);
43 }
44
45 void init_arr(int nn, int *init) {
46     // Initializes the persistent array, given an input array

```



```

44  n = nn;
45  for (int i = 0; i < n; i++) a[i] = init[i];
46  roots[0] = build();
47  }

```

6.21 Persistent Array

```

1  vector<pair<int, int>> arr[100001]; // The persistent array
2
3  int get_item(int index, int time) {
4      // Gets the array item at a given index and time
5      auto ub =
6          upper_bound(arr[index].begin(), arr[index].end(), make_pair(
7              time, INT_MAX));
8      return prev(ub)->second;
9  }
10 void update_item(int index, int value, int time) {
11     // Updates the array item at a given index and time
12     // Note that this only works if the time is later than all
13     // previous
14     // update times
15     assert(arr[index].back().first < time);
16     arr[index].push_back({time, value});
17 }
18 void init_arr(int n, int *init) {
19     // Initializes the persistent array, given an input array
20     for (int i = 0; i < n; i++) arr[i].push_back({0, init[i]});
21 }

```

6.22 Persistent Segment Tree

```

1  using ll = long long;
2
3  class PersistentSegtree {
4  private:
5      struct Node {
6          ll sum = 0;
7          int l = 0, r = 0;
8      };
9
10     const int n;
11     vector<Node> tree;
12     int timer = 1;
13
14     Node join(int l, int r) { return Node{tree[l].sum + tree[r].sum, l
15         , r}; }

```

```

15
16 int build(int tl, int tr, const vector<int> &arr) {
17     if (tl == tr) {
18         tree[timer] = {arr[tl], 0, 0};
19         return timer++;
20     }
21
22     int mid = (tl + tr) / 2;
23     tree[timer] = join(build(tl, mid, arr), build(mid + 1, tr, arr))
24         ;
25     return timer++;
26 }
27
28 int set(int v, int pos, int val, int tl, int tr) {
29     if (tl == tr) {
30         tree[timer] = {val, 0, 0};
31         return timer++;
32     }
33
34     int mid = (tl + tr) / 2;
35     if (pos <= mid) {
36         tree[timer] = join(set(tree[v].l, pos, val, tl, mid), tree[v].
37             r);
38     } else {
39         tree[timer] = join(tree[v].l, set(tree[v].r, pos, val, mid +
40             1, tr));
41     }
42     return timer++;
43 }
44
45 ll range_sum(int v, int ql, int qr, int tl, int tr) {
46     if (qr < tl || tr < ql) { return 0ll; }
47     if (ql <= tl && tr <= qr) { return tree[v].sum; }
48
49     int mid = (tl + tr) / 2;
50     return range_sum(tree[v].l, ql, qr, tl, mid) +
51         range_sum(tree[v].r, ql, qr, mid + 1, tr);
52 }
53
54 public:
55 PersistentSegtree(int n, int MX_NODES) : n(n), tree(MX_NODES) {}
56
57 int build(const vector<int> &arr) { return build(0, n - 1, arr); }
58
59 int set(int root, int pos, int val) { return set(root, pos, val,
60     0, n - 1); }
61
62 ll range_sum(int root, int l, int r) { return range_sum(root, l, r
63     , 0, n - 1); }
64
65 int add_copy(int root) {

```

```

63     tree[timer] = tree[root];
64     return timer++;
65 }
66 };

```

6.23 Prefix Suffix Segment Tree

```

1  typedef long long ll;
2  typedef vector<int> vec;
3  typedef vector<pair<int,int>> vpai;
4  const ll mod=1e9+7;
5  const int MAX=1e5+3;
6  const int limit=2e5+3;
7  const int TAM=2e5+1;
8  ll t[4*TAM];
9  ll prefix[4*TAM],suffix[4*TAM],biggest[4*TAM];
10 //ascii https://elcodigoascii.com.ar/
11 ll cero=0;
12 void build(int root,int l,int r,vector<ll> &arr)
13 {
14     if(l==r)
15     {
16         t[root]=arr[l];
17         suffix[root]=max(t[root],cero);
18         prefix[root]=max(t[root],cero);
19         biggest[root]=max(t[root],cero);
20         return;
21     }
22     int mid=(l+r)/2;
23     build(2*root,l,mid,arr);
24     build(2*root+1,mid+1,r,arr);
25     t[root]=t[2*root]+t[2*root+1];
26     biggest[root]=max(biggest[2*root],
27 max(biggest[2*root+1],suffix[2*root]+prefix[2*root+1]));
28     prefix[root]=max(prefix[2*root],t[2*root]+prefix[2*root+1]);
29     suffix[root]=max(suffix[2*root+1],t[2*root+1]+suffix[2*root]);
30 }
31 }
32 void update(int root,int l,int r,int pos,ll val)
33 {
34     if(l==r)
35     {
36         t[root]=val;
37         suffix[root]=max(cero,t[root]);
38         prefix[root]=max(cero,t[root]);
39         biggest[root]=max(t[root],cero);
40         return;
41     }
42     int mid=(l+r)/2;

```

```

44     if(pos>mid)
45     {
46         update(2*root+1,mid+1,r,pos,val);
47     }
48     else{
49         update(2*root,l,mid,pos,val);
50     }
51     t[root]=t[2*root]+t[2*root+1];
52     biggest[root]=max(biggest[2*root],
53 max(biggest[2*root+1],suffix[2*root]+prefix[2*root+1]));
54     prefix[root]=max(prefix[2*root],t[2*root]+prefix[2*root+1]);
55     suffix[root]=max(suffix[2*root+1],t[2*root+1]+suffix[2*root]);
56 }
57 ll consult(int root,int l,int r, int a,int b)
58 {
59     if(a>b) return 0;
60     if(l==a && r==b) return t[root];
61     int mid=(l+r)/2;
62     return consult(2*root,l,mid,a,min(b,mid))+
63 consult(2*root+1,mid+1,r,max(mid+1,a),b);
64 }
65 }

```

6.24 Wicho Lazy Segment Tree

```

1  typedef long long ll;
2  typedef vector<int> vec;
3  typedef vector<pair<int,int>> vpai;
4  const ll mod=1e9+7;
5  const int MAX=1e5+3;
6  const int limit=2e5+3;
7  const int TAM=2e5+1;
8  ll t[4*TAM];
9  ll op[4*TAM];
10 int type[4*TAM];
11 //ascii https://elcodigoascii.com.ar/
12 void propagate(int root,int l,int r)
13 {
14     if(type[root]==1)
15     {
16         t[root]+=op[root]*(r+1-l);
17         if(l!=r){
18             op[2*root]+=op[root];
19             op[2*root+1]+=op[root];
20             type[2*root+1]=max(1,type[2*root+1]);
21             type[2*root]=max(1,type[2*root]);
22         }
23     }
24     else
25

```

```

26 {
27     if(type[root]==2){
28         t[root]=op[root]*(r+1-1);
29         if(l!=r){
30             op[2*root]=op[root];
31             op[2*root+1]=op[root];
32             type[2*root+1]=2;
33             type[2*root]=2;
34         }
35     }
36 }
37 op[root]=0;
38 type[root]=0;
39 }
40
41 void build(int root,int l,int r,vector<ll> &arr)
42 {
43     if(l==r)
44     {
45         t[root]=arr[l];
46         op[root]=0;
47         type[root]=0;
48         return;
49     }
50     int mid=(l+r)/2;
51     build(2*root,l,mid,arr);
52     build(2*root+1,mid+1,r,arr);
53     t[root]=t[2*root]+t[2*root+1];
54     op[root]=0;
55     type[root]=0;
56 }
57
58 void sum(int root,int l,int r,int a,int b,ll val)
59 {
60     propagate(root,l,r);
61     if(a>b) return;
62     if(l==a && r==b)
63     {
64         op[root]=val;
65         type[root]=1;
66         propagate(root,l,r);
67         return;
68     }
69     int mid=(l+r)/2;
70     sum(2*root,l,mid,a,min(b,mid),val);
71     sum(2*root+1,mid+1,r,max(mid+1,a),b,val);
72     t[root]=t[2*root]+t[2*root+1];
73 }
74
75 void setR(int root,int l,int r,int a,int b,ll val)
76 {
77     propagate(root,l,r);
78     if(a>b) return;

```

```

79     if(l==a && r==b)
80     {
81         op[root]=val;
82         type[root]=2;
83         propagate(root,l,r);
84         return;
85     }
86     int mid=(l+r)/2;
87     setR(2*root,l,mid,a,min(b,mid),val);
88     setR(2*root+1,mid+1,r,max(mid+1,a),b,val);
89     t[root]=t[2*root]+t[2*root+1];
90 }
91
92 ll consult(int root,int l,int r, int a,int b)
93 {
94     propagate(root,l,r);
95     if(a>b) return 0;
96     if(l==a && r==b){
97         return t[root];
98     }
99     int mid=(l+r)/2;
100     return consult(2*root,l,mid,a,min(b,mid))+
101     consult(2*root+1,mid+1,r,max(mid+1,a),b);
102 }

```

6.25 Li Chao Tree

```

1 typedef long long ll;
2 typedef complex<ll> pt;
3 typedef vector<int> vec;
4
5 const ll mod=1e9+7;
6 const ll MAX=1e5+3;
7 const ll INF=1e18;
8
9 pt lines[4*MAX];
10
11 //Li Chao Tree
12
13 ll dot(pt a,pt b){
14     return a.X*b.X+a.Y*b.Y;
15 }
16
17 ll f(pt line,ll val){
18     return dot(line,pt{val,1});
19 }
20
21 void addLine(pt cur,int root=1,int l=0,int r=MAX)
22 {
23

```

```

24     int m=(l+r)/2;
25     bool left_border=f(cur,l)>f(lines[root],l);
26     bool mid=f(cur,m)>f(lines[root],m);
27
28     if(mid){
29         swap(lines[root],cur);
30     }
31     if(r-l==1) return;
32     else if(left_border!=mid){
33         addLine(cur,2*root,l,m);
34     }
35     else{
36         addLine(cur,2*root+1,m,r);
37     }
38 }
39
40 ll maximum(int x,int root=1,int l=0,int r=MAX)
41 {
42     int mid=(l+r)/2;
43
44     if(r-l==1){
45         return f(lines[root],x);
46     }
47     if(x<mid){
48         return max(f(lines[root],x),maximum(x,2*root,l,mid));
49     }
50     return max(f(lines[root],x),maximum(x,2*root+1,mid,r));
51 }
52
53 void addSegment(pt cur,int a,int b,int root=1,int l=0,int r=MAX)
54 {
55     if(a>b) return;
56
57     if(a==l && b==r){
58         addLine(cur,root,l,r);
59         return;
60     }
61     if(r-l==1) return;
62     int mid=(l+r)/2;
63     addSegment(cur,a,min(mid,b),2*root,l,mid);
64     addSegment(cur,max(mid,a),b,2*root+1,mid,r);
65 }

```

```

3     vector<vector<int>> table;
4     vector<int> log;
5     int n;
6
7 public:
8     SparseTable(vector<int>& arr) {
9         n = arr.size();
10        int maxLog = log2(n) + 1;
11        table.assign(n, vector<int>(maxLog));
12        log.assign(n + 1, 0);
13
14        // Precompute logarithms
15        for (int i = 2; i <= n; i++) {
16            log[i] = log[i/2] + 1;
17        }
18
19        // Build sparse table
20        for (int i = 0; i < n; i++) {
21            table[i][0] = arr[i];
22        }
23
24        for (int j = 1; j < maxLog; j++) {
25            for (int i = 0; i + (1 << j) <= n; i++) {
26                table[i][j] = min(table[i][j-1],
27                                   table[i + (1 << (j-1))][j-1]);
28            }
29        }
30    }
31
32    int query(int l, int r) {
33        int j = log[r - l + 1];
34        return min(table[l][j], table[r - (1 << j) + 1][j]);
35    }
36 };

```

7 Range Queries

7.1 Sparse Table

```

1 // Sparse Table for Range Minimum Query
2 class SparseTable {

```

8 Mathematics

8.1 Formula Sheet (Contest)

8.1.1 Combinatorics

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

$$P(n, k) = \frac{n!}{(n-k)!} \quad (\text{ordered, no repetition})$$

$$\text{Combinations with repetition} = \binom{n+k-1}{k}$$

$$\text{Permutations with repetition} = \frac{n!}{a_1!a_2!\cdots a_m!}$$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \quad (\text{Pascal identity})$$

Multinomial coefficient (split n items into groups $k_1, \dots, k_m$):

$$\binom{n}{k_1, k_2, \dots, k_m} = \frac{n!}{k_1!k_2!\cdots k_m!}, \quad \sum_i k_i = n$$

Catalan numbers (balanced parentheses, rooted full binary trees, non-crossing matchings):

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1, \quad C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$$

- **Valid Parentheses:** Count of balanced parentheses expressions with n pairs.
- **Full Binary Trees:** Structurally unique full binary trees with $n+1$ leaves.
- **Polygon Triangulation:** Ways to triangulate a convex $(n+2)$ -gon.
- **Dyck Paths:** Paths from $(0,0)$ to $(2n,0)$ that never dip below the x-axis.
- **Non-Crossing Partitions:** Ways to connect $2n$ points on a circle without crossing chords.

- **Stack Permutations:** Valid stack-sortable permutations of length n .
- **Mountain Ranges:** Sequences of $2n$ up/down steps forming valid mountain ranges.
- **Unique BSTs:** Number of distinct binary search trees with n keys.
- **Diagonal-Avoiding Paths:** Paths in a grid from $(0,0)$ to (n,n) without crossing the diagonal.

8.2 Cayley's Formula

Number of labeled trees of n vertices: n^{n-2} .

Number of rooted forest of n vertices is: $(n+1)^{n-1}$

8.3 Geometric series

Finite:

$$\sum_{k=0}^n ar^k = \begin{cases} a \frac{1-r^{n+1}}{1-r} & \text{if } r \neq 1, \\ a(n+1) & \text{if } r = 1. \end{cases}$$

Infinite:

$$\sum_{k=0}^{\infty} ar^k = \frac{a}{1-r} \quad (\text{converges iff } |r| < 1)$$

8.4 Divisors

The number of divisors of any number n is:

$$\begin{cases} \approx 100 & n < 5 \times 10^4 \\ \approx 500 & n < 1 \times 10^7 \\ \approx 2000 & n < 1 \times 10^{10} \\ \approx 200000 & n < 1 \times 10^{19} \end{cases}$$

8.5 Number of primes between 1 and n

$$\frac{n}{\ln(n)}$$

8.6 Pythagorean triplets

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2)$$

With $m > n > 0$, $k = 0$, $m \perp n$, and either m or n even.

8.7 Derangments

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \lfloor \frac{n!}{e} \rfloor$$

Pascal triangle facts:

$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad \sum_{k=0}^n (-1)^k \binom{n}{k} = 0$$

8.7.1 Sums

Gaussian sums (first powers to fourth powers):

$$\begin{aligned} \sum_{i=1}^n i &= \frac{n(n+1)}{2} \\ \sum_{i=1}^n i^2 &= \frac{n(n+1)(2n+1)}{6} \\ \sum_{i=1}^n i^3 &= \left(\frac{n(n+1)}{2} \right)^2 \\ \sum_{i=1}^n i^4 &= \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30} \end{aligned}$$

Useful finite progressions:

$$\sum_{i=0}^{n-1} ar^i = a \frac{1-r^n}{1-r} \quad (r \neq 1), \quad \sum_{i=0}^{n-1} (a+id) = \frac{n}{2} (2a + (n-1)d)$$

8.7.2 Geometry

For points $A(x_1, y_1), B(x_2, y_2)$:

$$\text{Euclidean}(A, B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

$$\text{Manhattan}(A, B) = |x_1 - x_2| + |y_1 - y_2|$$

$$\text{Chebyshev}(A, B) = \max\{|x_1 - x_2|, |y_1 - y_2|\}$$

For vectors $\vec{a} = (a_x, a_y)$, $\vec{b} = (b_x, b_y)$:

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y$$

$$\vec{a} \times \vec{b} = a_x b_y - a_y b_x$$

Note: In 2D, $\text{sign}(\vec{a} \times \vec{b})$ gives orientation (left/right turn), and $|\vec{a} \times \vec{b}|/2$ is the triangle area.

8.7.3 Probability

$$\mathbb{E}[X] = \sum_x x P(X = x)$$

$$\mathbb{E}[aX + b] = a \mathbb{E}[X] + b$$

$$\mathbb{E}\left[\sum_{i=1}^m X_i\right] = \sum_{i=1}^m \mathbb{E}[X_i] \quad (\text{linearity, no independence needed})$$

Indicator trick (very useful): If I_A is 1 when event A happens and 0 otherwise, then $\mathbb{E}[I_A] = P(A)$. Count expected objects by summing indicators.

8.7.4 Number Theory and Counting Essentials

$$\gcd(a, b) \cdot \text{lcm}(a, b) = ab$$

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

$$a^{\varphi(m)} \equiv 1 \pmod{m} \quad (\gcd(a, m) = 1)$$

Inclusion-exclusion for sets $A_1, \dots, A_m$:

$$\left| \bigcup_i A_i \right| = \sum_i |A_i| - \sum_{i < j} |A_i \cap A_j| + \sum_{i < j < k} |A_i \cap A_j \cap A_k| - \dots$$

Stars and bars (nonnegative solutions of $x_1 + \dots + x_k = n$):

$$\# \text{solutions} = \binom{n+k-1}{k-1}$$

8.8 Binary Exponentiation

```

1 const int MAX = 2e5+1;
2 const int MOD = 1e9+7;
3 int bnpow(int a, int b, int m){
4     int arns = 1;
5     a%=m;
6     while(b>0){
7         if(b&1) arns = (arns*a)%m;
8         a = (a*a)%m;
9         b>>=1;
10    }
11    return arns%m;
12 }
```

8.9 Binomial Coefficients

```

1 vector<ll> fact(MAX);
2 vector<ll> i_fact(MAX);
3
4 ll bnpow(ll a, ll b){
5     ll arns = 1;
6     a%=MOD;
7     while(b>0){
8         if(b&1)
9             arns = (arns*a)%MOD;
10        a = (a*a)%MOD;
11        b>>=1;
12    }
13    return arns;
14 }
15
16 ll modInv(ll a, ll m=MOD){
17     return bnpow(a,MOD-2);
18 }
19
20 void factorial(){
21     fact[0] = fact[1] = 1;
22     for(int i=2; i<MAX; i++){
23         fact[i] = (fact[i-1] * i)%MOD;
24     }
25
26     i_fact[MAX-1] = modInv(fact[MAX-1]);
```

```

27     for(int i=MAX-2; i>=0; i--){
28         i_fact[i] = (i_fact[i+1]*(i+1))%MOD;
29     }
30 }
31
32 ll bn_coef(ll n, ll k){
33     return fact[n] * i_fact[n-k]%MOD * i_fact[k]%MOD;
34 }
35 }
```

8.10 Extended Euclidean

```

1 // Extended Euclidean Algorithm
2 // Returns gcd(a, b) and finds x, y such that ax + by = gcd(a, b)
3 int extgcd(int a, int b, int &x, int &y) {
4     if (b == 0) {
5         x = 1;
6         y = 0;
7         return a;
8     }
9     int x1, y1;
10    int g = extgcd(b, a % b, x1, y1);
11    x = y1;
12    y = x1 - (a / b) * y1;
13    return g;
14 }
```

8.11 FFT

```

1 // MAX should be a power of 2 and >= 2*10^5
2 // FFT works with sizes that are powers of 2
3 // 2^18 = 262144 > 200000
4 const int MAX = pow(2,18);
5
6 typedef complex<double> cd;
7 double PI = acos(-1);
8
9 // My favorite black box <3
10 vector<cd> fft(vector<cd> a, int d = 1) {
11     int n = a.size();
12     vector<cd> r(n);
13
14     for (int k = 0; k < n; k++) {
15         int b = 0;
16         for (int z = 1; z < n; z *= 2) {
17             b *= 2;
18             if (k & z)
```

```

19         b++;
20     }
21     r[b] = a[k];
22 }
23
24 for (int m = 2; m <= n; m *= 2) {
25     cd wm = exp(cd{0, d * 2 * PI / m});
26
27     for (int k = 0; k < n; k += m) {
28         cd w = 1;
29         for (int j = 0; j < m / 2; j++) {
30             cd u = r[k + j];
31             cd t = w * r[k + j + m / 2];
32             r[k + j] = u + t;
33             r[k + j + m / 2] = u - t;
34             w = w * wm;
35         }
36     }
37 }
38
39 if (d == -1) {
40     for (int i = 0; i < n; i++)
41         r[i] /= n;
42 }
43
44 return r;
45 }
46
47 void solve() {
48     int n, m;
49     cin >> n >> m;
50
51     vector<int> s(n), msk(m);
52
53     for (int i = 0; i < n; i++) {
54         cin >> s[i];
55     }
56     for (int i = 0; i < m; i++) {
57         cin >> msk[i];
58     }
59
60     reverse(msk.begin(), msk.end());
61
62     vector<cd> a(2*MAX, cd(0,0)), b(2*MAX, cd(0,0));
63
64     for (int i=0; i<n; i++){
65         a[i]+=cd(s[i],0);
66     }
67
68     for (int i=0; i<m; i++){
69         b[i]+=cd(msk[i],0);
70     }
71 }

```

```

72
73     auto A = fft(a);
74     auto B = fft(b);
75     vector<cd> C(2*MAX);
76
77     for (int i = 0; i < 2*MAX; i++) {
78         C[i] = A[i] * B[i];
79     }
80
81     auto conv = fft(C, -1);
82
83     for (int i = 0; i < n+m-1; i++) {
84         cout << (ll)round(conv[i].real()) << " ";
85     }
86     cout << endl;
87 }

```

8.12 Big Integer

```

1 static const __int128 MOD = (__int128)10000000000000000031ULL;
2 static const __int128 g = (__int128)42ULL;
3
4 ostream &operator<<(ostream &os, const __int128 & value){
5     char buffer[64];
6     char *pos = end(buffer)-1;
7     *pos = '\0';
8     __int128 tmp = value < 0 ? -value : value;
9
10    do{
11        --pos;
12        *pos = tmp%10 + '0';
13        tmp/=10;
14    }while(tmp!=0);
15    if(value < 0){
16        --pos;
17        *pos = '-';
18    }
19
20    return os << pos;
21 }
22
23 inline __int128 mulm(__int128 a, __int128 b){
24     __uint128_t z = ( (__uint128_t)a * (__uint128_t)b );
25     return (__int128)( z % MOD );
26 }
27
28 __int128 binpow(__int128 a, __int128 e){
29     __int128 r = 1;
30     while (e > 0){
31         if (e & 1) r = mulm(r, a);
32         a = mulm(a, a);
33     }
34 }

```



```

32     e >>= 1;
33 }
34 return r;
35 }
36
37 void solve(){
38     int n; cin>>n;
39
40     __int128 p = 300;
41     __int128 cnt = 1;
42     for(ll i = 1000000LL; i>n; i--){
43         p = binpow(g,p);
44     }
45     cout << p << '\n';
46 }

```

8.13 NTT

```

1 // Number Theoretic Transform (NTT)
2 // Usage: exact polynomial convolution under modulo arithmetic (no
3 // floating-point precision issues).
4 // Good for multiplying polynomials / counting ways where answers
5 // are required modulo MOD.
6 // Constraints: n must be a power of 2 and n <= ROOT_PW for this
7 // modulus/root setup.
8
9 const int MOD = 7340033;           // 7 * 2^20 + 1
10 const int ROOT = 5;                // primitive root for MOD
11 const int ROOT_INV = 4404020;      // inverse(ROOT) mod MOD
12 const int ROOT_PW = 1 << 20;      // max power-of-two supported
13
14 int binpow(int a, int b, int m = MOD) {
15     int arns = 1;
16     while (b > 0) {
17         if (b & 1) arns = (int)(1LL * arns * a % m);
18         a = (int)(1LL * a * a % m);
19         b >>= 1;
20     }
21     return arns;
22 }
23
24 void ntt(vector<int> &a, bool invert) {
25     int n = (int)a.size();
26
27     // Bit-reversal permutation.
28     for (int i = 1, j = 0; i < n; i++) {
29         int bit = n >> 1;
30         while (j & bit) {
31             j ^= bit;
32             bit >>= 1;
33         }
34         if (i < j) swap(a[i], a[j]);
35     }
36
37     for (int len = 2; len <= n; len <= 1) {
38         int wlen = invert ? ROOT_INV : ROOT;
39         for (int i = len; i < ROOT_PW; i <= 1) {
40             wlen = (int)(1LL * wlen * wlen % MOD);
41         }
42
43         for (int i = 0; i < n; i += len) {
44             int w = 1;
45             for (int j = 0; j < len / 2; j++) {
46                 int u = a[i + j];
47                 int v = (int)(1LL * a[i + j + len / 2] * w % MOD);
48
49                 a[i + j] = u + v;
50                 if (a[i + j] >= MOD) a[i + j] -= MOD;
51
52                 a[i + j + len / 2] = u - v;
53                 if (a[i + j + len / 2] < 0) a[i + j + len / 2] +=
54                     MOD;
55
56                 w = (int)(1LL * w * wlen % MOD);
57             }
58         }
59     }
60
61     if (invert) {
62         int n_inv = binpow(n, MOD - 2, MOD);
63         for (int &x : a) {
64             x = (int)(1LL * x * n_inv % MOD);
65         }
66     }
67 }
68
69 // Polynomial multiplication modulo MOD with NTT.
70 vector<int> multiply_ntt(vector<int> a, vector<int> b) {
71     int n = 1;
72     while (n < (int)a.size() + (int)b.size() - 1) n <= 1;
73
74     // Must fit this NTT configuration. Use another modulus/CRT if
75     // larger needed.
76     if (n > ROOT_PW) return {};
77
78     a.resize(n);
79     b.resize(n);
80
81     ntt(a, false);
82     ntt(b, false);
83
84     for (int i = 0; i < n; i++) {

```

```

30     }
31     j ^= bit;
32     if (i < j) swap(a[i], a[j]);
33 }
34
35 for (int len = 2; len <= n; len <= 1) {
36     int wlen = invert ? ROOT_INV : ROOT;
37     for (int i = len; i < ROOT_PW; i <= 1) {
38         wlen = (int)(1LL * wlen * wlen % MOD);
39     }
40
41     for (int i = 0; i < n; i += len) {
42         int w = 1;
43         for (int j = 0; j < len / 2; j++) {
44             int u = a[i + j];
45             int v = (int)(1LL * a[i + j + len / 2] * w % MOD);
46
47             a[i + j] = u + v;
48             if (a[i + j] >= MOD) a[i + j] -= MOD;
49
50             a[i + j + len / 2] = u - v;
51             if (a[i + j + len / 2] < 0) a[i + j + len / 2] +=
52                 MOD;
53
54             w = (int)(1LL * w * wlen % MOD);
55         }
56     }
57
58     if (invert) {
59         int n_inv = binpow(n, MOD - 2, MOD);
60         for (int &x : a) {
61             x = (int)(1LL * x * n_inv % MOD);
62         }
63     }
64 }
65
66 // Polynomial multiplication modulo MOD with NTT.
67 vector<int> multiply_ntt(vector<int> a, vector<int> b) {
68     int n = 1;
69     while (n < (int)a.size() + (int)b.size() - 1) n <= 1;
70
71     // Must fit this NTT configuration. Use another modulus/CRT if
72     // larger needed.
73     if (n > ROOT_PW) return {};
74
75     a.resize(n);
76     b.resize(n);
77
78     ntt(a, false);
79     ntt(b, false);
80
81     for (int i = 0; i < n; i++) {

```

```

81     a[i] = (int)(1LL * a[i] * b[i] % MOD);
82 }
83
84 ntt(a, true);
85 return a;
86 }
87
88 void solve() {
89     // Example usage:
90     // Input two polynomials A (size n) and B (size m), output
91     // coefficients of A*B mod MOD.
92     int n, m;
93     cin >> n >> m;
94     vector<int> a(n), b(m);
95     for (int &x : a) cin >> x;
96     for (int &x : b) cin >> x;
97
98     vector<int> c = multiply_ntt(a, b);
99     if (c.empty()) {
100         // This means size exceeded ROOT_PW with current modulus
101         // configuration.
102         return;
103     }
104
105     for (int i = 0; i < n + m - 1; i++) {
106         cout << c[i] << " ";
107     }
108 }

```

8.14 Graph Paths

```

1 vector<vector<int>> mat_mult(vector<vector<int>> &A, vector<vector<
2 int>> &B, int n, int m, int p){
3     vector<vector<int>> arns(n,vector<int> (p));
4     for(int i=0; i<n; i++){
5         for(int j=0; j<m; j++){
6             for(int k=0; k<p; k++){
7                 arns[i][k] += A[i][j] * B[j][k];
8                 arns[i][k]%=MOD;
9             }
10        }
11    }
12    return arns;
13 }
14
15 vector<vector<int>> mat_pow(vector<vector<int>> &A, int n){
16     int sz = A.size();
17     vector<vector<int>> arns(sz,vector<int> (sz,0));

```

```

18     for(int i=0; i<sz; i++)
19         arns[i][i] = 1;
20     while(n>0){
21         if(n&1) arns = mat_mult(arns,A,sz,sz,sz);
22         A = mat_mult(A,A,sz,sz,sz);
23         n>>=1;
24     }
25     return arns;
26 }
27
28 void solve(){
29     int n, m, k; cin>>n>>m>>k;
30     vector<vector<int>> adj(n,vector<int>(n,0));
31     for(int i=0; i<m; i++){
32         int a,b; cin>>a>>b;
33         a--; b--;
34         adj[a][b]++;
35     }
36     vector<vector<int>> arns = mat_pow(adj,k);
37     cout<<arns[0][n-1]<<endl;
38 }

```

8.15 Graph Paths Min-Plus

```

1 const int MAX = 2e6+1;
2 const int MOD = 1e9+7;
3 const int INF = 4e18;
4
5 int modInv(int num){
6     if(num == 1) return 1;
7     return (MOD - ((MOD/num)*modInv(MOD%num))%MOD+MOD)%MOD;
8 }
9
10 vector<vector<int>> mat_zum(vector<vector<int>> &A, vector<vector<
11 int>> &B, int n, int m, int p){
12     vector<vector<int>> arns(n,vector<int> (p, INF));
13     for(int i=0; i<n; i++){
14         for(int j=0; j<m; j++){
15             for(int k=0; k<p; k++){
16                 if (A[i][j] < INF && B[j][k] < INF) {
17                     arns[i][k] = min(arns[i][k], A[i][j] + B[j][k]);
18                 }
19             }
20         }
21     }
22     return arns;
23 }
24
25 vector<vector<int>> mat_pow(vector<vector<int>> &A, int n){

```

```

26     int sz = A.size();
27     vector<vector<int>> arns(sz,vector<int> (sz,INF));
28     for(int i = 0; i < sz; i++) arns[i][i] = 0;
29     while(n>0){
30         if(n&1) arns = mat_zum(arns,A,sz,sz,sz);
31         A = mat_zum(A,A,sz,sz,sz);
32         n>>=1;
33     }
34     return arns;
35 }
36
37 void solve(){
38     int n, m, k; cin>>n>>m>>k;
39     vector<vector<int>> adj(n,vector<int>(n,INF));
40     for(int i=0; i<m; i++){
41         int a,b,c; cin>>a>>b>>c;
42         a--; b--;
43         adj[a][b]=min(adj[a][b], c);
44     }
45     vector<vector<int>> arns = mat_pow(adj,k);
46     if(arns[0][n-1] == INF) cout<<"-1"<<endl;
47     else cout<<arns[0][n-1]<<endl;
48 }

```

8.16 Log Fibonacci

```

1  const int MAX = 2e6+1;
2  const int MOD = 1e9+7;
3
4  vector<vector<int>> mat_mult(vector<vector<int>> &A, vector<vector<
    int>> &B, int n, int m, int p){
5      vector<vector<int>> arns(n,vector<int> (p));
6      for(int i=0; i<n; i++){
7          for(int j=0; j<m; j++){
8              for(int k=0; k<p; k++){
9                  arns[i][k] += A[i][j] * B[j][k];
10                 arns[i][k]%=MOD;
11             }
12         }
13     }
14
15     return arns;
16 }
17
18 vector<vector<int>> mat_pow(vector<vector<int>> &A, int n){
19     vector<vector<int>> arns = {{1,0},{0,1}};
20     while(n>0){
21         if(n&1) arns = mat_mult(arns,A,2,2,2);
22         A = mat_mult(A,A,2,2,2);
23         n>>=1;

```

```

24     }
25     return arns;
26 }
27
28 void solve(){
29     int n; cin>>n;
30     vector<vector<int>> a = {{1,1},{1,0}};
31     vector<vector<int>> arns = mat_pow(a,n);
32
33     cout<<arns[0][1]<<endl;
34 }

```

8.17 Matrix Determinant

```

1  const double EPS = 1E-9;
2  int n;
3  vector < vector<double> > a (n, vector<double> (n));
4
5  double det = 1;
6  for (int i=0; i<n; ++i) {
7      int k = i;
8      for (int j=i+1; j<n; ++j)
9          if (abs (a[j][i]) > abs (a[k][i]))
10             k = j;
11     if (abs (a[k][i]) < EPS) {
12         det = 0;
13         break;
14     }
15     swap (a[i], a[k]);
16     if (i != k)
17         det = -det;
18     det *= a[i][i];
19     for (int j=i+1; j<n; ++j)
20         a[i][j] /= a[i][i];
21     for (int j=0; j<n; ++j)
22         if (j != i && abs (a[j][i]) > EPS)
23             for (int k=i+1; k<n; ++k)
24                 a[j][k] -= a[i][k] * a[j][i];
25 }
26
27 cout << det;

```

8.18 Matrix Rank

```

1  const double EPS = 1E-9;
2
3  int compute_rank(vector<vector<double>> A) {

```

```

4   int n = A.size();
5   int m = A[0].size();
6
7   int rank = 0;
8   vector<bool> row_selected(n, false);
9   for (int i = 0; i < m; ++i) {
10      int j;
11      for (j = 0; j < n; ++j) {
12         if (!row_selected[j] && abs(A[j][i]) > EPS)
13            break;
14      }
15
16      if (j != n) {
17         ++rank;
18         row_selected[j] = true;
19         for (int p = i + 1; p < m; ++p)
20            A[j][p] /= A[j][i];
21         for (int k = 0; k < n; ++k) {
22            if (k != j && abs(A[k][i]) > EPS) {
23               for (int p = i + 1; p < m; ++p)
24                  A[k][p] -= A[j][p] * A[k][i];
25            }
26         }
27      }
28   }
29   return rank;
30 }

```

```

20      for (int i=0; i<n; ++i)
21         if (i != row) {
22            double c = a[i][col] / a[row][col];
23            for (int j=col; j<=m; ++j)
24               a[i][j] -= a[row][j] * c;
25         }
26      ++row;
27   }
28
29   ans.assign (m, 0);
30   for (int i=0; i<m; ++i)
31      if (where[i] != -1)
32         ans[i] = a[where[i]][m] / a[where[i]][i];
33   for (int i=0; i<n; ++i) {
34      double sum = 0;
35      for (int j=0; j<m; ++j)
36         sum += ans[j] * a[i][j];
37      if (abs (sum - a[i][m]) > EPS)
38         return 0;
39   }
40
41   for (int i=0; i<m; ++i)
42      if (where[i] == -1)
43         return INF;
44   return 1;
45 }

```

8.19 Gauss-Jordan

```

1  const double EPS = 1e-9;
2  const int INF = 2; // it doesn't actually have to be infinity or a
   big number
3
4  int gauss (vector < vector<double> > a, vector<double> & ans) {
5     int n = (int) a.size();
6     int m = (int) a[0].size() - 1;
7
8     vector<int> where (m, -1);
9     for (int col=0, row=0; col<m && row<n; ++col) {
10        int sel = row;
11        for (int i=row; i<n; ++i)
12           if (abs (a[i][col]) > abs (a[sel][col]))
13              sel = i;
14        if (abs (a[sel][col]) < EPS)
15           continue;
16        for (int i=col; i<=m; ++i)
17           swap (a[sel][i], a[row][i]);
18        where[col] = row;
19

```

8.20 Miller-Rabin (Primality Test)

```

1  // Deterministic Miller-Rabin primality test for unsigned 64-bit
   integers.
2  // Purpose: fast primality checks for big n (up to 1e18 / uint64
   range).
3  // Complexity per query: O(k * log n), where k is the number of
   bases (here constant 7).
4  // Typical CP usage: primality checks in factorization, number
   theory tasks, random tests.
5
6  using u64 = uint64_t;
7  using u128 = __uint128_t;
8
9  u64 mod_mul(u64 a, u64 b, u64 mod) {
10     return (u128)a * b % mod;
11 }
12
13 u64 mod_pow(u64 a, u64 e, u64 mod) {
14     u64 r = 1;
15     while (e > 0) {
16         if (e & 1) r = mod_mul(r, a, mod);
17         a = mod_mul(a, a, mod);

```

```

18     e >>= 1;
19 }
20 return r;
21 }
22
23 bool isPrime(u64 n) {
24     if (n < 2) return false;
25
26     // Fast small-prime filter.
27     for (u64 p : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL,
28                 23ULL, 29ULL, 31ULL, 37ULL}) {
29         if (n % p == 0) return n == p;
30     }
31
32     // Write  $n - 1 = d * 2^s$  with  $d$  odd.
33     u64 d = n - 1;
34     int s = 0;
35     while ((d & 1) == 0) {
36         d >>= 1;
37         s++;
38     }
39
40     // Deterministic bases for testing all uint64 values.
41     // Valid for  $n < 2^{64}$ .
42     for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504
43                 ULL, 1795265022ULL}) {
44         if (a % n == 0) continue;
45
46         u64 x = mod_pow(a % n, d, n);
47         if (x == 1 || x == n - 1) continue;
48
49         bool witness = true;
50         for (int r = 1; r < s; r++) {
51             x = mod_mul(x, x, n);
52             if (x == n - 1) {
53                 witness = false;
54                 break;
55             }
56         }
57         if (witness) return false;
58     }
59     return true;
60 }
61
62 // Usage example:
63 // u64 n; cin >> n;
64 // cout << (isPrime(n) ? "YES" : "NO") << '\n';

```

8.21 Linear Sieve

```

1 const int N = 10000000;
2 vector<int> lp(N+1);
3 vector<int> pr;
4
5 for (int i=2; i <= N; ++i) {
6     if (lp[i] == 0) {
7         lp[i] = i;
8         pr.push_back(i);
9     }
10    for (int j = 0; i * pr[j] <= N; ++j) {
11        lp[i * pr[j]] = pr[j];
12        if (pr[j] == lp[i]) {
13            break;
14        }
15    }
16 }

```

8.22 Euler Totient Function

```

1 void phi_1_to_n(int n) {
2     vector<int> phi(n + 1);
3     for (int i = 0; i <= n; i++)
4         phi[i] = i;
5
6     for (int i = 2; i <= n; i++) {
7         if (phi[i] == i) {
8             for (int j = i; j <= n; j += i)
9                 phi[j] -= phi[j] / i;
10        }
11    }
12 }
13
14 void phi_1_to_n(int n) {
15     vector<int> phi(n + 1);
16     phi[0] = 0;
17     phi[1] = 1;
18     for (int i = 2; i <= n; i++)
19         phi[i] = i - 1;
20
21     for (int i = 2; i <= n; i++)
22         for (int j = 2 * i; j <= n; j += i)
23             phi[j] -= phi[i];
24 }

```

8.23 Fast Totient Function

```

1 int prime[MAX_PRIMES];
2 int SPF[MAX_SIZE];
3 int et[MAX_SIZE];
4 int idx=0;
5
6
7 ll bin_pow(ll x,ll k){
8     ll res=1;
9     while(k){
10         if(k&1){
11             res=(res*x)%MOD;
12         }
13         x=(x*x%MOD);
14         k/=2;
15     }
16     return res;
17 }
18
19 ll inv(ll x){ return bin_pow(x,MOD-2);}
20
21 void manipulated_seive(int N)
22 {
23
24     for (int i=2; i<N ; i++)
25     {
26
27         if (SPF[i]==0)
28         {
29             prime[idx++]=i;
30             SPF[i] = i;
31             et[i]=i-1;
32         }
33         else{
34             //et[i]=(SPF[i]-1)*et[i/SPF[i]];
35         }
36         for (int j=0;
37             j < idx &&
38             i*prime[j] < N && prime[j] <= SPF[i];
39             j++)
40         {
41
42             if(prime[j]==SPF[i]){
43                 et[i*prime[j]]=prime[j]*et[i];
44             }
45             else{
46                 et[i*prime[j]]=(prime[j]-1)*et[i];
47             }
48             SPF[i*prime[j]] = prime[j] ;
49         }
50     }
51 }
52 }

```

```

53
54 // driver program to test above function
55 int main()
56 {
57     ios_base::sync_with_stdio(false);
58     cin.tie(NULL);
59     int N,q; cin>>N>>q;
60     //memset(isprime,1,sizeof(isprime));
61     manipulated_seive(N+1);
62
63     ll Q=inv(N-1);
64     for(int i=0;i<q;i++){
65         int x; cin>>x;
66         ll P=(N-x)+et[x]-1;
67         //cout<<et[x]<<endl;
68         cout<<(P*Q)%MOD<<"\n";
69     }
70     return 0;
71 }

```

8.24 Diophantine Equations

```

1 void shift_solution(int & x, int & y, int a, int b, int cnt) {
2     x += cnt * b;
3     y -= cnt * a;
4 }
5
6 int find_all_solutions(int a, int b, int c, int minx, int maxx, int
7 miny, int maxy) {
8     int x, y, g;
9     if (!find_any_solution(a, b, c, x, y, g))
10         return 0;
11     a /= g;
12     b /= g;
13
14     int sign_a = a > 0 ? +1 : -1;
15     int sign_b = b > 0 ? +1 : -1;
16
17     shift_solution(x, y, a, b, (minx - x) / b);
18     if (x < minx)
19         shift_solution(x, y, a, b, sign_b);
20     if (x > maxx)
21         return 0;
22     int lx1 = x;
23
24     shift_solution(x, y, a, b, (maxx - x) / b);
25     if (x > maxx)
26         shift_solution(x, y, a, b, -sign_b);
27     int rx1 = x;

```

```

28 shift_solution(x, y, a, b, -(miny - y) / a);
29 if (y < miny)
30     shift_solution(x, y, a, b, -sign_a);
31 if (y > maxy)
32     return 0;
33 int lx2 = x;
34
35 shift_solution(x, y, a, b, -(maxy - y) / a);
36 if (y > maxy)
37     shift_solution(x, y, a, b, sign_a);
38 int rx2 = x;
39
40 if (lx2 > rx2)
41     swap(lx2, rx2);
42 int lx = max(lx1, lx2);
43 int rx = min(rx1, rx2);
44
45 if (lx > rx)
46     return 0;
47 return (rx - lx) / abs(b) + 1;
48 }

```

8.25 Factorials

```

1 void precompute(ll n = MAXN - 1) {
2     factorial[0] = factorial[1] = 1;
3
4     // Compute factorials
5     for (ll i = 2; i <= n; i++) {
6         factorial[i] = (factorial[i - 1] * i) % MOD;
7     }
8
9     // Compute inverse factorials efficiently
10    inv_factorial[n] = modInv(factorial[n]);
11    for (ll i = n - 1; i >= 0; i--) {
12        inv_factorial[i] =
13            (inv_factorial[i + 1] * (i + 1)) % MOD;
14    }
15 }

```

8.26 Number of Divisors

8.27 Modular Inverse

```

1 ll modInv(ll a, ll mod = MOD) {
2     return power(a, mod - 2, mod);
3 }

```

8.28 Discrete Logarithm

```

1 // Returns minimum x for which a ^ x % m = b % m.
2 int solve(int a, int b, int m) {
3     a %= m, b %= m;
4     int k = 1, add = 0, g;
5     while ((g = gcd(a, m)) > 1) {
6         if (b == k)
7             return add;
8         if (b % g)
9             return -1;
10        b /= g, m /= g, ++add;
11        k = (k * 1ll * a / g) % m;
12    }
13
14    int n = sqrt(m) + 1;
15    int an = 1;
16    for (int i = 0; i < n; ++i)
17        an = (an * 1ll * a) % m;
18
19    unordered_map<int, int> vals;
20    for (int q = 0, cur = b; q <= n; ++q) {
21        vals[cur] = q;
22        cur = (cur * 1ll * a) % m;
23    }
24
25    for (int p = 1, cur = k; p <= n; ++p) {
26        cur = (cur * 1ll * an) % m;
27        if (vals.count(cur)) {
28            int ans = n * p - vals[cur] + add;
29            return ans;
30        }
31    }
32    return -1;
33 }

```

8.29 Segmented Sieve

```

1 vector<char> segmentedSieve(long long L, long long R) {
2     // generate all primes up to sqrt(R)
3     long long lim = sqrt(R);
4     vector<char> mark(lim + 1, false);
5     vector<long long> primes;

```

```

6   for (long long i = 2; i <= lim; ++i) {
7       if (!mark[i]) {
8           primes.emplace_back(i);
9           for (long long j = i * i; j <= lim; j += i)
10              mark[j] = true;
11       }
12   }
13
14   vector<char> isPrime(R - L + 1, true);
15   for (long long i : primes)
16       for (long long j = max(i * i, (L + i - 1) / i * i); j <= R;
17            j += i)
18           isPrime[j - L] = false;
19   if (L == 1)
20       isPrime[0] = false;
21   return isPrime;
22 }

```

```

31         int start_idx = (start + p - 1) / p;
32         int j = max(start_idx, p) * p - start;
33         for (; j < S; j += p)
34             block[j] = false;
35     }
36     if (k == 0)
37         block[0] = block[1] = false;
38     for (int i = 0; i < S && start + i <= n; i++) {
39         if (block[i])
40             result++;
41     }
42 }
43 return result;
44 }

```

8.30 Sieve of Eratosthenes

```

1  int n;
2  vector<bool> is_prime(n+1, true);
3  is_prime[0] = is_prime[1] = false;
4  for (int i = 2; i * i <= n; i++) {
5      if (is_prime[i]) {
6          for (int j = i * i; j <= n; j += i)
7              is_prime[j] = false;
8      }
9  }
10
11 int count_primes(int n) {
12     const int S = 10000;
13
14     vector<int> primes;
15     int nsqrt = sqrt(n);
16     vector<char> is_prime(nsqrt + 2, true);
17     for (int i = 2; i <= nsqrt; i++) {
18         if (is_prime[i]) {
19             primes.push_back(i);
20             for (int j = i * i; j <= nsqrt; j += i)
21                 is_prime[j] = false;
22         }
23     }
24
25     int result = 0;
26     vector<char> block(S);
27     for (int k = 0; k * S <= n; k++) {
28         fill(block.begin(), block.end(), true);
29         int start = k * S;
30         for (int p : primes) {

```

8.31 Simpson Method

```

1  const int N = 1000 * 1000; // number of steps (already multiplied by
2                               2)
3
4  double simpson_integration(double a, double b){
5      double h = (b - a) / N;
6      double s = f(a) + f(b); // a = x_0 and b = x_2n
7      for (int i = 1; i <= N - 1; ++i) { // Refer to final Simpson's
8          // formula
9          double x = a + h * i;
10         s += f(x) * ((i & 1) ? 4 : 2);
11     }
12     s *= h / 3;
13     return s;
14 }

```

8.32 Precompute Pascal Triangle

```

1  int binomialCoeff(int n, int k) {
2      vector<int> dp(k + 1);
3
4      // nC0 is 1
5      dp[0] = 1;
6
7      for (int i = 1; i <= n; i++) {
8
9          // Compute next row of pascal triangle using
10         // the previous row
11         for (int j = min(i, k); j > 0; j--)
12             dp[j] = dp[j] + dp[j - 1];
13     }

```



```

14     return dp[k];
15 }

```

8.33 Sqrt and Inverse Sqrt

```

1 double sqrt_newton(double n) {
2     const double eps = 1E-15;
3     double x = 1;
4     for (;;) {
5         double nx = (x + n / x) / 2;
6         if (abs(x - nx) < eps)
7             break;
8         x = nx;
9     }
10    return x;
11 }
12
13 int isqrt_newton(int n) {
14     int x = 1;
15     bool decreased = false;
16     for (;;) {
17         int nx = (x + n / x) >> 1;
18         if (x == nx || nx > x && decreased)
19             break;
20         decreased = nx < x;
21         x = nx;
22     }
23     return x;
24 }

```

8.34 Sum of Divisors

```

1 long long SumOfDivisors(long long num) {
2     long long total = 1;
3
4     for (int i = 2; (long long)i * i <= num; i++) {
5         if (num % i == 0) {
6             int e = 0;
7             do {
8                 e++;
9                 num /= i;
10            } while (num % i == 0);
11
12            long long sum = 0, pow = 1;
13            do {
14                sum += pow;
15                pow *= i;

```

```

16            } while (e-- > 0);
17            total *= sum;
18        }
19    }
20    if (num > 1) {
21        total *= (1 + num);
22    }
23    return total;
24 }

```

8.35 Ternary Search

```

1 double ternary_search(double l, double r) {
2     double eps = 1e-9;           //set the error limit here
3     while (r - l > eps) {
4         double m1 = l + (r - l) / 3;
5         double m2 = r - (r - l) / 3;
6         double f1 = f(m1);       //evaluates the function at m1
7         double f2 = f(m2);       //evaluates the function at m2
8         if (f1 < f2)
9             l = m1;
10        else
11            r = m2;
12    }
13    return f(l);                  //return the maximum of f(x) in
14    [l, r]

```

8.36 Trivial Factorization

```

1 vector<long long> trial_division2(long long n) {
2     vector<long long> factorization;
3     while (n % 2 == 0) {
4         factorization.push_back(2);
5         n /= 2;
6     }
7     for (long long d = 3; d * d <= n; d += 2) {
8         while (n % d == 0) {
9             factorization.push_back(d);
10            n /= d;
11        }
12    }
13    if (n > 1)
14        factorization.push_back(n);
15    return factorization;
16 }

```

9 String Algorithms

9.1 Aho-Corasick

```

1 // Aho-Corasick Automaton (lowercase a-z)
2 struct AhoCorasick {
3     static const int ALPH = 26;
4
5     struct Node {
6         int next[ALPH];
7         int link;
8         vector<int> out; // pattern ids ending here
9
10        Node() : link(0) {
11            fill(next, next + ALPH, -1);
12        }
13    };
14
15    vector<Node> t;
16    vector<int> patLen;
17
18    AhoCorasick() { t.push_back(Node()); }
19
20    int addString(const string& s) {
21        int v = 0;
22        for (char ch : s) {
23            int c = ch - 'a';
24            if (t[v].next[c] == -1) {
25                t[v].next[c] = (int)t.size();
26                t.push_back(Node());
27            }
28            v = t[v].next[c];
29        }
30        int id = (int)patLen.size();
31        patLen.push_back((int)s.size());
32        t[v].out.push_back(id);
33        return id;
34    }
35
36    void build() {
37        queue<int> q;
38
39        for (int c = 0; c < ALPH; c++) {
40            if (t[0].next[c] == -1) {
41                t[0].next[c] = 0;
42            } else {
43                int u = t[0].next[c];
44                t[u].link = 0;
45                q.push(u);
46            }
47        }

```

```

48
49        while (!q.empty()) {
50            int v = q.front();
51            q.pop();
52
53            for (int c = 0; c < ALPH; c++) {
54                int u = t[v].next[c];
55                if (u == -1) {
56                    t[v].next[c] = t[t[v].link].next[c];
57                    continue;
58                }
59
60                t[u].link = t[t[v].link].next[c];
61                for (int id : t[t[u].link].out) t[u].out.push_back(id);
62                q.push(u);
63            }
64        }
65    }
66
67    // Returns all matches as {ending_index_in_text, pattern_id}
68    vector<pair<int, int>> findAll(const string& text) const {
69        vector<pair<int, int>> ans;
70        int v = 0;
71        for (int i = 0; i < (int)text.size(); i++) {
72            int c = text[i] - 'a';
73            v = t[v].next[c];
74            for (int id : t[v].out) {
75                ans.push_back({i, id});
76            }
77        }
78        return ans;
79    }
80 };
81
82 // Example of use:
83 // Count occurrences of each pattern in text.
84 vector<int> countPatternOccurrences(const string& text, const vector<
85     <string>& patterns) {
86     AhoCorasick ac;
87     for (const string& p : patterns) ac.addString(p);
88     ac.build();
89
90     vector<int> cnt(patterns.size(), 0);
91     auto matches = ac.findAll(text);
92     for (auto [endIdx, id] : matches) {
93         (void)endIdx;
94         cnt[id]++;
95     }
96     return cnt;
97 }

```

9.2 DP on Trie

```

1 class Trie{
2 private:
3     Trie *children[26];
4     bool term_st; // is_end
5
6     Trie* searchPrefix(const string &p){
7         Trie* node = this;
8
9         for(char c:p){
10             int pos = c - 'a';
11             if(!node->children[pos]) return nullptr;
12             node = node->children[pos];
13         }
14         return node;
15     }
16
17 public:
18     Trie() : term_st(false) {
19         for(int i=0; i<26; i++)
20             children[i] = nullptr;
21     }
22
23     void insert(string word) {
24         Trie* node = this;
25         for(char c:word){
26             int pos = c - 'a';
27             if(!node->children[pos]) node->children[pos] = new Trie
28                 ();
29             node = node->children[pos];
30         }
31         node->term_st = true;
32     }
33
34     bool search(string word) {
35         Trie* node = searchPrefix(word);
36         return node != nullptr && node->term_st;
37     }
38
39     bool startsWith(string prefix) {
40         Trie* node = searchPrefix(prefix);
41         return node != nullptr;
42     }
43
44     int arnsDP(string s, int n){
45         vector<int> dp(n+1, 0);
46         // dp[i] -> ways to form the prefix s[0..i-1]
47         dp[0] = 1;
48
49         for(int i = 0; i < n; i++){

```

```

50         Trie *it = this; // Iterate the Trie from the root
51         for(int j = i; j < n; j++){
52             int pos = s[j] - 'a';
53             if(!it->children[pos]) break;
54             it = it->children[pos];
55
56             if(it->term_st)
57                 dp[j+1] = (dp[j+1] + dp[i]) % MOD;
58         }
59     }
60     return dp[n];
61 }
62 }
63 };

```

9.3 KMP

```

1 // Knuth-Morris-Pratt Algorithm
2 vector<int> computeLPS(string pattern) {
3     int m = pattern.length();
4     vector<int> lps(m);
5     int len = 0;
6     lps[0] = 0;
7
8     for (int i = 1; i < m; ) {
9         if (pattern[i] == pattern[len]) {
10             len++;
11             lps[i] = len;
12             i++;
13         } else {
14             if (len != 0) {
15                 len = lps[len - 1];
16             } else {
17                 lps[i] = 0;
18                 i++;
19             }
20         }
21     }
22     return lps;
23 }
24
25 vector<int> KMP(string text, string pattern) {
26     vector<int> lps = computeLPS(pattern);
27     vector<int> matches;
28     int n = text.length();
29     int m = pattern.length();
30     int i = 0, j = 0;
31
32     while (i < n) {
33         if (text[i] == pattern[j]) {

```

```

34         i++;
35         j++;
36     }
37
38     if (j == m) {
39         matches.push_back(i - j);
40         j = lps[j - 1];
41     } else if (i < n && text[i] != pattern[j]) {
42         if (j != 0) {
43             j = lps[j - 1];
44         } else {
45             i++;
46         }
47     }
48 }
49 return matches;
50 }

```

9.4 Manacher Algorithm

```

1 vector<int> manacher(string s) {
2     string t;
3     for(auto c: s) {
4         t += string("#") + c;
5     }
6     auto res = manacher_odd(t + "#");
7     return vector<int>(begin(res) + 1, end(res) - 1);
8 }

```

9.5 Palindromic Tree

```

1 // Palindromic Tree (Eertree)
2 struct PalindromicTree {
3     struct Node {
4         int len;
5         int link;
6         int next[26];
7         int occ; // occurrences as palindrome suffix while adding
8                 // characters
9
10        Node(int l = 0) : len(l), link(0), occ(0) {
11            fill(next, next + 26, -1);
12        }
13    };
14
15    vector<Node> tree;
16    string s;

```

```

16 int suff; // node of the longest suffix-palindrome of current
17           // string
18
19 PalindromicTree() { init(); }
20
21 void init() {
22     tree.clear();
23     tree.push_back(Node(-1)); // root 0: odd length root
24     tree.push_back(Node(0)); // root 1: even length root
25     tree[0].link = 0;
26     tree[1].link = 0;
27     s.clear();
28     suff = 1;
29 }
30
31 int getLink(int v, int pos) const {
32     while (true) {
33         int curLen = tree[v].len;
34         if (pos - 1 - curLen >= 0 && s[pos - 1 - curLen] == s[
35             pos]) {
36             return v;
37         }
38         v = tree[v].link;
39     }
40 }
41
42 void addChar(char ch) {
43     int c = ch - 'a';
44     s.push_back(ch);
45     int pos = (int)s.size() - 1;
46
47     int cur = getLink(suff, pos);
48
49     if (tree[cur].next[c] != -1) {
50         suff = tree[cur].next[c];
51         tree[suff].occ++;
52         return;
53     }
54
55     int newNode = (int)tree.size();
56     tree.push_back(Node(tree[cur].len + 2));
57     tree[cur].next[c] = newNode;
58
59     if (tree[newNode].len == 1) {
60         tree[newNode].link = 1;
61     } else {
62         int linkCand = getLink(tree[cur].link, pos);
63         tree[newNode].link = tree[linkCand].next[c];
64     }
65
66     suff = newNode;
67     tree[suff].occ++;
68 }

```

```

67 void build(const string& str) {
68     init();
69     for (char ch : str) addChar(ch);
70 }
71
72 // Number of distinct palindromic substrings.
73 int distinctPalindromes() const {
74     return max(0, (int)tree.size() - 2);
75 }
76 };
77
78 // Example of use:
79 int countDistinctPalindromes(const string& s) {
80     PalindromicTree pt;
81     pt.build(s);
82     return pt.distinctPalindromes();
83 }
84

```

9.6 Polynomial Hash

```

1 #define ll long long
2 #define all(x) x.begin(), x.end()
3
4 mt19937 rng((uint32_t) chrono::steady_clock::now().time_since_epoch
5   ().count());
6 const int MOD = 1e9 + 9;
7 const ll BASE = uniform_int_distribution<ll>(0, MOD-1)(rng);
8
9 vector<ll> pows = {1};
10 vector<ll> p_hash;
11
12 void hash_str(const string &s){
13     p_hash.assign(s.size()+1, 0);
14
15     while(pows.size() <= s.size()){
16         pows.push_back((pows.back()* BASE)%MOD);
17     }
18
19     p_hash[0] = 0;
20     for(int i=0; i<s.size(); i++){
21         p_hash[i+1] = ((p_hash[i]* BASE)% MOD + s[i])%MOD;
22     }
23 }
24
25 ll get_hash(int l, int r){
26     ll raw = (p_hash[r+1] - (p_hash[l] * pows[r-l+1]));
27     return (raw % MOD + MOD) % MOD;
28 }

```

```

29 inline void solve(){
30     string s; cin>>s;
31     int n = s.size();
32
33     hash_str(s);
34
35     for(int i=0; i<n-1; i++){
36         if(get_hash(0,i) == get_hash(n-i-1, n-1)){
37             cout<<i+1<<" ";
38         }
39     }
40     cout<<endl;
41 }

```

9.7 Rabin-Karp

```

1 // Rabin-Karp (single pattern) with double mod hash.
2 // Finds all occurrences of pattern in text in O(n + m).
3
4 using ll = long long;
5
6 const ll MOD1 = 1000000007LL;
7 const ll MOD2 = 1000000009LL;
8 const ll BASE = 911382323LL;
9
10 vector<int> rabinKarp(const string &text, const string &pattern) {
11     int n = (int)text.size();
12     int m = (int)pattern.size();
13     vector<int> arns;
14
15     if (m == 0 || m > n) return arns;
16
17     vector<ll> p1(n + 1, 1), p2(n + 1, 1);
18     vector<ll> h1(n + 1, 0), h2(n + 1, 0);
19
20     for (int i = 1; i <= n; i++) {
21         p1[i] = (p1[i - 1] * BASE) % MOD1;
22         p2[i] = (p2[i - 1] * BASE) % MOD2;
23
24         h1[i] = (h1[i - 1] * BASE + (unsigned char)text[i - 1] + 1)
25             % MOD1;
26         h2[i] = (h2[i - 1] * BASE + (unsigned char)text[i - 1] + 1)
27             % MOD2;
28     }
29
30     ll ph1 = 0, ph2 = 0;
31     for (char c : pattern) {
32         ph1 = (ph1 * BASE + (unsigned char)c + 1) % MOD1;
33         ph2 = (ph2 * BASE + (unsigned char)c + 1) % MOD2;
34     }
35 }

```

```

33
34     auto getHash = [&](int l, int r) {
35         ll x1 = (h1[r + 1] - (h1[l] * p1[r - l + 1]) % MOD1 + MOD1)
36             % MOD1;
37         ll x2 = (h2[r + 1] - (h2[l] * p2[r - l + 1]) % MOD2 + MOD2)
38             % MOD2;
39         return pair<ll, ll>{x1, x2};
40     };
41     for (int i = 0; i + m - 1 < n; i++) {
42         if (getHash(i, i + m - 1) == pair<ll, ll>{ph1, ph2}) {
43             arns.push_back(i);
44         }
45     }
46     return arns;
47 }
48
49 inline void solve() {
50     string text, pattern;
51     cin >> text >> pattern;
52
53     vector<int> pos = rabinKarp(text, pattern);
54     cout << pos.size() << endl;
55     for (int i : pos) cout << i << " ";
56     cout << endl;
57 }

```

9.8 Suffix Array + LCP

```

1  // Suffix Array + LCP (Kasai)
2  struct SuffixArray {
3      string s;
4      vector<int> sa;    // Suffix array: indices of suffixes in
5                          // sorted order
6      vector<int> rank;  // rank[i] = position of suffix i in sa
7      vector<int> lcp;   // lcp[i] = LCP(sa[i], sa[i-1]), lcp[0] = 0
8
9      SuffixArray() {}
10     explicit SuffixArray(const string& str) { build(str); }
11
12     void build(const string& str) {
13         s = str;
14         int n = (int)s.size();
15         sa.resize(n);
16         rank.resize(n);
17
18         for (int i = 0; i < n; i++) {
19             sa[i] = i;
20             rank[i] = s[i];
21         }
22     }
23 }

```

```

20 }
21
22 vector<int> tmp(n);
23 for (int k = 1; k <= 1) {
24     auto cmp = [&](int i, int j) {
25         if (rank[i] != rank[j]) return rank[i] < rank[j];
26         int ri = (i + k < n) ? rank[i + k] : -1;
27         int rj = (j + k < n) ? rank[j + k] : -1;
28         return ri < rj;
29     };
30
31     sort(sa.begin(), sa.end(), cmp);
32     tmp[sa[0]] = 0;
33
34     for (int i = 1; i < n; i++) {
35         tmp[sa[i]] = tmp[sa[i - 1]] + (cmp(sa[i - 1], sa[i])
36             ? 1 : 0);
37     }
38
39     rank = tmp;
40     if (rank[sa[n - 1]] == n - 1) break;
41 }
42
43 buildLCP();
44
45 void buildLCP() {
46     int n = (int)s.size();
47     lcp.assign(n, 0);
48     for (int i = 0; i < n; i++) rank[sa[i]] = i;
49
50     int k = 0;
51     for (int i = 0; i < n; i++) {
52         if (rank[i] == 0) {
53             k = 0;
54             continue;
55         }
56         int j = sa[rank[i] - 1];
57         while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
58         lcp[rank[i]] = k;
59         if (k) k--;
60     }
61 }
62 };
63
64 // Example of use:
65 // Find the longest repeated substring in s.
66 string longestRepeatedSubstring(const string& s) {
67     if (s.empty()) return "";
68     SuffixArray sa(s);
69
70     int bestLen = 0;

```

```

71     int bestPos = 0;
72     for (int i = 1; i < (int)sa.lcp.size(); i++) {
73         if (sa.lcp[i] > bestLen) {
74             bestLen = sa.lcp[i];
75             bestPos = sa.sa[i];
76         }
77     }
78     return s.substr(bestPos, bestLen);
79 }

```

9.9 Suffix Automaton

```

1  // Suffix Automaton
2  struct SuffixAutomaton {
3      struct State {
4          int link;
5          int len;
6          int next[26];
7
8          State() : link(-1), len(0) {
9              fill(next, next + 26, -1);
10         }
11     };
12
13     vector<State> st;
14     int last;
15
16     SuffixAutomaton() {
17         st.reserve(1);
18         st.push_back(State());
19         last = 0;
20     }
21
22     void extend(char ch) {
23         int c = ch - 'a';
24
25         int cur = (int)st.size();
26         st.push_back(State());
27         st[cur].len = st[last].len + 1;
28
29         int p = last;
30         while (p != -1 && st[p].next[c] == -1) {
31             st[p].next[c] = cur;
32             p = st[p].link;
33         }
34
35         if (p == -1) {
36             st[cur].link = 0;
37         } else {
38             int q = st[p].next[c];

```

```

39         if (st[p].len + 1 == st[q].len) {
40             st[cur].link = q;
41         } else {
42             int clone = (int)st.size();
43             st.push_back(st[q]);
44             st[clone].len = st[p].len + 1;
45
46             while (p != -1 && st[p].next[c] == q) {
47                 st[p].next[c] = clone;
48                 p = st[p].link;
49             }
50
51             st[q].link = st[cur].link = clone;
52         }
53     }
54
55     last = cur;
56 }
57
58 void build(const string& s) {
59     st.clear();
60     st.push_back(State());
61     last = 0;
62     for (char ch : s) extend(ch);
63 }
64
65 // Number of distinct substrings of the built string.
66 long long countDistinctSubstrings() const {
67     long long ans = 0;
68     for (int v = 1; v < (int)st.size(); v++) {
69         ans += st[v].len - st[st[v].link].len;
70     }
71     return ans;
72 }
73
74 bool contains(const string& t) const {
75     int v = 0;
76     for (char ch : t) {
77         int c = ch - 'a';
78         if (st[v].next[c] == -1) return false;
79         v = st[v].next[c];
80     }
81     return true;
82 }
83 };
84
85 // Example of use:
86 long long distinctSubstringsIn(const string& s) {
87     SuffixAutomaton sam;
88     sam.build(s);
89     return sam.countDistinctSubstrings();
90 }

```

9.10 Suffix Automata 2

```

1 struct state {
2     int len, link;
3     map<char, int> next;
4 };
5
6 const int MAXLEN = 100000;
7 state st[MAXLEN * 2];
8 int sz, last;
9
10 void sa_init() {
11     st[0].len = 0;
12     st[0].link = -1;
13     sz++;
14     last = 0;
15 }
16
17 void sa_extend(char c) {
18     int cur = sz++;
19     st[cur].len = st[last].len + 1;
20     int p = last;
21     while (p != -1 && !st[p].next.count(c)) {
22         st[p].next[c] = cur;
23         p = st[p].link;
24     }
25     if (p == -1) {
26         st[cur].link = 0;
27     } else {
28         int q = st[p].next[c];
29         if (st[p].len + 1 == st[q].len) {
30             st[cur].link = q;
31         } else {
32             int clone = sz++;
33             st[clone].len = st[p].len + 1;
34             st[clone].next = st[q].next;
35             st[clone].link = st[q].link;
36             while (p != -1 && st[p].next[c] == q) {
37                 st[p].next[c] = clone;
38                 p = st[p].link;
39             }
40             st[q].link = st[cur].link = clone;
41         }
42     }
43     last = cur;
44 }
45
46 long long get_diff_strings(){
47     long long tot = 0;
48     for(int i = 1; i < sz; i++) {
49         tot += st[i].len - st[st[i].link].len;
50     }

```

```

51     return tot;
52 }
53
54 long long get_tot_len_diff_substings() {
55     long long tot = 0;
56     for(int i = 1; i < sz; i++) {
57         long long shortest = st[st[i].link].len + 1;
58         long long longest = st[i].len;
59
60         long long num_strings = longest - shortest + 1;
61         long long cur = num_strings * (longest + shortest) / 2;
62         tot += cur;
63     }
64     return tot;
65 }
66
67 string lcs (string S, string T) {
68     sa_init();
69     for (int i = 0; i < S.size(); i++)
70         sa_extend(S[i]);
71
72     int v = 0, l = 0, best = 0, bestpos = 0;
73     for (int i = 0; i < T.size(); i++) {
74         while (v && !st[v].next.count(T[i])) {
75             v = st[v].link ;
76             l = st[v].len;
77         }
78         if (st[v].next.count(T[i])) {
79             v = st[v].next[T[i]];
80             l++;
81         }
82         if (l > best) {
83             best = l;
84             bestpos = i;
85         }
86     }
87     return T.substr(bestpos - best + 1, best);
88 }

```

9.11 String Stream

```

1 // Create a stringstream object
2 stringstream ss(fullString);
3 // Extract the strings
4 ss >> a >> colon >> b >> comma >> c;

```

9.12 XOR Hash


```

1 // XOR Hash for strings (very fast, collision-prone compared to
2 // polynomial hash).
3 // Good for randomized checks, grouping, and quick fingerprints.
4 using ull = unsigned long long;
5
6 mt19937_64 rng_xor((uint64_t)chrono::steady_clock::now().
7     time_since_epoch().count());
8 array<ull, 256> H;
9
10 void initXorHash() {
11     for (int i = 0; i < 256; i++) H[i] = rng_xor();
12 }
13
14 // Order-sensitive XOR hash with position mixing.
15 ull xorHash(const string &s) {
16     ull arns = 0;
17     for (int i = 0; i < (int)s.size(); i++) {
18         ull x = H[(unsigned char)s[i]];
19         arns ^= (x + 0x9e3779b97f4a7c15ULL + (arns << 6) + (arns >>
20             2) + (ull)(i + 1));
21     }
22     return arns;
23 }
24
25 inline void solve() {
26     initXorHash();
27
28     int n; cin >> n;
29     unordered_map<ull, int> freq;
30
31     for (int i = 0; i < n; i++) {
32         string s; cin >> s;
33         freq[xorHash(s)]++;
34     }
35
36     // Number of distinct hashes (not guaranteed distinct strings
37     // due to collisions).
38     cout << (int)freq.size() << endl;
39 }

```

9.13 Z Algorithm

```

1 // Z-Algorithm
2 vector<int> z_function(string s) {
3     int n = s.length();
4     vector<int> z(n);
5     int l = 0, r = 0;
6

```

```

7     for (int i = 1; i < n; i++) {
8         if (i <= r) {
9             z[i] = min(r - i + 1, z[i - 1]);
10        }
11        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
12            z[i]++;
13        }
14        if (i + z[i] - 1 > r) {
15            l = i;
16            r = i + z[i] - 1;
17        }
18    }
19    return z;
20 }
21
22 // Pattern matching using Z-algorithm
23 vector<int> patternMatch(string text, string pattern) {
24     string combined = pattern + "$" + text;
25     vector<int> z = z_function(combined);
26     vector<int> matches;
27     int m = pattern.length();
28
29     for (int i = m + 1; i < combined.length(); i++) {
30         if (z[i] == m) {
31             matches.push_back(i - m - 1);
32         }
33     }
34     return matches;
35 }

```

10 Computational Geometry

10.1 Convex Hull

```

1 const ll mod=1e9+7;
2 const ll limit=4e9;
3 //ascii https://elcodigoascii.com.ar/
4
5 int orientation(point a,point b,point c){
6     ll ori=(b.y-c.y)*(b.x-a.x)-(b.y-a.y)*(b.x-c.x);
7     if(ori==0) return 0;
8     if(ori>0) return 1;
9     return 2;
10 }
11
12 void getLastTwo(point &a,point &b,stack<point> &s)
13 {
14     a=s.top();
15     s.pop();

```

```

16     b=s.top();
17     s.pop();
18 }
19
20 void show(point a){
21     cout<<a.x<<" "<<a.y<<endl;
22 }
23
24 //Graham scan
25
26 void solve(){
27     int n; cin>>n;
28     vector<point> puntos(n);
29     FO(i,n){
30         ll a,b; cin>>a>>b;
31         puntos[i]=make_pair(a,b);
32     }
33     sort(all(puntos));
34     //Lower Part
35     stack<point> lower;
36     FO(i,n)
37     {
38         if(lower.size()<2){
39             lower.push(puntos[i]);
40             continue;
41         }
42         point a,b;
43         getLastTwo(a,b,lower);
44         if(orientation(a,b,puntos[i])<2)
45         {
46             lower.push(b);
47             lower.push(a);
48             lower.push(puntos[i]);
49         }
50         else{
51             lower.push(b);
52             i--;
53         }
54     }
55     stack<point> upper;
56     for(int i=n-1;i>=0;i--)
57     {
58         if(upper.size()<2){
59             upper.push(puntos[i]);
60             continue;
61         }
62         point a,b;
63         getLastTwo(a,b,upper);
64         if(orientation(a,b,puntos[i])<2)
65         {
66             upper.push(b);
67             upper.push(a);
68             upper.push(puntos[i]);

```

```

69     }
70     else{
71         upper.push(b);
72         i++;
73     }
74 }
75
76 set<point> res;
77
78 while(!lower.empty()){
79     res.insert(lower.top());
80     lower.pop();
81 }
82 while(!upper.empty()){
83     res.insert(upper.top());
84     upper.pop();
85 }
86 cout<<res.size()<<endl;
87 for(auto c:res) show(c);
88 }

```

10.2 Complex Point

```

1 typedef long long ll;
2 typedef vector<int> vec;
3 const ll mod=1e9+7;
4 const int MAX=2e5+3;
5
6 //ascii https://elcodigoascii.com.ar/
7
8 T norma(pt a){return a.x*a.x+a.y*a.y;}
9
10
11 int sgn(T X){
12     return (T(0)<X)-(T(0)>X);
13 }
14
15
16 pt translate(pt a,pt v){return a+v;}
17 pt scale(pt p,pt c,T factor){return c+(p-c)*factor;}
18 pt rot(pt p,T a){return p*polar(1.0,a);}
19 pt perp(pt p){return pt({-p.y,p.x});}
20 pt linearFunc(pt p,pt q,pt r,pt fp,pt fq){
21     return fp+(r-p)*(fq-fp)/(q-p);
22 }
23
24 T dot(pt v,pt w){ return v.x*w.x+v.y*w.y;}
25 T cross(pt v,pt w){ return v.x*w.y-v.y*w.x;}
26
27 bool isperp(pt a,pt b){return dot(a,b)==0;}

```

```

28 double angle(pt v, pt w) {
29     return acos(clamp(dot(v, w) / abs(v) / abs(w), -1.0, 1.0));
30 }
31
32 T orientation(pt a, pt b, pt c) { return cross(b - a, c - a); }
33
34 bool inAngle(pt a, pt b, pt c, pt p) {
35     if (orientation(a, b, c) < 0) swap(b, c);
36     return sgn(orientation(a, b, p)) * sgn(orientation(a, c, p)) >= 0;
37 }
38
39 bool isconvex(vector<pt> p) {
40     bool hasPos = false, hasNeg = false;
41     for (int i = 0, n = p.size(); i < n; i++) {
42         int o = orientation(p[i], p[(i + 1) % n], p[(i + 2) % n]);
43         if (o > 0) hasPos = true;
44         if (o < 0) hasNeg = true;
45     }
46     return !(hasPos && hasNeg);
47 }
48
49 inline void solve()
50 {
51     pt p{3, -4};
52     p += pt({1, 2});
53     cout << p << endl;
54     cout << norm(p) << endl;
55 }

```

```

20 } else {
21     return 1;
22 }
23 }
24
25 long long trigonometric_sense(Point p, Point p1, Point p2) {
26     return sign(1LL * (p1.x - p.x) * (p2.y - p.y) -
27               1LL * (p2.x - p.x) * (p1.y - p.y));
28 }
29
30 // Check if the rectangles with [P1, P2] and [P3, P4] as diagonals
   intersect
31 bool quick_check(Point p1, Point p2, Point p3, Point p4) {
32     int x1, x2, x3, x4, y1, y2, y3, y4;
33     x1 = min(p1.x, p2.x), x2 = max(p1.x, p2.x);
34     y1 = min(p1.y, p2.y), y2 = max(p1.y, p2.y);
35     x3 = min(p3.x, p4.x), x4 = max(p3.x, p4.x);
36     y3 = min(p3.y, p4.y), y4 = max(p3.y, p4.y);
37     return x2 < x3 || x4 < x1 || y2 < y3 || y4 < y1;
38 }
39
40 bool check(Point p1, Point p2, Point p3, Point p4) {
41     if (trigonometric_sense(p1, p2, p3) * trigonometric_sense(p1, p2,
42     p4) > 0) {
43         return false;
44     }
45     if (trigonometric_sense(p3, p4, p1) * trigonometric_sense(p3, p4,
46     p2) > 0) {
47         return false;
48     }
49     return true;
50 }
51
52 int main() {
53     int test_num;
54     cin >> test_num;
55     for (int t = 0; t < test_num; t++) {
56         Point p1, p2, p3, p4;
57         cin >> p1 >> p2 >> p3 >> p4;
58
59         if (quick_check(p1, p2, p3, p4)) {
60             cout << "NO" << endl;
61         } else if (check(p1, p2, p3, p4)) {
62             cout << "YES" << endl;
63         } else {
64             cout << "NO" << endl;
65         }
66     }
67 }

```

10.3 Line Segment Intersection

```

1 \begin{lstlisting}[style=compactcpp]
2 // BeginCodeSnip{Point Class}
3 struct Point {
4     int x, y;
5     Point(int a = 0, int b = 0) : x(a), y(b) {}
6
7     friend istream &operator>>(istream &in, Point &p) {
8         int x, y;
9         in >> p.x >> p.y;
10        return in;
11    }
12 };
13 // EndCodeSnip
14
15 int sign(long long num) {
16     if (num < 0) {
17         return -1;
18     } else if (num == 0) {
19         return 0;

```

10.4 Minimum Euclidean Distance

```

1  const ll mod=1e9+7;
2  const ll MAX=8e18;
3  const ll limit=1e9+1;
4  //ascii https://elcodigoascii.com.ar/
5
6  ll distance(point a,point b){
7      return (a.X-b.X)*(a.X-b.X)+(a.Y-b.Y)*(a.Y-b.Y);
8  }
9
10 inline void solve()
11 {
12     int n; cin>>n;
13     vector<point> sortedX(n);
14     set<point> sortedY;
15     FO(i,n)
16     {
17         ll x,y; cin>>x>>y;
18         sortedX[i]=make_pair(x,y);
19     }
20     sort(all(sortedX));
21     sortedY.insert(make_pair(sortedX[0].Y,sortedX[0].X));
22     ll d,minSquare=MAX;
23     int j=0;
24     FOR(i,1,n)
25     {
26         d=ceil(sqrt(minSquare));
27         while(sortedX[i].X-sortedX[j].X>d)
28         {
29             sortedY.erase(make_pair(sortedX[j].Y,sortedX[j].X));
30             j++;
31         }
32         auto lower=sortedY.lower_bound(make_pair(sortedX[i].Y-d,0));
33         auto upper=sortedY.upper_bound(make_pair(sortedX[i].Y+d,0));
34         for(auto pointer=lower;pointer!=upper;pointer++)
35         {
36             minSquare=min(minSquare,distance(*pointer,make_pair(
37                 sortedX[i].Y,sortedX[i].X)));
38             sortedY.insert(make_pair(sortedX[i].Y,sortedX[i].X));
39         }
40         cout<<minSquare<<endl;
41     }

```

```

4  }
5
6  ll max_coord(ll a,set<ll>& v)
7  {
8      ll min_val=*v.begin();
9      ll max_val=*std::prev(v.end());
10     return max(abs(a-min_val),abs(a-max_val));
11 }
12
13
14
15 inline void solve()
16 {
17     int n; cin>>n;
18     vector<pt> points(n);
19     set<ll> x_;
20     set<ll> y_;
21     ll res=0;
22     cout<<res<<endl;
23     ll a,b; cin>>a>>b;
24     pt aux=rot({a,b});
25     x_.insert(aux.x);
26     y_.insert(aux.y);
27     FO(i,n-1){
28         cin>>a>>b;
29         aux=rot({a,b});
30         res=max({res,max_coord(aux.x,x_),max_coord(aux.y,y_)});
31         cout<<res<<endl;
32         x_.insert(aux.x);
33         y_.insert(aux.y);
34     }
35 }
36

```

10.5 Minimum Manhattan Distance

```

1  pt rot(pt a)
2  {
3      return {a.x+a.y,a.y-a.x};

```

10.6 Point In Polygon

```

1  \begin{lstlisting}[style=compactcpp]
2  struct point{
3      ll x,y;
4      void show(){
5          cout<<x<<" "<<y<<endl;
6      }
7  };
8
9  int sign(ll a){
10     if(a<0) return -1;
11     if(a==0) return 0;
12     if(a>0) return 1;
13 }
14

```

```

15 int signCP(point p,point p1,point p2)
16 {
17     return sign(1LL*((p1.x-p.x)*(p2.y-p.y)-(p1.y-p.y)*(p2.x-p.x)));
18 }
19
20 bool intersect(point n, point m,point a,point b)
21 {
22     if(signCP(n,a,b)*signCP(m,a,b)>0) return false;
23     if(signCP(a,n,m)*signCP(b,n,m)>0) return false;
24     return true;
25 }
26
27 bool inside(point a,point b,point c){
28     return a.x>=min(b.x,c.x) && a.x<=max(b.x,c.x) && a.y>=min(b.y,c.
29         y)
30     && a.y<=max(b.y,c.y);
31 }
32 inline void solve()
33 {
34     int n,m; cin>>n>>m;
35     vector<point> vertices(n);
36     FO(i,n)
37     {
38         cin>>vertices[i].x>>vertices[i].y;
39     }
40     point query,par,init,first,second;
41     int counter;
42     int resta=0;
43     FO(i,m)
44     {
45         resta=0;
46         counter=0;
47         cin>>query.x>>query.y;
48         par.x=query.x;
49         par.y=-MAX-1;
50         init.x=vertices[0].x;
51         init.y=vertices[0].y;
52         first.x=init.x;
53         first.y=init.y;
54         bool ver=false;
55         for(int j=1;j<=n;j++)
56         {
57             second.x=vertices[j%n].x;
58             second.y=vertices[j%n].y;
59             point AB,u;
60             AB.x=second.x-first.x;
61             AB.y=second.y-first.y;
62             u.x=second.x-query.x;
63             u.y=second.y-query.y;
64             if((AB.x*u.y-AB.y*u.x)==0 && inside(query,first,second))
65                 {

```

```

66                 ver=true;
67                 break;
68             }
69             if(intersect(query,par,first,second) && first.x<=query.x
70                 && query.x<second.x)
71             {
72                 counter++;
73             }
74             if(intersect(query,par,first,second) && second.x<=query.
75                 x && query.x<first.x){
76                 counter++;
77             }
78             first.x=second.x;
79             first.y=second.y;
80         }
81         point AB,u;
82         AB.x=init.x-first.x;
83         AB.y=init.y-first.y;
84         u.x=init.x-query.x;
85         u.y=init.y-query.y;
86         if(!ver){
87             //if(intersect(query,par,first,init)) counter++;
88             if((counter)&1) cout<<"INSIDE";
89             else cout<<"OUTSIDE";
90             cout<<endl;
91         }
92     }
93 }

```

10.7 Point Location Test

```

1 struct point{
2     double x,y;
3 };
4
5 struct Vector{
6     double a=0,b=0;
7     void getVector(point p1,point p2){
8         a=p2.x-p1.x;
9         b=p2.y-p1.y;
10    }
11
12    double getModulo(){
13        return pow(a*a+b*b,0.5);
14    }
15
16    Vector getUnitarian(){
17        Vector x;
18        x.a=a/getModulo();
19        x.b=b/getModulo();

```

```

20     //cout<<x.a<<" "<<x.b<<endl;
21     return x;
22 }
23 };
24 };
25
26 double dotProduct(Vector x,Vector y)
27 {
28     return x.a*y.a+x.b*y.b;
29 }
30
31 double CrossProduct(Vector x,Vector y)
32 {
33     return x.a*y.b-x.b*y.a;
34 }
35
36 inline void solve()
37 {
38
39     point p1,p2,p3,p4;
40     cin>>p1.x>>p1.y>>p2.x>>p2.y>>p3.x>>p3.y;
41     Vector u,v,t;
42     u.getVector(p1,p3);
43     //cout<<u.a<<" "<<u.b<<endl;
44     v.getVector(p2,p3);
45     if(CrossProduct(u,v)>0) cout<<"LEFT"<<endl;
46     else if(CrossProduct(u,v)<0) cout<<"RIGHT"<<endl;
47     else cout<<"TOUCH"<<endl;
48
49 }

```

10.8 Polar Sort

```

1 #define x real()
2 #define y imag()
3
4 typedef long long ll;
5 typedef double T;
6 typedef complex<T> pt;
7 typedef vector<int> vec;
8 const ll mod=1e9+7;
9 const int MAX=2e5+3;
10
11 T cross(pt v,pt w){ return v.x*w.y-v.y*w.x;}
12 T norma(pt a){return a.x*a.x+a.y*a.y;}
13 //ascii https://elcodiogoascii.com.ar/
14
15 bool half(pt p){
16     assert(p.x!=0 || p.y!=0);
17     return p.y>0 || (p.y==0 && p.x<0);

```

```

18 }
19
20 void polarSort(vector<pt> &v){
21     sort(all(v),[](pt v,pt w){
22         return make_tuple(half(v),0)<make_tuple(half(w),cross(v,w));
23     });
24 }
25
26 void polarSortNorm(vector<pt> &v){
27     sort(all(v),[](pt v,pt w){
28         return make_tuple(half(v),0,norma(v))<make_tuple(half(w),
29             cross(v,w),norma(w));
30     });
31 }

```

10.9 Polygon Area

```

1 struct point{
2     ll x,y;
3 };
4
5 ll CrossP(point a,point b){
6     return a.x*b.y-a.y*b.x;
7 }
8
9 inline void solve()
10 {
11     int n; cin>>n;
12     ll res=0;
13     point p1,p2,p3;
14     cin>>p3.x>>p3.y;
15     p1.x=p3.x;
16     p1.y=p3.y;
17     F0(i,n-1)
18     {
19         cin>>p2.x>>p2.y;
20         res+=CrossP(p1,p2);
21         p1.x=p2.x;
22         p1.y=p2.y;
23     }
24     res+=CrossP(p1,p3);
25     cout<<abs(res)<<endl;
26 }

```

11 Dynamic Programming

11.1 Coin Problem

```

1 void solve(){
2     int n, x; cin>>n>>x;
3     vector<ll> coins(n);
4     for(int i=0; i<n; i++){
5         cin>>coins[i];
6     }
7
8     vector<ll> dp(x+1, INT_MAX);
9     dp[0] = 0;
10    for(int i=0; i<=x; i++){
11        for(int j=0; j<n; j++){
12            if(i-coins[j]>=0){
13                dp[i] = min(dp[i], dp[i-coins[j]]+1);
14            }
15        }
16    }
17
18    if(dp[x] != INT_MAX){
19        cout<<dp[x]<<endl;
20    }else{
21        cout<<"-1"<<endl;
22    }
23 }
24
25
26 }
```

11.2 Coin Problem (Min Coins)

```

1 void solve(){
2     int n, x; cin>>n>>x;
3     vector<ll> coins(n);
4     for(int i=0; i<n; i++){
5         cin>>coins[i];
6     }
7
8     vector<ll> dp(x+1, 0);
9     dp[0] = 1;
10    for(int i=0; i<=x; i++){
11        for(int j=0; j<n; j++){
12            if(i-coins[j]>=0){
13                dp[i] = (dp[i] + dp[i-coins[j]]);
14                dp[i]%=MOD;
15            }
16        }
17    }
18 }
```

```

17 }
18
19
20     cout<<dp[x]<<endl;
21 }
```

11.3 Coin Problem (Sorted)

```

1 void solve(){
2     int n, x; cin>>n>>x;
3     vector<ll> coins(n);
4     for(int i=0; i<n; i++){
5         cin>>coins[i];
6     }
7
8     int dp[102][1000005];
9     dp[0][0] = 1;
10    for(int i=1; i<=n; i++){
11        for(int j=0; j<=x; j++){
12            dp[i][j] = dp[i-1][j];
13            int l = j-coins[i-1];
14            if(l>=0){
15                dp[i][j] += (dp[i][l])%MOD;
16                dp[i][j]%=MOD;
17            }
18        }
19    }
20
21
22    cout<<dp[n][x]%MOD<<endl;
23 }
```

11.4 Knapsack

```

1 void solve(){
2     int n,x; cin>>n>>x;
3     vector<int> prices(n+1);
4     vector<int> pages(n+1);
5     for(int i=1; i<=n; i++){cin>>prices[i];}
6     for(int i=1; i<=n; i++){cin>>pages[i];}
7
8     vector<int> dp(x+1); // dp[prices[i]] -> Max pages I can buy
9                          // with price i
10
11    dp[0] = 0;
12    for(int i=1; i<=n; i++){
13        for(int j = x; j>=prices[i]; j--){
```

```

13         dp[j] = max(dp[j], dp[j-prices[i]]+pages[i]);
14     }
15 }
16
17 //for(int i=1; i<=x; i++) cout<<dp[i]<<" ";
18 //cout<<endl;
19
20 cout<<dp[x]<<endl;
21 }

```

11.5 LCS

```

1 vector<int> lcs(vector<int> const& a, vector<int> const &b, int n,
2 int m){
3     vector<vector<int>> dp (n+1, vector<int> (m+1, 0));
4
5     for(int i=1; i<=n; i++){
6         for(int j=1; j<=m; j++){
7             if(a[i-1] == b[j-1]) dp[i][j] = dp[i-1][j-1]+1;
8             else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
9         }
10    }
11
12    int ind = dp[n][m];
13    // Note: if we have characters, we must set ind+1 size
14    // and set the last character as terminal '\0'
15    vector<int> arns(ind);
16    int i = n, j = m;
17
18    while(i>0 and j>0){
19        if(a[i-1] == b[j-1]){
20            arns[ind-1] = a[i-1];
21            i--;
22            j--;
23            ind--;
24        }else if(dp[i-1][j] > dp[i][j-1]) i--;
25        else j--;
26    }
27
28    return arns;
29 }

```

11.6 LIS

```

1 int lis(vector<ll> const& a) {
2     int n = a.size();
3     const int INF = 1e9;

```

```

4     vector<int> d(n+1, INF);
5     d[0] = -INF;
6
7     for (int i = 0; i < n; i++) {
8         int l = upper_bound(d.begin(), d.end(), a[i]) - d.begin();
9         if (d[l-1] < a[i] && a[i] < d[l])
10             d[l] = a[i];
11     }
12
13     int ans = 0;
14     for (int l = 0; l <= n; l++) {
15         if (d[l] < INF)
16             ans = l;
17     }
18     return ans;
19 }

```

11.7 Two Sets

```

1 void solve(){
2     int n; cin>>n;
3
4     int tw = n*(n+1)/2;
5     if(tw%2 == 1){
6         cout<<0<<endl;
7         return;
8     }
9     tw = tw/2;
10
11     vector<int> dp(tw+1, 0);
12
13     dp[0] = 1;
14
15     for(int i=1; i<=n; i++){
16         for(int j=tw; j>=i; j--){
17             dp[j] = (dp[j] + dp[j-i])%MOD;
18         }
19     }
20
21     cout<<dp[tw]<<endl;
22 }
23 }

```

11.8 Bitmask DP

```

1 const int limit=20;
2 vector<pair<ll, ll>> dp((1<<limit));

```



```

3 //ascii https://elcodigoascii.com.ar/
4
5 inline void solve()
6 {
7     int n; cin>>n;
8     ll x; cin>>x;
9     vector<ll> weight(n);
10    dp[0]={1,0};
11    F0(i,n) cin>>weight[i];
12    for(ll i=1;i<(1<n);i++)
13    {
14        dp[i]={n+1,0};
15        for(int j=0;j<n;j++)
16        {
17            if(i&(1<j))
18            {
19                pair<ll,ll> aux=dp[i^(1<j)];
20                if(aux.second+weight[j]<=x){
21                    aux.second+=weight[j];
22                }
23                else{
24                    aux.first++;
25                    aux.second=weight[j];
26                }
27                dp[i]=min(dp[i],aux);
28            }
29        }
30    }
31    cout<<dp[(1<n)-1].first<<endl;
32 }

```

11.9 Digit DP

```

1 const ll mod=1e9+7;
2 ll dp[20][10][2][2];
3 //ascii https://elcodigoascii.com.ar/
4
5 ll mem(int idx,int tight,int prev,int ld,string s)
6 {
7     if(idx==0)
8     {
9         return 1;
10    }
11    if(dp[idx][prev][ld][tight]!=-1){
12        return dp[idx][prev][ld][tight];
13    }
14    int k=9;
15    if(tight) k=s[s.size()-idx]-'0';
16    ll sum=0;
17    for(int i=0;i<=k;i++)

```

```

18 {
19     if(ld || prev!=i)
20     {
21         int new_ld,new_tight;
22         if(i==0 && ld) new_ld=1;
23         else new_ld=0;
24         if(tight && k==i) new_tight=1;
25         else new_tight=0;
26         sum+=mem(idx-1,new_tight,i,new_ld,s);
27     }
28 }
29 dp[idx][prev][ld][tight]=sum;
30 return sum;
31 }

```

11.10 Edit Distance

```

1 int editDistance(string &s1, string &s2) {
2
3     int m = s1.length();
4     int n = s2.length();
5
6     // Create a table to store results of subproblems
7     vector<vector<int>> dp(m + 1, vector<int>(n + 1));
8
9     // Fill the known entries in dp[][]
10    // If one string is empty, then answer
11    // is length of the other string
12    for (int i = 0; i <= m; i++)
13        dp[i][0] = i;
14    for (int j = 0; j <= n; j++)
15        dp[0][j] = j;
16
17    // Fill the rest of dp[][]
18    for (int i = 1; i <= m; i++) {
19        for (int j = 1; j <= n; j++) {
20            if (s1[i - 1] == s2[j - 1])
21                dp[i][j] = dp[i - 1][j - 1];
22            else
23                dp[i][j] = 1 + min({dp[i][j - 1],
24                                    dp[i - 1][j],
25                                    dp[i - 1][j - 1]});
26        }
27    }
28
29    return dp[m][n];
30 }

```

11.11 Retrieve LIS

```

1 vector<int> lis(vector<int> const& a) {
2     int n = a.size();
3     vector<int> d(n, 1), p(n, -1);
4     for (int i = 0; i < n; i++) {
5         for (int j = 0; j < i; j++) {
6             if (a[j] < a[i] && d[i] < d[j] + 1) {
7                 d[i] = d[j] + 1;
8                 p[i] = j;
9             }
10        }
11    }
12
13    int ans = d[0], pos = 0;
14    for (int i = 1; i < n; i++) {
15        if (d[i] > ans) {
16            ans = d[i];
17            pos = i;
18        }
19    }
20
21    vector<int> subseq;
22    while (pos != -1) {
23        subseq.push_back(a[pos]);
24        pos = p[pos];
25    }
26    reverse(subseq.begin(), subseq.end());
27    return subseq;
28 }

```

11.12 SOS DP

```

1 typedef long long ll;
2 typedef vector<int> vec;
3 const ll mod=1e9+7;
4 const int NMAX=2e5+3;
5 const int XMAX=1e6+1e5;
6 const int BT=20;
7
8 //ascii https://elcodigoascii.com.ar/
9
10
11 inline void solve()
12 {
13     int n; cin>>n;
14     vector<int> arr(n);
15     vector<ll> r(XMAX),d(XMAX),zero(XMAX);
16     vector<vector<int>> res(n,vector<int>(3));
17     int maxi=0;

```

```

18     FO(i,n){
19         cin>>arr[i];
20         r[arr[i]]++;
21         d[arr[i]]++;
22     }
23
24     for(int i=0;i<BT;i++){
25         for(int j=0;j<1<<BT;j++){
26             {
27                 if(j&(1<<i)){
28                     r[j]+=r[j^(1<<i)];
29                 }
30             }
31             for(int j=(1<<BT)-1;j>=0;j--){
32                 if(j&(1<<i)){
33                     d[j^(1<<i)]+=d[j];
34                 }
35             }
36         }
37     }
38
39     FO(i,n){
40         cout<<r[arr[i]]<<" "<<d[arr[i]]<<" "<<n-r[(1<<BT)-1-arr[i]]<<endl;
41     }
42 }
43

```

12 Trees

12.1 Diameter

```

1 const int MAX = 2e5+3;
2
3 #define INF LLONG_MAX
4
5 vector<vector<int>> adj(MAX);
6 bool vis[MAX];
7 vector<int> dist(MAX);
8
9 int far;
10 void dfs(int s, int &max_count, int c){
11     vis[s] = true;
12     c++;
13     for(auto u : adj[s]){
14         if(!vis[u]){
15             if(c >= max_count){
16                 max_count = c;
17                 far = u;

```

```

18     }
19     dfs(u,max_count, c);
20 }
21 }
22 }
23
24 void resetVisited(int n){
25     for(int i=1; i<=n; i++){
26         vis[i] = false;
27     }
28
29 void solve(){
30     int n; cin>>n;
31     for(int i=1; i<=n-1; i++){
32         int u,v; cin>>u>>v;
33         adj[u].push_back(v);
34         adj[v].push_back(u);
35     }
36
37     int max_count = INT_MIN;
38     dfs(1,max_count,0);
39     resetVisited(n);
40     dfs(far,max_count,0);
41     if(max_count == INT_MIN){
42         cout<<0<<endl;
43         return;
44     }
45
46     cout<<max_count<<endl;
47 }

```

12.2 Euler Tour

```

1 vector<vector<int>> adj(MAX);
2 vector<int> in(MAX);
3 vector<int> out(MAX);
4 vector<ll> values(MAX);
5 vector<ll> flat(MAX);
6 int timer = 1;
7
8 ll t[2*MAX];
9
10 void build(int n){
11     for(int i=1; i<=n; i++){
12         t[n+i] = flat[i];
13
14         for(int i=n; i>=1; i--){
15             t[i] = t[i<<1] + t[i<<1 | 1];
16         }
17 }

```

```

18
19 void update(int idx, int v, int n){
20     for(t[idx += n] = v; idx > 1; idx >>= 1)
21         t[idx >> 1] = t[idx ^ 1] + t[idx];
22 }
23
24 // Query [l,r)
25
26 ll query(int l, int r, int n){
27     ll arns = 0;
28     for(l+=n, r+=n; l<r; l>>=1, r>>=1){
29         if (l & 1) arns += t[l++];
30         if (r & 1) arns += t[--r];
31     }
32     return arns;
33 }
34
35 void eulerTour(int s, int e){
36     in[s] = timer;
37     flat[timer++] = values[s];
38     for(auto u : adj[s]){
39         if (u == e) continue;
40         eulerTour(u,s);
41     }
42     out[s] = timer;
43 }
44
45 void solve(){
46     int n,q; cin>>n>>q;
47     for(int i=1; i<=n; i++){
48         cin>>values[i];
49     }
50
51     for(int i=1; i<=n-1; i++){
52         int u,v; cin>>u>>v;
53         adj[u].push_back(v);
54         adj[v].push_back(u);
55     }
56
57     eulerTour(1,0);
58     /*
59     for(int i=1; i<=n; i++){
60         cout<<in[i]<<endl;
61     }
62
63     for(int i=1; i<=n; i++){
64         cout<<out[i]<<endl;
65     }
66     */
67
68     build(n);
69
70 }

```

```

71     for(int i=1; i<=q; i++){
72         int a; cin>>a;
73         if(a == 1){
74             int v; int x; cin>>v>>x;
75             update(in[v],x,n);
76         }else{
77             int v; cin>>v;
78             cout<<query(in[v],out[v],n)<<endl;
79         }
80     }
81 }

```

12.3 Tree Isomorphism

```

1 vector<ull> h_map;
2 vector<ull> t2_map;
3 vector<ull> d; // Random variable per depth
4 vector<vector<int>> tree;
5 vector<vector<int>> tree2;
6 int arns = 0;
7
8 ull mix(ull x){
9     x += 0x9e3779b97f4a7c15;
10    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
11    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
12    return x ^ (x >> 31);
13 }
14
15 void init_rng(int depth){
16     uint64_t seed = chrono::steady_clock::now().time_since_epoch().
17     count();
18     mt19937_64 rng(seed);
19     d.resize(depth+1);
20     for(int i=0; i<=depth; i++){
21         d[i] = rng();
22     }
23 }
24
25 void hash_tree1(int u, int prev, int depth){
26     h_map[u] = 1;
27     for(int v : tree[u]){
28         if(v!=prev){
29             hash_tree1(v,u,depth+1);
30             h_map[u] += mix(h_map[v]);
31         }
32     }
33 }
34
35 void hash_tree2(int u, int prev, int depth){
36     t2_map[u] = 1;

```

```

36     for(int v : tree2[u]){
37         if(v!=prev){
38             hash_tree2(v,u,depth+1);
39             t2_map[u] += mix(t2_map[v]);
40         }
41     }
42 }

```

12.4 Max Distance

```

1 #define INF LLONG_MAX
2
3 vector<vector<int>> adj(MAX);
4 vector<int> f(MAX,0); // f(x) = maximum distance from node x to any
5 // node of its subtree
6 vector<int> g(MAX,0); // g(x) = maximum distance from any node to x
7 // outside its subtree
8 vector<int> arns(MAX,0); // arns(x) = max(f(x),g(x))
9
10 void dfsA(int s, int e){
11     for(auto u : adj[s]){
12         if(u == e) continue;
13         dfsA(u,s);
14         if(f[u]+1 > f[s]){ // Maximum distance + 1 of u > Max dist
15             // of predecessor
16             g[s] = f[s]; // Maximum distance from any node outside
17             // of s subtree is f[s]
18             f[s] = f[u] + 1;
19         }else if(f[u]+1>g[s]){
20             g[s] = f[u]+1; // Maximum distance from any node outside
21             // of s subtree is f[u]+1
22         }
23     }
24 }
25
26 void dfsB(int s, int e, int to){
27     arns[s] = max(to, f[s]);
28     for(auto u : adj[s]){
29         if(u == e) continue;
30         if(f[u]+1 == f[s]) dfsB(u,s, max(to,g[s])+1);
31         else dfsB(u,s, arns[s]+1);
32     }
33 }
34
35 void solve(){
36     int n; cin>>n;
37     for(int i=1; i<=n-1; i++){
38         int u,v; cin>>u>>v;
39         adj[u].push_back(v);

```

```

36     adj[v].push_back(u);
37 }
38
39 dfsA(1,0);
40 dfsB(1,0,0);
41
42 for(int i=1;i<=n; i++)
43     cout<<arns[i]<<" ";
44 cout<<endl;
45 }

```

```

38     return up[u][0];
39 }
40
41 void preprocess(int root) {
42     tin.resize(n);
43     tout.resize(n);
44     timer = 0;
45     l = ceil(log2(n));
46     up.assign(n, vector<int>(l + 1));
47     dfs(root, root);
48 }

```

12.5 Binary Lifting

```

1 int n, l;
2 vector<vector<int>> adj;
3
4 int timer;
5 vector<int> tin, tout;
6 vector<vector<int>> up;
7
8 void dfs(int v, int p)
9 {
10     tin[v] = ++timer;
11     up[v][0] = p;
12     for (int i = 1; i <= l; ++i)
13         up[v][i] = up[up[v][i-1]][i-1];
14
15     for (int u : adj[v]) {
16         if (u != p)
17             dfs(u, v);
18     }
19
20     tout[v] = ++timer;
21 }
22
23 bool is_ancestor(int u, int v)
24 {
25     return tin[u] <= tin[v] && tout[u] >= tout[v];
26 }
27
28 int lca(int u, int v)
29 {
30     if (is_ancestor(u, v))
31         return u;
32     if (is_ancestor(v, u))
33         return v;
34     for (int i = l; i >= 0; --i) {
35         if (!is_ancestor(up[u][i], v))
36             u = up[u][i];
37     }

```

12.6 Centroid Decomposition

```

1 bool v[MAX];
2 int dst[MAX];
3 int sz[MAX];
4 vector<int> adj[MAX];
5
6 // Buffer estatico global para recolectar profundidades rapidisimo
7 int depth_buffer[MAX];
8
9 class CentroidDecomposition {
10 private:
11     int k;
12     ll res=0;
13
14     void sztree(int s, int par=-1) {
15         sz[s]=1;
16         for(int ch : adj[s]){
17             if(ch==par || v[ch]) continue;
18             sztree(ch, s);
19             sz[s] += sz[ch];
20         }
21     }
22
23     // Un solo DFS por subarbol: guarda los resultados en el
24     // buffer continuo
25     void get_depths(int s, int par, int depth, int &tail, int &
26         max_d) {
27         if(depth > k) return;
28         depth_buffer[tail++] = depth;
29         max_d = max(max_d, depth);
30         for(int ch : adj[s]){
31             if(ch==par || v[ch]) continue;
32             get_depths(ch, s, depth+1, tail, max_d);
33         }
34     }
35 public:

```

```

35 CentroidDecomposition(int _k): k(_k) {
36     res=0;
37 }
38
39 void addEdge(int a, int b){
40     adj[a].push_back(b);
41     adj[b].push_back(a);
42 }
43
44 // VERSION ITERATIVA: A prueba de Stack Overflow (evita TLE/
45 // RTE en casos borde)
46 int getCentroid(int s, int n) {
47     int par = -1;
48     bool found = true;
49     while(found) {
50         found = false;
51         for(int ch : adj[s]) {
52             if(ch != par && !v[ch] && sz[ch] > n/2) {
53                 par = s;
54                 s = ch;
55                 found = true;
56                 break;
57             }
58         }
59     }
60     return s;
61 }
62
63 void Decompose(int s, int par=-1) {
64     sztree(s, par);
65     s = getCentroid(s, sz[s]); // Encontramos centroide de
66     // forma segura
67     v[s] = true;
68
69     dst[0] = 1;
70     int current_max_depth = 0;
71     sort(all(adj[s]), [](int a, int b){
72         return sz[a] < sz[b];
73     });
74
75     for(int ch : adj[s]){
76         if(v[ch]) continue; // Solo nos importa no cruzar
77         // componentes visitados
78
79         int tail = 0;
80         int sub_max_depth = 0;
81
82         // 1. Recolectar (1 solo DFS)
83         get_depths(ch, s, 1, tail, sub_max_depth);
84
85         // 2. Contar caminos cruzados (bucle cache-friendly)
86         for(int i = 0; i < tail; i++) {
87             res += dst[k - depth_buffer[i]];
88         }
89     }
90 }

```

```

85 }
86
87 // 3. Agregar subarbol actual al arreglo maestro
88 for(int i = 0; i < tail; i++) {
89     dst[depth_buffer[i]]++;
90 }
91
92 current_max_depth = max(current_max_depth,
93     sub_max_depth);
94 }
95
96 // Limpieza ultra rapida usando memset implicito
97 fill(dst, dst + current_max_depth + 1, 0);
98
99 // Recursion para el resto de los componentes
100 for(int ch : adj[s]){
101     if(v[ch]) continue;
102     Decompose(ch, s);
103 }
104
105 ll getRes(){ return res; }
106 };

```

12.7 Heavy-Light Decomposition

```

1 vector<int> parent, depth, heavy, head, pos;
2 int cur_pos;
3
4 int dfs(int v, vector<vector<int>> const& adj) {
5     int size = 1;
6     int max_c_size = 0;
7     for (int c : adj[v]) {
8         if (c != parent[v]) {
9             parent[c] = v, depth[c] = depth[v] + 1;
10            int c_size = dfs(c, adj);
11            size += c_size;
12            if (c_size > max_c_size)
13                max_c_size = c_size, heavy[v] = c;
14        }
15    }
16    return size;
17 }
18
19 void decompose(int v, int h, vector<vector<int>> const& adj) {
20     head[v] = h, pos[v] = cur_pos++;
21     if (heavy[v] != -1)
22         decompose(heavy[v], h, adj);
23     for (int c : adj[v]) {
24         if (c != parent[v] && c != heavy[v])

```

```

25         decompose(c, c, adj);
26     }
27 }
28
29 void init(vector<vector<int>> const& adj) {
30     int n = adj.size();
31     parent = vector<int>(n);
32     depth = vector<int>(n);
33     heavy = vector<int>(n, -1);
34     head = vector<int>(n);
35     pos = vector<int>(n);
36     cur_pos = 0;
37
38     dfs(0, adj);
39     decompose(0, 0, adj);
40 }
41
42 int query(int a, int b) {
43     int res = 0;
44     for (; head[a] != head[b]; b = parent[head[b]]) {
45         if (depth[head[a]] > depth[head[b]])
46             swap(a, b);
47         int cur_heavy_path_max = segment_tree_query(pos[head[b]],
48             pos[b]);
49         res = max(res, cur_heavy_path_max);
50     }
51     if (depth[a] > depth[b])
52         swap(a, b);
53     int last_heavy_path_max = segment_tree_query(pos[a], pos[b]);
54     res = max(res, last_heavy_path_max);
55     return res;
56 }

```

12.8 Lowest Common Ancestor

```

1 struct LCA {
2     vector<int> height, euler, first, segtree;
3     vector<bool> visited;
4     int n;
5
6     LCA(vector<vector<int>> &adj, int root = 0) {
7         n = adj.size();
8         height.resize(n);
9         first.resize(n);
10        euler.reserve(n * 2);
11        visited.assign(n, false);
12        dfs(adj, root);
13        int m = euler.size();
14        segtree.resize(m * 4);
15        build(1, 0, m - 1);

```

```

16    }
17
18    void dfs(vector<vector<int>> &adj, int node, int h = 0) {
19        visited[node] = true;
20        height[node] = h;
21        first[node] = euler.size();
22        euler.push_back(node);
23        for (auto to : adj[node]) {
24            if (!visited[to]) {
25                dfs(adj, to, h + 1);
26                euler.push_back(node);
27            }
28        }
29    }
30
31    void build(int node, int b, int e) {
32        if (b == e) {
33            segtree[node] = euler[b];
34        } else {
35            int mid = (b + e) / 2;
36            build(node << 1, b, mid);
37            build(node << 1 | 1, mid + 1, e);
38            int l = segtree[node << 1], r = segtree[node << 1 | 1];
39            segtree[node] = (height[l] < height[r]) ? l : r;
40        }
41    }
42
43    int query(int node, int b, int e, int L, int R) {
44        if (b > R || e < L)
45            return -1;
46        if (b >= L && e <= R)
47            return segtree[node];
48        int mid = (b + e) >> 1;
49
50        int left = query(node << 1, b, mid, L, R);
51        int right = query(node << 1 | 1, mid + 1, e, L, R);
52        if (left == -1) return right;
53        if (right == -1) return left;
54        return height[left] < height[right] ? left : right;
55    }
56
57    int lca(int u, int v) {
58        int left = first[u], right = first[v];
59        if (left > right)
60            swap(left, right);
61        return query(1, 0, euler.size() - 1, left, right);
62    }
63 };

```

12.9 Successor DP

```

1  const int m=30;
2  int sucesorM[limit][m];
3  //ascii https://elcodigoascii.com.ar/
4
5  inline void solve()
6  {
7      int n,q; cin>>n>>q;
8      int res,aux;
9      ll k;
10     lFOR(i,n){
11         cin>>sucesorM[i][0];
12     }
13     FOR(j,1,m)
14     {
15         lFOR(i,n)
16         {
17             sucesorM[i][j]=sucesorM[sucesorM[i][j-1]][j-1];
18         }
19     }
20     FO(i,q)
21     {
22         cin>>res>>k;
23         aux=0;
24         while(k)
25         {
26             if(k%2){
27                 res=sucesorM[res][aux];
28             }
29             k/=2;
30             aux++;
31         }
32         cout<<res<<endl;
33     }
34 }

```

13 Miscellaneous

13.1 2D Prefix Sum

```

1  void solve(){
2      int n,q; cin>>n>>q;
3      char forest[n+1][n+1];
4
5      for(int i=1; i<=n; i++){
6          for(int j=1; j<=n; j++){
7              cin>>forest[i][j];
8          }
9      }

```

```

10
11     vector<vector<int>> dp(n+1, vector<int> (n+1,0));
12
13
14     for(int i=1; i<=n; i++){
15         for(int j=1; j<=n; j++){
16             dp[i][j] = dp[i][j-1] + dp[i-1][j] - dp[i-1][j-1];
17             if(forest[i][j] == '*') dp[i][j]++;
18         }
19     }
20
21
22     for(int i=0; i<q; i++){
23         int a,b,c,d;
24         cin>>a>>b>>c>>d;
25         cout<<dp[c][d] - dp[a-1][d] - dp[c][b-1] + dp[a-1][b-1]<<
26             endl;
27     }
28
29 }

```

13.2 Kadane

```

1  ll kadane(vector<int> &a, int n){
2      ll arns = a[0], zum = 0;
3
4      for (int i = 0; i < n; i++){
5          zum += a[i];
6          arns = max(arns, zum);
7          zum = max(zum, 0LL);
8      }
9
10     return arns;
11 }

```

13.3 Index Compression

```

1  sort(all(a));
2
3  auto compression = [&](int x) {
4      return (int)(lower_bound(all(a), x) - a.begin());
5  };

```

13.4 Berlekamp-Massey


```

1 vector<T> berlekampMassey(const vector<T> &s) {
2     vector<T> c; // the linear recurrence sequence we are
3         building
4     vector<T> oldC; // the best previous version of c to use (the
5         one with the rightmost left endpoint)
6     int f = -1; // the index at which the best previous version
7         of c failed on
8     for (int i=0; i<(int)s.size(); i++) {
9         // evaluate c(i)
10        // delta = s_i - \sum_{j=1}^n c_j s_{i-j}
11        // if delta == 0, c(i) is correct
12        T delta = s[i];
13        for (int j=1; j<=(int)c.size(); j++)
14            delta -= c[j-1] * s[i-j]; // c_j is one-indexed, so we
15                actually need index j - 1 in the code
16        if (delta == 0)
17            continue; // c(i) is correct, keep going
18        // now at this point, delta != 0, so we need to adjust it
19        if (f == -1) {
20            // this is the first time we're updating c
21            // s_i was the first non-zero element we encountered
22            // we make c of length i + 1 so that s_i is part of the
23                base case
24            c.resize(i + 1);
25            mt19937 rng(chrono::steady_clock::now().time_since_epoch
26                ().count());
27            for (T &x : c)
28                x = rng(); // just to prove that the initial values
29                    don't matter in the first step, I will set to
30                    random values
31            f = i;
32        } else {
33            // we need to use a previous version of c to improve on
34                this one
35            // apply the 5 steps to build d
36            // 1. set d equal to our chosen sequence
37            vector<T> d = oldC;
38            // 2. multiply the sequence by -1
39            for (T &x : d)
40                x = -x;
41            // 3. insert a 1 on the left
42            d.insert(d.begin(), 1);
43            // 4. multiply the sequence by delta / d(f + 1)
44            T df1 = 0; // d(f + 1)
45            for (int j=1; j<=(int)d.size(); j++)
46                df1 += d[j-1] * s[f+1-j];
47            assert(df1 != 0);
48            T coef = delta / df1; // storing this in outer
49                variable so it's O(n^2) instead of O(n^2 log MOD)
50            for (T &x : d)
51                x *= coef;
52            // 5. insert i - f - 1 zeros on the left

```

```

43        vector<T> zeros(i - f - 1);
44        zeros.insert(zeros.end(), d.begin(), d.end());
45        d = zeros;
46        // now we have our new recurrence: c + d
47        vector<T> temp = c; // save the last version of c
48            because it might have a better left endpoint
49        c.resize(max(c.size(), d.size()));
50        for (int j=0; j<(int)d.size(); j++)
51            c[j] += d[j];
52        // finally, let's consider updating oldC
53        if (i - (int) temp.size() > f - (int) oldC.size()) {
54            // better left endpoint, let's update!
55            oldC = temp;
56            f = i;
57        }
58    }
59    return c;
60}

```

13.5 Moore's Voting

```

1 int majorityElement(vector<int>& nums) {
2     int vote = 0, r = 0;
3     for(int i=0; i<nums.size(); i++){
4         if(nums[i] == nums[r]){
5             vote++;
6         }
7         else
8             vote--;
9         if(vote == 0){
10            r = i;
11            vote = 1;
12        }
13    }
14
15    int cnt = 0;
16    int goal = (nums.size())/2;
17    for(int i=0; i<nums.size(); i++){
18        if(nums[i] == nums[r]){
19            cnt++;
20            if(cnt > goal){
21                break;
22            }
23        }
24    }
25    return nums[r];
26}

```

13.6 Compiler Optimizations

```
1 // C++ compiler optimizations for competitive programming.
2 // Use in submission files when heavy loops / math / bit operations
  are bottlenecks.
3 // Avoid for interactive/debug-heavy tasks because aggressive
  optimizations can make debugging harder.
4
5 #pragma GCC optimize("O3,unroll-loops")
6 #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
7
8 // Optional fast I/O toggle:
9 // ios::sync_with_stdio(false);
10 // cin.tie(nullptr);
11
12 // Notes:
13 // - O3 + unroll-loops: usually safe and useful in most non-
  interactive problems.
14 // - target(...): can speed up bitset/popcount-heavy code, but may
  fail on judges without these CPU features.
15 // - If unsure about judge hardware, remove target(...) and keep O3
  only.
```